



惠州学院

HUIZHOU UNIVERSITY

毕 业 论 文 （设 计）

中文题目：融合 3D 定制和 AI 试穿的服装创意分享
平台的设计与实现

英文题目：Design of a Fashion Platform with 3D
Customization and AI Virtual Fitting

姓 名 罗泽鑫

学 号 2214100741

专业班级 22 软件工程 2 班

指导教师 刘利 副教授

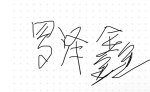
提交日期 2026 年 3 月 22 日

教务部制

诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议，除论文（设计）中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体均已在论文（设计）中以明确方式标明。本声明的法律结果由本人承担。

本人签名：



日期：2026 年 3 月 22 日

摘 要

随着数字时尚和个性化表达需求的增长，传统的服装设计模式存在显著的“高门槛”与“体验断层”问题。一方面，普通用户虽然拥有丰富的穿搭创意，但难以将脑海中的创意具像化；另一方面，现有的在线服装设计工具大多停留在平面的、抽象的版型拼接，用户无法直观感知服装设计完成后的真实穿着效果与环境氛围。这种“所想”与“所见”之间的鸿沟，极大的抑制了用户的创作积极性与分享欲望。

本文研究基于 Three.js，设计了一套融合 3D 定制和 AI 试穿的服装创意分享平台及对应的后台管理系统。用户可以在平台上实时预览自己的设计，与其他用户分享自己的创作，同时可以通过 AI 生图功能，了解定制服装的真实穿着效果。

关键词： Three.js 框架；3D 定制；AI 试穿；服装创意分享

Abstract

With the rapid growth of digital fashion and the demand for personalized expression, traditional apparel design workflows face two prominent issues: a high entry barrier and a discontinuity in user experience. On the one hand, many ordinary users have rich outfit ideas but struggle to materialize them into concrete designs. On the other hand, most existing online design tools remain limited to 2D and abstract pattern editing, preventing users from intuitively perceiving the actual wearing effect and the ambient visual style after the design is completed. This gap between what users imagine and what they can visualize greatly reduces their motivation to create and share.

This thesis designs and implements a fashion creativity sharing platform and a corresponding administrative management system that integrate 3D customization and AI-based virtual fitting, using Three.js as the core technology. Users can preview their designs in real time, share creations with others, and leverage AI image generation to better understand the realistic wearing effect of customized garments.

Key words: Three.js; 3D customization; AI virtual fitting; fashion creativity sharing

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 主要研究内容	2
1.3 主要解决的问题	2
2 开发框架及技术	4
2.1 前端开发技术	4
2.1.1 Vue3	4
2.1.2 Nuxt3	4
2.1.3 Three.js	4
2.2 后端开发技术	4
2.2.1 Go 语言	4
2.2.2 Gin 框架	5
2.2.3 Redis	5
3 需求分析	6
3.1 需求分析概述	6
3.2 系统用户及用例分析	6
3.2.1 普通用户	6
3.2.2 系统管理员	8
3.3 主要功能需求	10
3.3.1 用户管理模块	10
3.3.2 3D 服装定制模块	10
3.3.3 AI 虚拟试穿模块	10
3.3.4 创意广场模块	10
3.3.5 作品管理模块	10
3.3.6 后台管理模块	10
3.4 系统数据需求	10
3.4.1 数据流图	10
3.4.2 数据项	11

4 系统设计	14
4.1 系统总体设计	14
4.1.1 系统架构设计	14
4.1.2 系统功能结构设计	14
4.2 系统功能模块设计	15
4.2.1 用户与权限模块	15
4.2.2 3D 服装定制模块	17
4.2.3 AI 虚拟试穿模块	20
4.2.4 作品管理模块	21
4.2.5 创意广场与社区互动模块	23
4.2.6 后台管理模块	26
4.3 系统数据库设计	27
4.3.1 数据库实体联系分析	27
4.3.2 数据库核心数据表	29
5 系统实现	38
5.1 系统开发环境	38
5.2 系统功能模块实现	38
5.2.1 用户管理模块实现	39
5.2.2 3D 服装定制模块实现	40
5.2.3 AI 虚拟试穿模块实现	41
5.2.4 创意广场模块实现	42
5.2.5 作品管理模块实现	44
5.2.6 后台管理模块实现	45
6 系统测试	48
6.1 功能测试	48
6.1.1 用户模块测试	48
6.1.2 3D 服装定制模块测试	51
6.1.3 AI 虚拟试穿模块测试	55
6.1.4 创意广场模块测试	57
6.1.5 作品管理模块测试	61
6.1.6 后台管理模块测试	63

7 总结与展望	68
7.1 总结	68
7.2 展望	69
参考文献	70
致谢	72
附录	73

1 绪论

1.1 研究背景与意义

近年来，数字化消费与社交媒体推动的“穿搭内容化”趋势，使服装创意从专业工作室逐步走向大众参与。用户对快速试错、即时预览与个性化表达的期待不断提升，但传统服装设计流程往往依赖专业软件、打版及样衣验证，学习成本高、制作周期长，难以支撑普通用户的轻量化创作。同时，现有线上定制服务多以二维参数配置或素材拼贴为主，缺少对版型、材质与光照场景的三维可视化反馈，也缺乏从设计、展示到交流的连续体验，导致创意难以沉淀与传播。

与此同时，Web3D 技术与人工智能生成内容（AIGC）技术的快速发展为解决上述问题提供了新的技术路径^[1]。图像虚拟试衣技术已从早期基于图像变换的 VITON^[2]、CP-VTON^[3] 等方案发展到高分辨率生成框架 VITON-HD^[4] 等方法，近年虚拟试衣综述^[5] 对上述进展进行了系统梳理。深度学习^[6] 尤其是生成对抗网络^[7] 与扩散模型的兴起，极大提升了合成图像的逼真程度。Web3D 技术能够实现浏览器端的实时三维渲染，为用户提供沉浸式的交互体验；而基于深度学习的 AI 图像生成技术，使得从 3D 模型引导生成高保真真人试穿图像成为可能^[8]。将这两项技术有机结合，构建融合 3D 定制与 AI 试穿的服装创意分享平台，不仅能够降低服装设计的专业门槛，还能打通从创意构思到效果呈现的完整链路，具有重要的理论价值和实践意义。

本课题的研究意义主要体现在以下三个方面：

- (1) 降低服装设计与创意可视化的门槛。本项目通过 Web3D 技术提供直观的服装定制工具，让用户无需专业背景即可进行款式、材质、图案的个性化编辑；结合 AI 生图技术，解决传统设计工具无法实时生成高保真试穿效果图的痛点，真正实现“所见即所得”的创作体验，使普通大众也能参与服装创意设计。
- (2) 推动数字时尚与创意分享生态建设。通过构建创意广场与分享社区，鼓励用户将生成的 AI 试穿图和 3D 设计方案进行展示与交流，不仅能够激发大众的审美潜能，也为数字服装、虚拟穿搭等新兴领域提供了一个可落地的实践平台，促进数字时尚文化的传播与发展。
- (3) 探索 AIGC 技术在垂直领域的应用落地。相比于通用的 AI 绘图应用，本课题专注于服装试穿这一特定场景，深入研究如何通过 3D 模型引导 AI 生成的准确性，对于研究 AIGC 技术在具体行业场景中的应用优化具有重要的工

程参考价值。

1.2 主要研究内容

本课题旨在设计并实现一个融合 3D 定制和 AI 试穿的服装创意分享平台，主要研究内容包括以下六个方面：

- (1) **多角色用户体系构建**设计并实现普通用户与系统管理员的双重角色权限体系。普通用户拥有创意设计、作品管理、社区互动等权限；系统管理员负责平台内容审核、基础数据维护和用户管理等功能，保障平台的良性运营。
- (2) **3D 服装可视化定制模块**基于 Web3D 技术（主要采用 Three.js 框架）实现服装模型的实时渲染展示，支持用户对服装款式、颜色、材质、图案等参数进行个性化编辑与定制。研究 3D 模型在浏览器端的高效加载、实时渲染和交互响应技术，确保用户获得流畅的定制体验。
- (3) **AI 虚拟试穿生成模块**接入 AI 图像生成服务，实现“3D 模型转真人试穿图”的核心功能^[9]。将用户定制的 3D 服装模型作为条件输入，引导 AI 生成高保真的真人上身效果预览图像，填补 3D 模型与真实穿搭效果之间的视觉鸿沟。
- (4) **创意广场与社区互动模块**构建服装创意分享广场，支持用户发布自己的设计作品，并实现点赞、收藏、评论等社区互动功能。促进用户间的灵感交流与创意碰撞，打破单机创作的孤岛效应。
- (5) **数字资产与作品管理模块**实现用户对个人创作作品（从 3D 草稿到 AI 成图）的完整生命周期管理，支持作品的保存、修改、版本控制和公开展示。研究数字服装资产的存储结构和元数据标准，确保创作成果的完整留存。
- (6) **系统后台管理模块**实现管理员对用户数据、创意作品内容、3D 基础模型库的统一维护与管理。提供可视化的数据管理界面，支持内容的审核、模型的管理和用户信息配置等功能。

1.3 主要解决的问题

针对现有服装设计与分享领域存在的痛点，本课题主要解决以下四个核心问题：

- (1) 传统服装设计门槛高、体验抽象的问题。现有服装设计软件通常需要专业的版型知识和操作技能，普通用户难以快速上手。本平台通过 3D 可视化配置工具，将复杂的服装设计简化为直观的参数选择和实时预览，让用户能够轻松实现创意的具象化，无需专业背景即可完成个性化服装定制。

- (2) 缺乏高保真虚拟试穿体验的问题。传统在线设计工具仅提供平面版型或简单的 3D 模型展示，用户无法直观感受服装的真实穿着效果。本平台利用 AI 生图技术，将用户定制的 3D 服装模型转换为逼真的真人试穿图像，填补 3D 模型与真实穿搭效果之间的视觉鸿沟，让用户能够从视觉上沉浸式体验自己的设计作品。
- (3) 个性化创意展示与交流渠道缺失的问题。现有设计工具多为单机或封闭系统，用户的设计创意难以有效分享和传播，形成创作孤岛。本平台构建垂直领域的创意广场，提供作品发布、浏览、互动等完整社交功能，打破用户单机创作的局限，通过社区互动激发用户的创作积极性与审美探索。
- (4) 创作成果闭环不完整的问题。传统设计流程中，从灵感构思到最终作品展示存在多个断点，创意难以形成完整的数字资产进行留存与展示。本平台实现从“灵感构思”、“3D 定制”、“AI 试穿”到“作品发布”的完整数字创作链路，确保用户的每一次创意都能得到完整的数字化呈现和长期保存，形成闭环的创作体验。

2 开发框架及技术

2.1 前端开发技术

2.1.1 Vue3

Vue3 是 Vue.js 框架的最新主要版本，于 2020 年正式发布^[10]。相较于 Vue2，Vue3 引入了组合式 API（Composition API），通过 `setup()` 函数或 `<script setup>` 语法允许开发者按逻辑关注点组织代码，提升了复杂组件的可维护性。其响应式系统基于 Proxy 实现，解决了 Vue2 中数组和对象属性操作的响应式局限，同时带来更好的性能表现。Vue3 通过编译时优化将渲染性能提升 1.3~2 倍，内存占用减少约 50%，为大型前端应用提供了更高效的基础框架。

2.1.2 Nuxt3

Nuxt3 是基于 Vue3 的全栈框架，2022 年正式发布。该框架支持服务端渲染（SSR）、静态生成（SSG）和客户端渲染（CSR）多种渲染模式，开发者可根据页面特性灵活选择。Nuxt3 采用基于文件系统的自动路由机制，简化了多页面应用的开发流程。其内置的 Nitro 服务端引擎支持多种运行时环境，模块生态丰富，并提供完善的 TypeScript 支持，显著提升了 Vue 应用的工程化水平和开发效率。

2.1.3 Three.js

Three.js 是目前最流行的 Web3D JavaScript 库，基于 WebGL 封装，提供了创建和显示三维计算机图形的简化 API^[11]。该库采用场景图（Scene Graph）架构，核心概念包括 Scene（场景）、Camera（相机）、Renderer（渲染器）、Mesh（网格）和 Light（光源）。Three.js 支持多种 3D 模型格式加载、材质编辑、光照设置和后期处理效果，并提供了 OrbitControls 等交互控制组件，使开发者能够在浏览器中构建复杂的 3D 交互应用^[12]，无需深入了解底层的 WebGL 细节。在服装可视化场景中，参数化人体模型（如 SMPL^[13]）为服装与人体的三维绑定提供了形状表示基础。

2.2 后端开发技术

2.2.1 Go 语言

Go（Golang）是 Google 开发的开源编程语言，以简洁的语法、出色的并发性能和高效的编译速度著称。该语言原生支持轻量级线程 Goroutine 和通道

Channel，通过 `go` 关键字即可启动并发执行，配合 `select` 语句实现高效的并发调度。Go 编译生成单一静态二进制文件，无运行时依赖，便于部署和运维。其标准库完善，第三方生态成熟，在云原生和微服务领域有广泛应用。

2.2.2 Gin 框架

Gin 是 Go 语言中高性能的 Web 框架，基于 `httprouter` 实现高效的路由匹配^[14]，遵循 REST 架构风格^[15]。该框架采用中间件机制处理 HTTP 请求流程，内置 `Logger`、`Recovery` 等常用中间件，并支持自定义中间件扩展。Gin 提供了强大的数据绑定与验证功能，支持 JSON、XML、Form 等多种数据格式的自动解析和参数校验。其路由性能较 Go 标准库提升约 40 倍，是构建 RESTful API 服务的常用选择。

2.2.3 Redis

Redis 是开源的高性能键值存储数据库^[16]，支持 `String`、`Hash`、`List`、`Set`、`Sorted Set` 等多种数据结构。该数据库基于内存运行，读写性能优异，常用于缓存、会话存储、消息队列和实时统计等场景。Redis 支持数据持久化、主从复制和高可用集群部署，其发布订阅模式和 `Lua` 脚本扩展能力，使其能够灵活应对各类高性能数据存储需求。

3 需求分析

3.1 需求分析概述

本平台面向普通用户和系统管理员两类角色，旨在构建一个融合 3D 服装定制与 AI 虚拟试穿的创意分享社区。普通用户的核心诉求在于低门槛的服装设计工具、高保真的试穿效果预览以及便捷的创意分享渠道；系统管理员则需要完善的平台运营工具，确保内容质量和社区秩序。

通过竞品调研和用户访谈，梳理出以下核心需求：在创作层面，用户需要直观的 3D 可视化编辑工具，支持款式、颜色、材质、图案等参数的个性化调整，并能通过 AI 技术生成真实的上身效果图；在社交层面，用户希望作品能够被展示、发现和互动，形成创作-分享-反馈的良性循环；在管理层面，需要建立内容审核机制、基础资源库维护和用户权限管理体系。

基于以上分析，本平台定位为“工具 + 社区”型应用，既要保证 3D 定制和 AI 试穿的功能完整性，也要注重社交体验的流畅性，同时确保系统具备良好的可扩展性和稳定性。

3.2 系统用户及用例分析

本平台涉及两类用户角色：普通用户和系统管理员。普通用户是平台的核心使用者，进行服装设计、AI 试穿和社区互动；系统管理员负责模型管理和内容监管。

3.2.1 普通用户

普通用户是平台的主要服务对象，涵盖服装设计爱好者、时尚博主、支持模型定制服装的商家，或想表达创意的一般消费者。该类用户无需专业设计背景，通过平台提供的可视化工具完成服装创意表达。

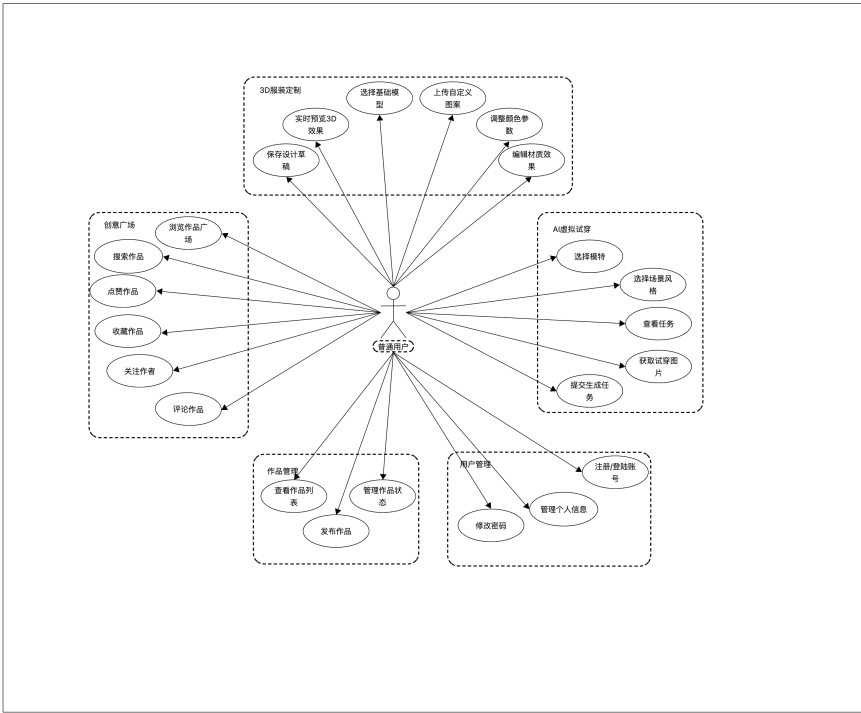


图 3.1 普通用户用例图

选择普通用户中的关键用例进行用例规约编写，如表 3.1 所示。

表 3.1 普通用户关键用例规约

用例编号	用例名称	前置条件	基本流程	扩展流程	后置条件
UC01	用户注册/登录	用户未登录	1. 输入账号密码 2. 系统校验 3. 登录成功进入系统	1. 账号或密码错误 2. 提示重新输入	用户成功登录系统
UC02	浏览创意广场	用户已登录	1. 进入创意广场 2. 系统加载作品列表 3. 用户浏览作品	1. 数据加载失败 2. 提示刷新	成功展示作品列表
UC03	3D 服装定制	已选择服装模型	1. 选择模型 2. 调整颜色/材质/图案 3. 实时预览效果	1. 模型加载失败 2. 参数异常	生成个性化 3D 设计
UC04	AI 虚拟试穿	已完成 3D 设计	1. 点击生成试穿 2. 调用 AI 接口 3. 返回试穿图	1. AI 生成失败 2. 网络异常	成功生成试穿图
UC05	发布作品	已生成设计作品	1. 填写作品信息 2. 提交发布 3. 系统保存作品	1. 提交失败 2. 信息不完整	作品成功发布
UC06	点赞/评论/收藏	浏览作品中	1. 点击点赞/评论/收藏 2. 系统记录行为	1. 操作失败 2. 网络异常	互动数据更新
UC07	个人作品管理	用户已登录	1. 查看个人作品 2. 编辑或删除作品	1. 操作失败 2. 权限异常	作品被成功管理

3.2.2 系统管理员

系统管理员负责平台的日常运营和维护，包括用户管理、内容审核、作品管理和模型库维护。

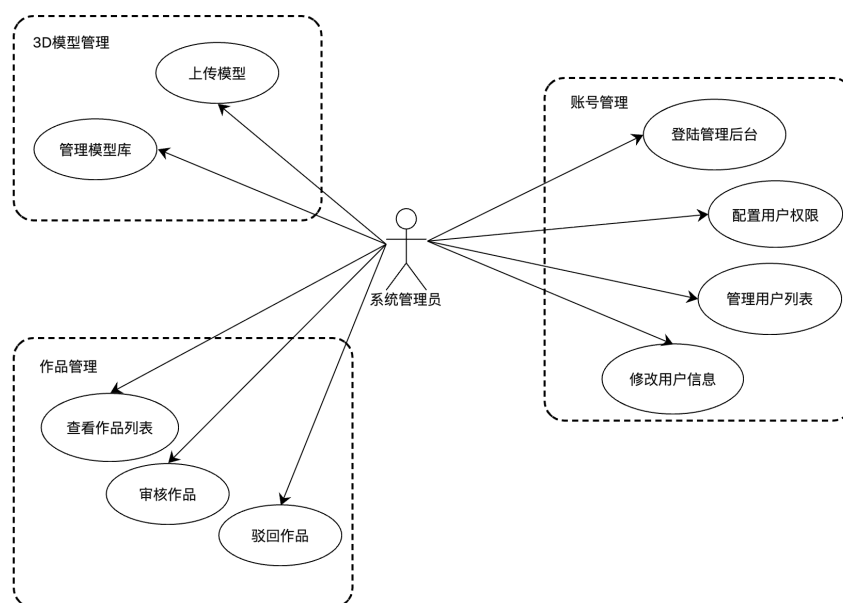


图 3.2 系统管理员用例图

选择系统管理员中的关键用例进行用例规约编写，如表 3.2 所示。

表 3.2 系统管理员关键用例规约

用例编号	用例名称	前置条件	基本流程	扩展流程	后置条件
AC01	管理员登录	管理员账号存在	1. 输入账号密码 2. 系统校验 3. 登录后台	1. 登录失败 2. 权限不足	成功进入管理后台
AC02	用户管理	已登录后台	1. 查看用户列表 2. 编辑/禁用用户	1. 操作失败 2. 用户不存在	用户信息更新
AC03	作品审核	存在待审核作品	1. 查看作品 2. 审核通过或拒绝	1. 审核失败 2. 数据异常	审核结果生效
AC04	模型库管理	已登录后台	1. 上传模型 2. 修改/删除模型	1. 上传失败 2. 格式错误	模型库更新
AC05	材质库管理	已登录后台	1. 上传材质 2. 编辑/删除材质	1. 操作失败 2. 文件异常	材质库更新
AC06	分类与内容管理	已登录后台	1. 新增分类 2. 修改/删除分类	1. 操作失败	分类信息更新

3.3 主要功能需求

3.3.1 用户管理模块

支持用户注册、登录、个人信息管理功能。系统采用 JWT 认证机制，区分普通用户和管理员两种角色。普通用户可管理个人作品、参与社区互动；管理员拥有内容审核、用户管理和 3D 模型管理权限。

3.3.2 3D 服装定制模块

提供基于 Web3D 的服装可视化编辑工具。用户可选择基础服装模型，实时调整颜色、材质参数，上传自定义图案并指定应用部位，通过鼠标交互旋转、缩放查看模型细节。系统需保证渲染流畅性，编辑操作响应延迟控制在 100 ms 以内。

3.3.3 AI 虚拟试穿模块

支持将定制完成的 3D 服装转换为真人试穿图像。用户选择模特体型和场景风格后，系统调用 AI 服务生成高保真效果图。该模块需处理任务队列管理、生成进度反馈和结果展示功能，平均生成耗时控制在 30 秒以内。

3.3.4 创意广场模块

提供作品发布、浏览、搜索功能。支持按热度、时间、标签等排序，实现点赞、收藏、评论等互动操作。用户可关注其他创作者，便于查看喜欢的作品类型。

3.3.5 作品管理模块

支持用户对 3D 草稿和 AI 成图的完整生命周期管理，包括保存、编辑、删除、提审、发布设置。提供作品历史版本回溯功能。

3.3.6 后台管理模块

管理员可进行用户账号状态管理、作品内容审核、3D 模型库维护等操作。

3.4 系统数据需求

3.4.1 数据流图

根据系统功能需求分析，画出主要功能的数据流图。系统的数据流图如图 3.3 所示。

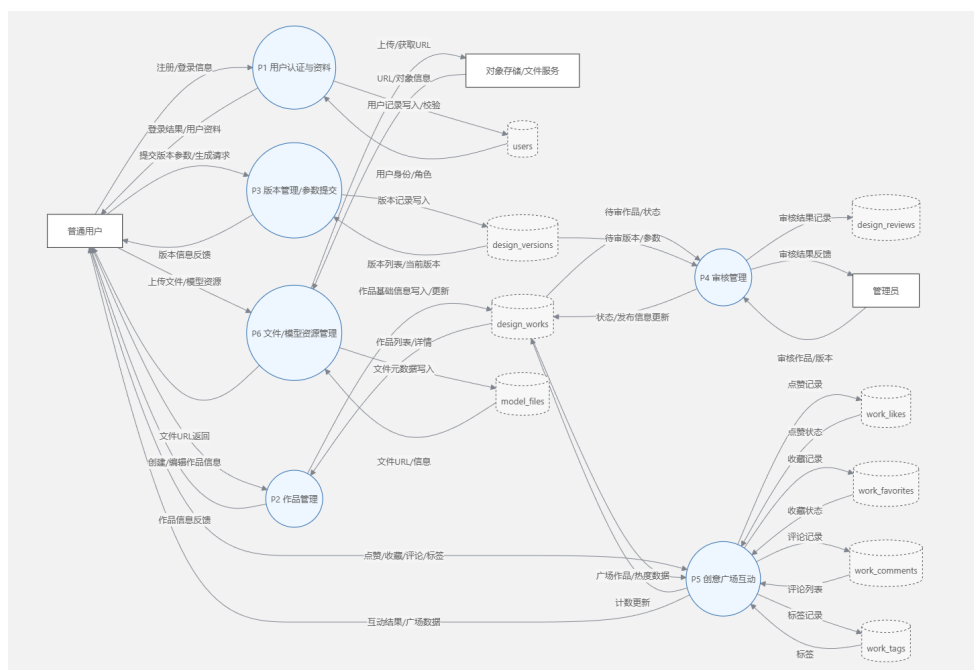


图 3.3 系统数据流图

3.4.2 数据项

根据数据流图以及数据流的描述，可以得到数据库设计所需数据项。系统的数据库设计所需的各数据项描述如表 3.3 所示。

表 3.3 系统数据库设计数据项

数据项名	数据项含义	别名	数据类型	长度	取值范围
id	主键 ID	user_id	bigint unsigned	20	≥ 1 ，自增
username	登录用户名（唯一）	login_name	varchar	50	非空，唯一
password	加密后的密码	pwd_hash	varchar	255	非空
avatar	头像 URL 地址	avatar_url	varchar	255	为空或合法 URL
role	角色权限：1 普通用户，2 管理员	user_role	tinyint unsigned	2	{1,2}
created_at	注册时间	register_time	datetime	-	合法时间
updated_at	最后更新时间	update_time	datetime	-	合法时间
id	主键 ID	file_id	bigint unsigned	20	≥ 1 ，自增

续表 3-3

数据项名	数据项含义	别名	数据类型	长度	取值范围
name	展示名称	file_name	varchar	255	非空
url	访问 URL	file_url	varchar	1024	非空，合法 URL
size	文件大小 (byte)	file_size	bigint unsigned	20	≥ 0
created_at	创建时间	create_time	datetime	-	合法时间
updated_at	更新时间	update_time	datetime	-	合法时间
id	主键 ID	work_id	bigint unsigned	20	≥ 1 ，自增
user_id	作者用户 ID	author_id	bigint unsigned	20	≥ 1
title	作品标题	work_title	varchar	255	非空
description	作品描述	work_desc	text	-	可为空
cover_file_id	封面文件 ID	cover_id	bigint unsigned	20	可为空或 ≥ 1
status	作品状态	work_status	tinyint unsigned	3	≥ 0 (按业务定义状态码)
published_at	发布时间	publish_time	datetime	-	可为空或合法时间
created_at	创建时间	create_time	datetime	-	合法时间
updated_at	更新时间	update_time	datetime	-	合法时间
likes_count	点赞数	like_cnt	int	-	≥ 0
comments_count	评论数	comment_cnt	int	-	≥ 0
favorites_count	收藏数	fav_cnt	int	-	≥ 0
is_featured	是否精选	featured_flag	tinyint	1	{0,1}
id	主键 ID	review_id	bigint unsigned	20	≥ 1 ，自增
work_id	作品 ID	work_id	bigint unsigned	20	≥ 1
admin_id	审核管理员 ID	reviewer_id	bigint unsigned	20	≥ 1

续表 3-3

数据项名	数据项含义	别名	数据类型	长度	取值范围
result	审核结果码	review_result	tinyint unsigned	3	≥ 0 （按业务定义结果码）
reason	审核原因/说明	review_reason	varchar	1024	可为空或字符串
reviewed_at	审核时间	review_time	datetime	-	合法时间
id	主键 ID	tag_id	bigint unsigned	20	≥ 1 , 自增
work_id	作品 ID	work_id	bigint unsigned	20	≥ 1
tag	标签内容	tag_name	varchar	32	非空
id	主键 ID	like_id	bigint unsigned	20	≥ 1 , 自增
work_id	作品 ID	work_id	bigint unsigned	20	≥ 1
user_id	点赞用户 ID	user_id	bigint unsigned	20	≥ 1
created_at	点赞时间	like_time	datetime	-	合法时间
id	主键 ID	fav_id	bigint unsigned	20	≥ 1 , 自增
work_id	作品 ID	work_id	bigint unsigned	20	≥ 1
user_id	收藏用户 ID	user_id	bigint unsigned	20	≥ 1
created_at	收藏时间	fav_time	datetime	-	合法时间
id	主键 ID	comment_id	bigint unsigned	20	≥ 1 , 自增
work_id	作品 ID	work_id	bigint unsigned	20	≥ 1
user_id	评论用户 ID	user_id	bigint unsigned	20	≥ 1
content	评论内容	comment_text	text	-	非空
created_at	评论时间	comment_time	datetime	-	合法时间

4 系统设计

4.1 系统总体设计

4.1.1 系统架构设计

本平台面向普通用户与系统管理员两类角色，围绕“3D 服装可视化定制”和“AI 真人试穿效果生成”构建“工具 + 社区”的一体化应用形态，系统采用“前端—后端—数据层—外部 AI 服务”的分层架构（见图 4.1）。其中，前端聚焦三维渲染与交互，将复杂的服装编辑过程以参数化方式呈现并实时预览，降低创作门槛；后端统一承载鉴权与业务规则，对作品、版本与互动等数据进行一致性管理，并以标准接口向前端返回业务数据；通过任务化/异步化方式接入 AI 试穿，由后端负责创建任务、维护状态与结果回写；数据层将结构化业务数据与大文件资源存储至云端，以 url 形式保存，并引入缓存与队列机制提升热点访问与任务处理效率，从而支撑平台的稳定运行与后续功能扩展。

系统架构图：

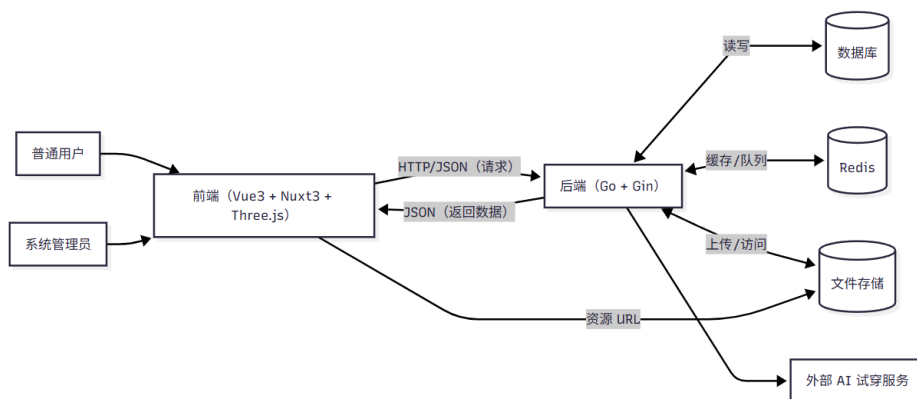


图 4.1 系统架构图

4.1.2 系统功能结构设计

系统功能结构采用“角色—业务域—功能点”的划分方式，保证前台创作与社区能力、后台治理能力相互独立又可协同。系统总体功能结构如图 4.2 所示。

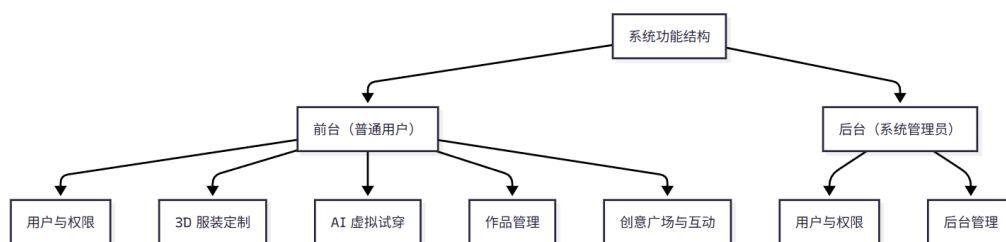


图 4.2 系统功能结构图

整体功能结构如下：

- (1) **用户与权限**注册登录、JWT 鉴权、角色区分（普通用户/管理员）、个人信息维护。
- (2) **3D 服装定制**基础模型选择、颜色与材质参数调整、图案上传与部位应用、三维交互（旋转/缩放/查看细节）、设计草稿保存。
- (3) **AI 虚拟试穿**模特体型/场景风格选择、生成任务创建、生成进度反馈、结果展示与保存。
- (4) **作品管理**作品信息编辑、草稿、发布/下架/删除、编辑回溯。
- (5) **创意广场与互动**作品浏览/搜索/排序（热度/时间/标签）、点赞/收藏/评论、关注创作者、个人作品主页展示。
- (6) **后台管理**管理员登录、用户管理、作品审核（通过/拒绝/下架）、模型库维护。

4.2 系统功能模块设计

系统功能模块设计以“前台创作与社区 + 后台治理运营”为主线，围绕用户体系、3D 定制、AI 试穿、作品管理、创意广场互动与后台管理等核心业务模块展开。各模块之间以统一的后端接口进行解耦协作，以“作品/分享”为核心业务对象贯通数据，形成完整的创作与运营闭环。

4.2.1 用户与权限模块

模块功能：该模块为系统提供身份认证与权限控制能力，覆盖用户注册、登录、退出、个人信息维护与角色权限校验等功能。系统采用 JWT 作为鉴权载体，在保证前端无状态访问的同时，实现普通用户与管理层接口的权限隔离，为作品发布、互动与后台治理提供统一的安全入口。

模块功能结构图：

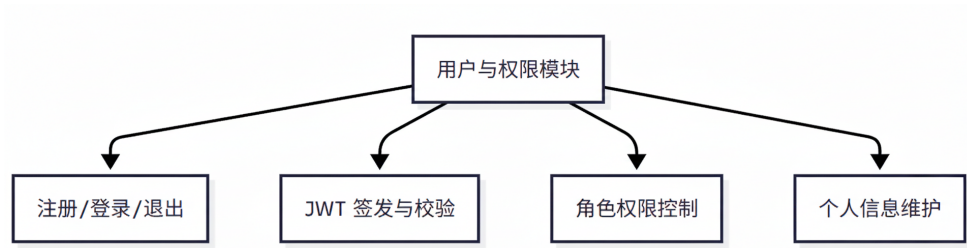


图 4.3 用户与权限模块功能结构

模块流程描述：用户提交账号密码完成登录后，后端校验凭证并签发 JWT；前端在后续请求中携带令牌，后端完成令牌校验与角色鉴权后返回业务数据；当用户退出或令牌失效时，前端清理本地令牌并重新进入登录流程。

模块流程图：

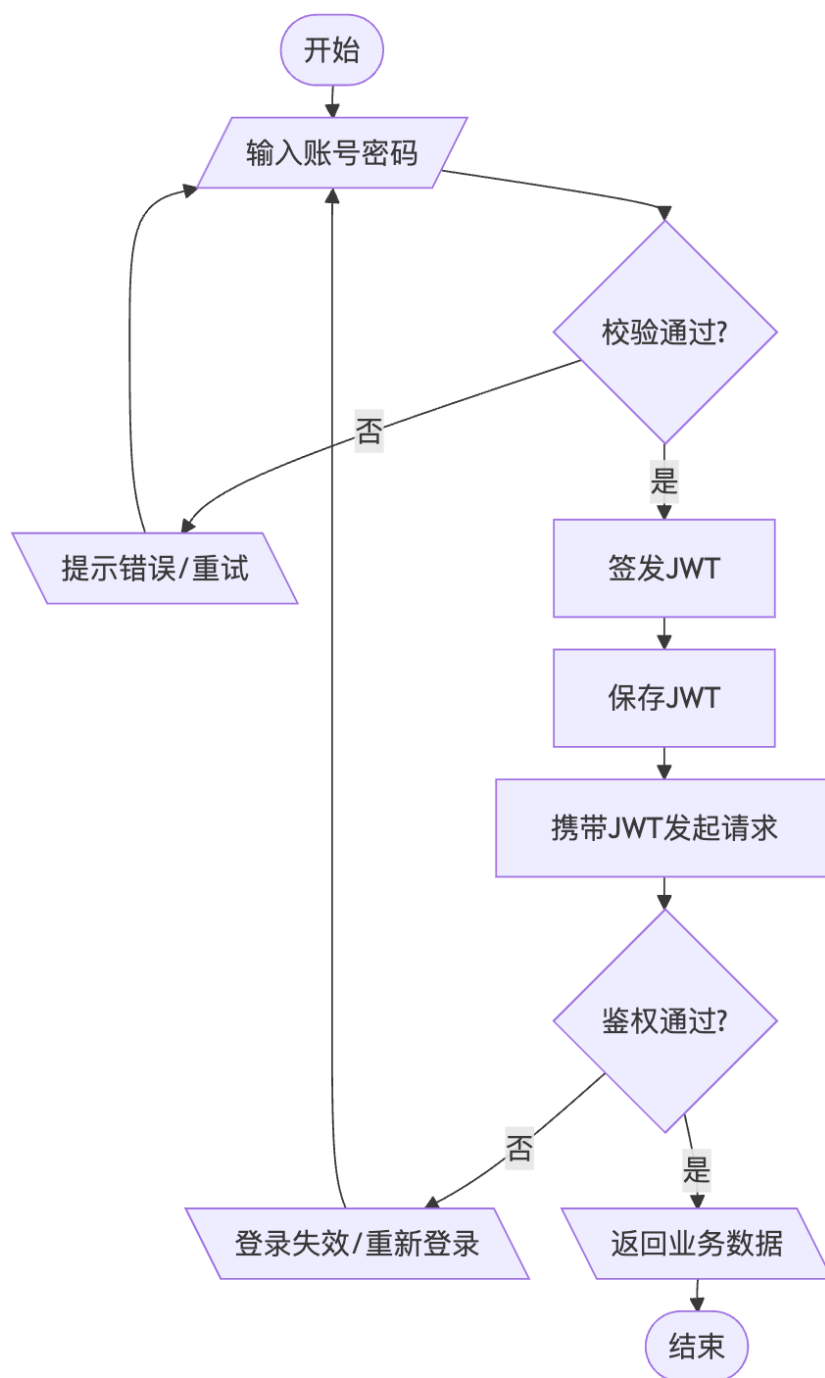


图 4.4 用户流程图

4.2.2 3D 服装定制模块

模块功能：该模块在浏览器端提供三维可视化服装定制能力，支持基础模型选择、颜色与材质参数调整、图案上传与部位应用、视角控制与细节预览等操作。服装三维捕获与重定向技术^[17]的研究成果为本模块的服装形变处理提供了

参考基础。用户可将定制配置保存为草稿或作品版本，为作品发布提供可复用的设计数据。

模块功能结构图：

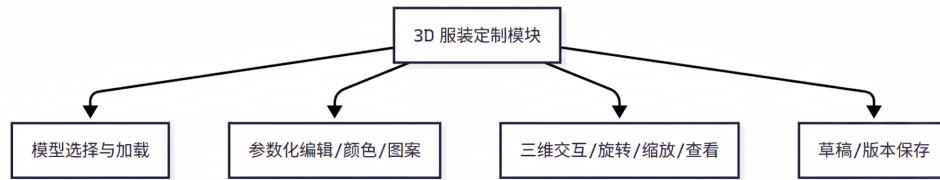


图 4.5 3D 服装定制模块功能结构

模块流程描述：用户选择基础服装模型后，前端加载模型与材质资源并渲染；用户通过交互面板调整参数并实时预览；当用户保存时，前端将关键配置参数（模型引用、材质与贴图、颜色与部位信息等）提交至后端形成草稿/版本记录，支持后续继续编辑或提审发布。

模块流程图：

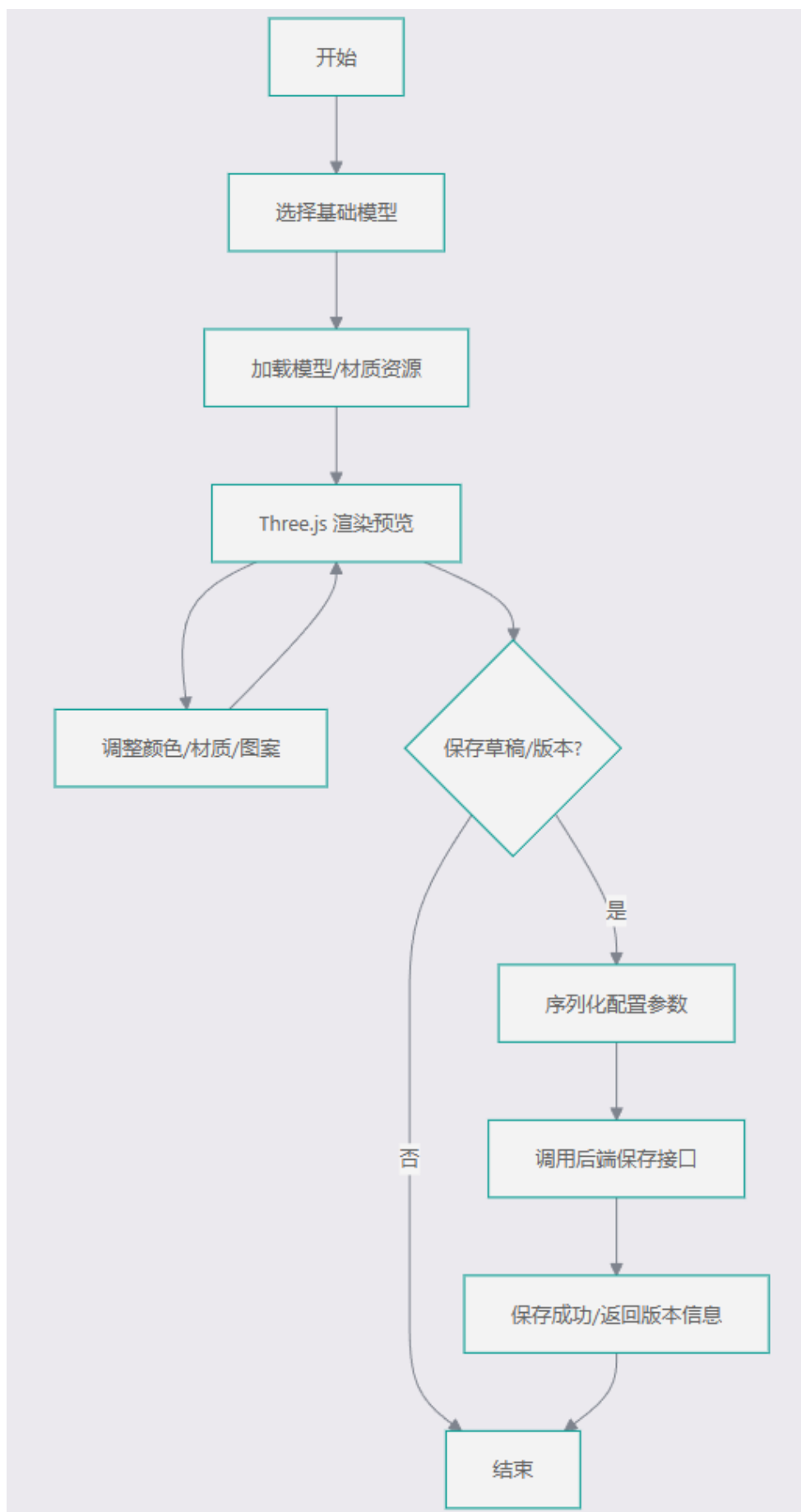


图 4.6 3D 服装定制模块流程图

4.2.3 AI 虚拟试穿模块

模块功能：该模块将用户的 3D 设计各角度平面图作为条件输入，生成高保真的真人试穿效果图。本系统 AI 试穿功能以去噪扩散概率模型 (DDPM)^[18] 为核心基础，借助潜在扩散模型 (LDM)^[19] 的高效图像合成能力，并采用 ControlNet^[20] 实现以 3D 服装渲染图作为控制条件的引导生成。试穿前处理流程中涉及人体姿态估计^[21] 与人体语义解析^[22]，以保证服装与人体的对齐精度。最终参考 M3D-VTON^[23] 的单目转三维方案提升试穿立体感。由于 AI 生成属于耗时任务，系统采用任务化/异步化机制：后端创建任务并返回任务 ID，任务执行模块从队列取出任务调用外部 AI 服务，生成结果保存至文件存储并写入数据库，前端通过查询接口获取进度并展示结果。

模块功能结构图：

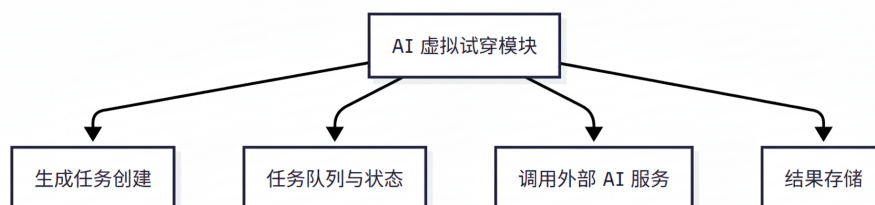


图 4.7 AI 虚拟试穿模块功能结构

模块流程描述：用户在前端选择模特体型与场景风格后提交生成请求；后端创建任务写入队列并返回任务 ID；前端基于任务 ID 查询进度；任务执行模块消费队列调用外部 AI，生成完成后上传结果文件并写入版本数据，更新任务状态；前端获取到完成状态后展示试穿图并允许用户保存。

模块流程图：

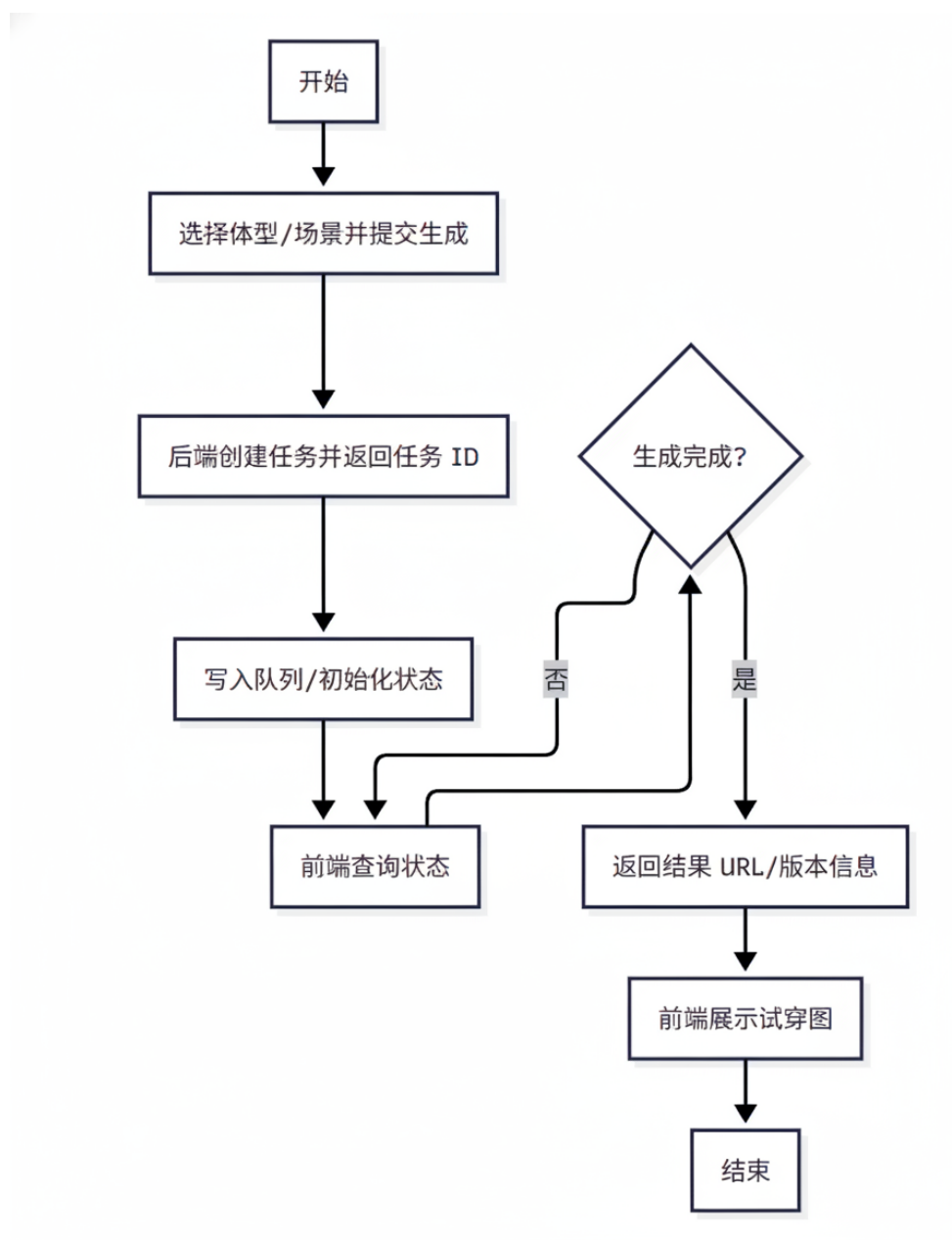


图 4.8 AI 虚拟试穿模块流程图

4.2.4 作品管理模块

模块功能：该模块面向普通用户提供作品全生命周期管理能力，包含作品创建、信息编辑、草稿、发布/下架、删除等功能。

模块功能结构图：

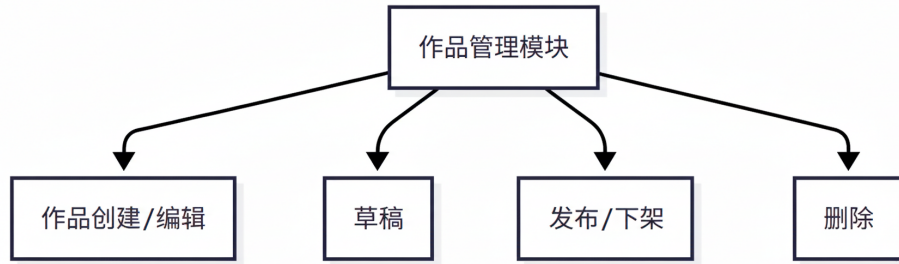


图 4.9 作品管理模块功能结构

模块流程描述：用户创建作品并编辑标题、描述等信息；编辑完成后存储为草稿可选择提审发布，后端更新作品状态与发布时间；当用户下架或删除作品时，后端校验归属权限并更新状态，保证数据一致性与可审计性。

模块流程图：

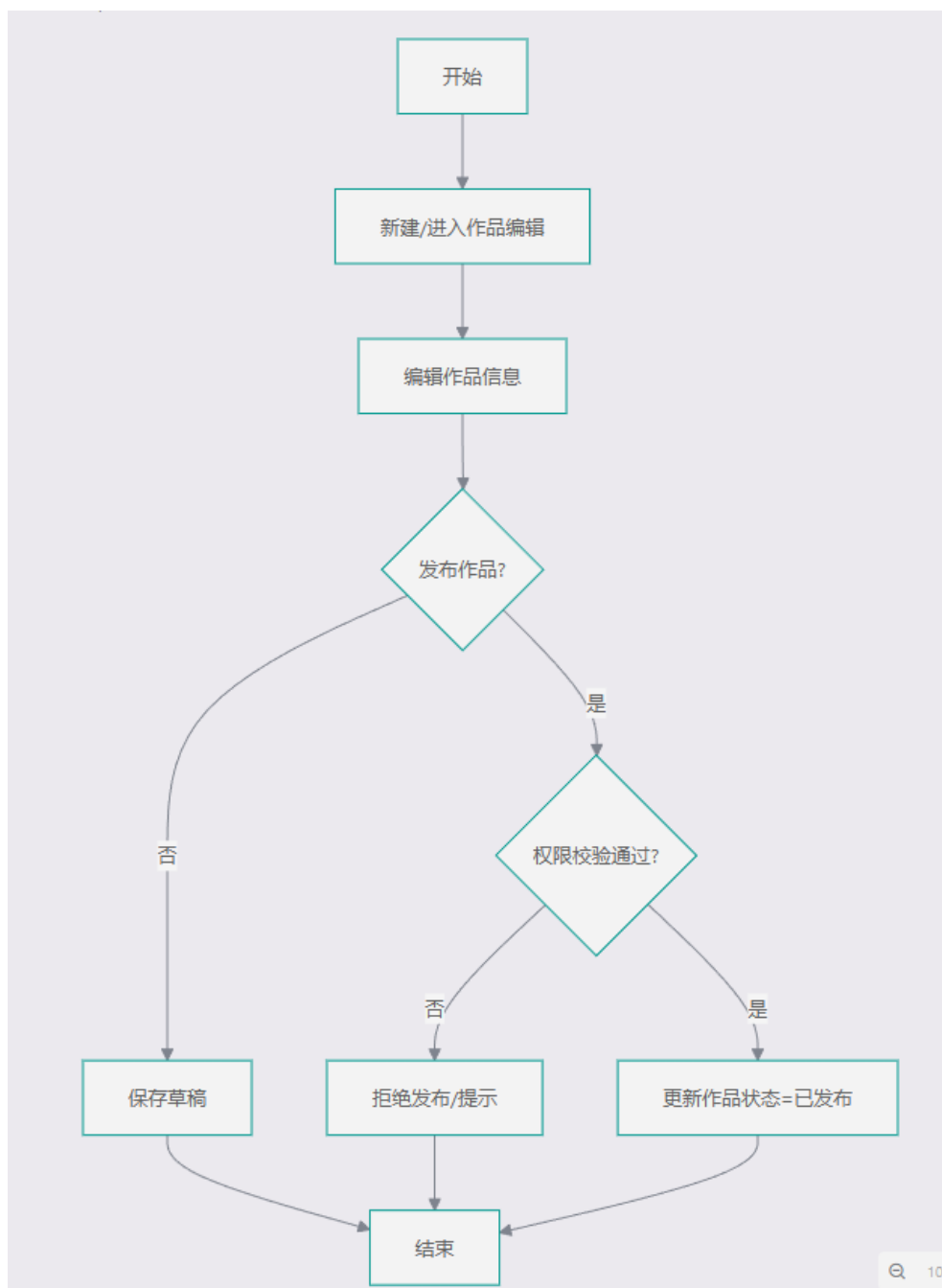


图 4.10 作品管理模块流程图

4.2.5 创意广场与社区互动模块

模块功能：该模块提供作品展示与社区交互能力，包含作品列表浏览、搜索与排序（热度/时间/标签等）、作品详情查看，以及点赞、收藏、评论、关注等互动功能。模块通过将互动行为与作品数据关联，形成反馈闭环，提升作品传播与社区活跃度。

模块功能结构图：

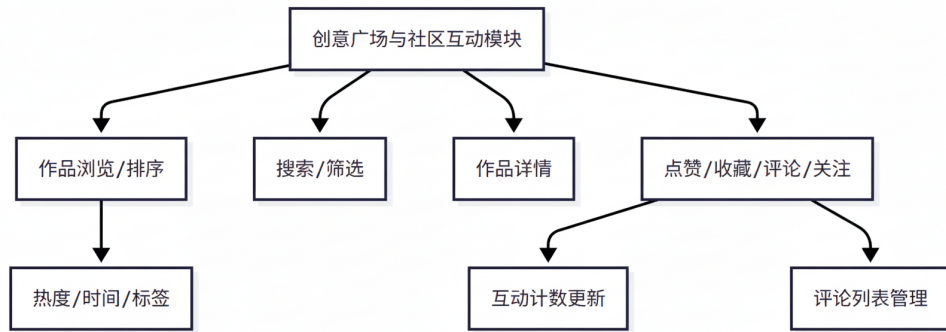


图 4.11 创意广场与社区互动模块功能结构

模块流程描述：用户进入创意广场后，前端请求作品列表并支持排序与筛选；用户打开作品详情页后加载作品信息、互动数据；当用户进行点赞、收藏或评论时，前端提交互动请求，后端校验登录状态并写入互动记录，更新计数并返回最新状态以刷新页面展示。

模块流程图：

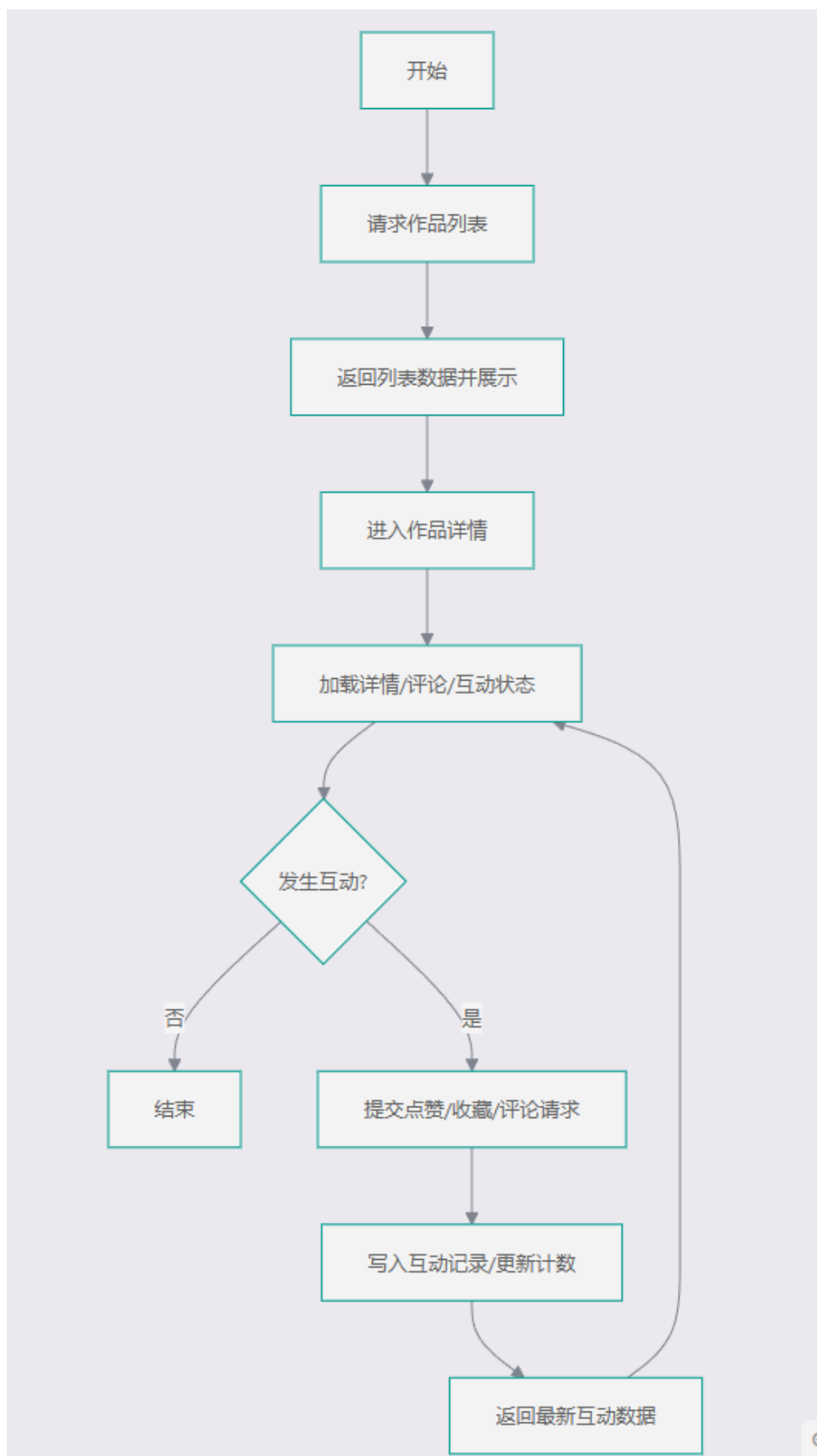


图 4.12 创意广场与社区互动模块流程图

4.2.6 后台管理模块

模块功能：该模块面向系统管理员提供用户管理、模型管理与内容治理能力，包括用户管理（信息维护，权限分配）、作品内容审核（通过/拒绝/下架）、模型库维护等功能。模块通过权限控制与审核流程保障平台内容质量与资源可用性。

模块功能结构图：

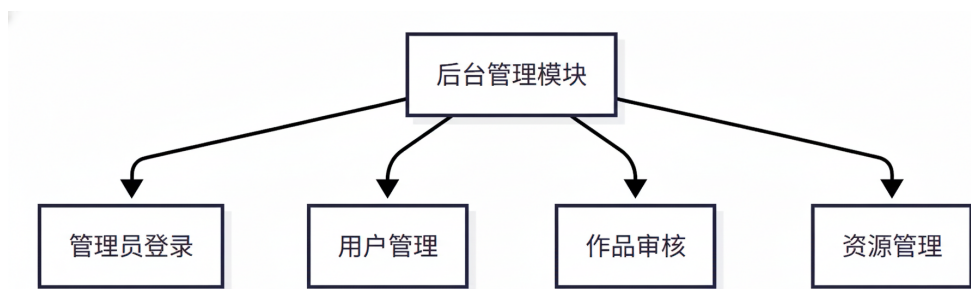


图 4.13 后台管理模块功能结构

模块流程描述：管理员登录后进入后台；在用户管理中可检索用户并执行角色分配，信息修改等操作；在作品审核中查看待审内容并做出通过或拒绝决定，必要时可对已发布内容执行下架；在资源管理中维护模型资源，保证前端定制能力与内容展示的基础数据完整。

模块流程图：

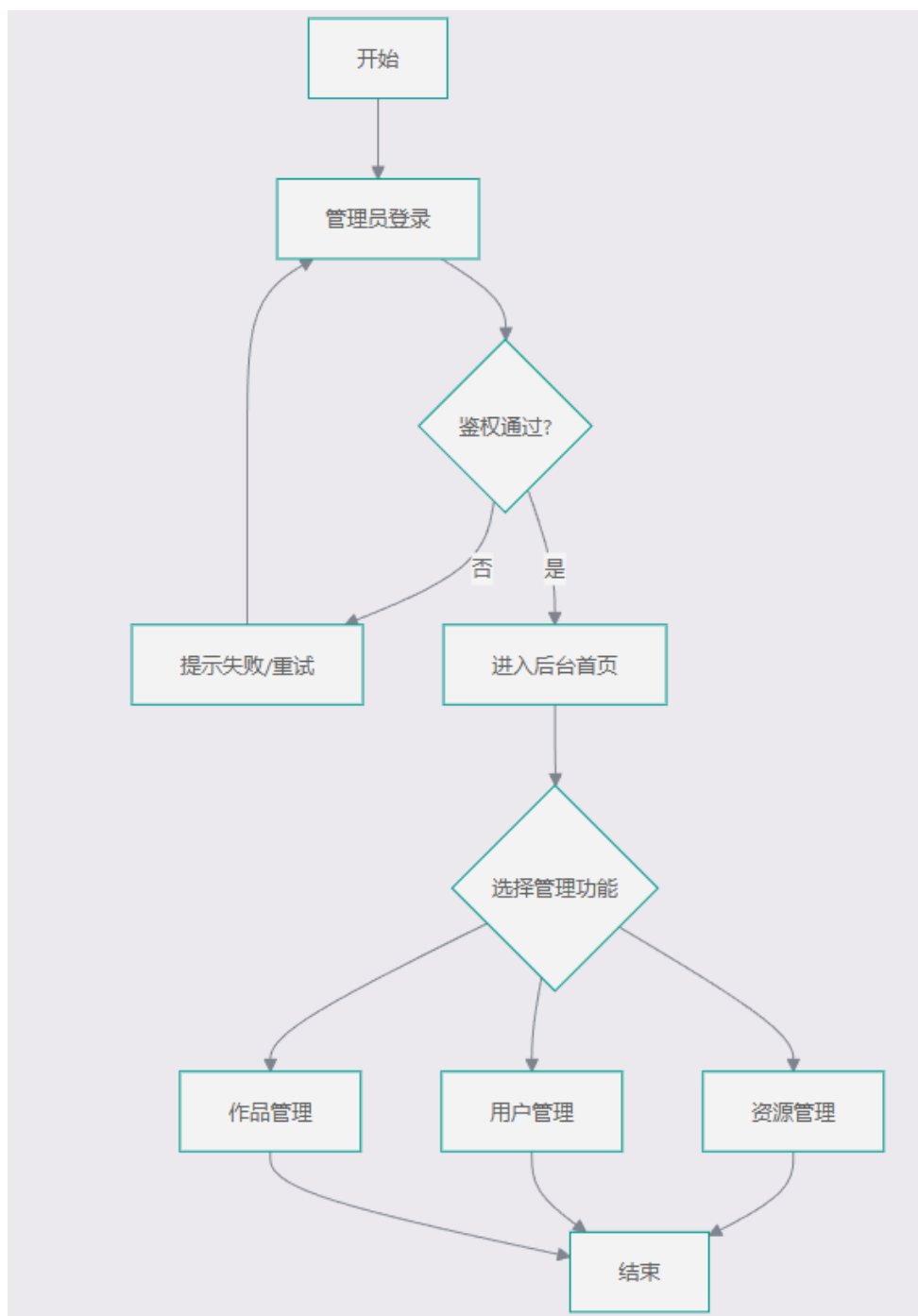


图 4.14 后台管理流程图

4.3 系统数据库设计

4.3.1 数据库实体联系分析

根据第 3 章系统需求分析，得出以下结论：

- 本系统数据库可划分为三个核心子域：用户域（users）、作品广场域（design_works 及互动表）、任务域（tryon_tasks），并由资源域（model_files）

提供文件支撑。

- **users** 为全局主体实体，承担身份认证与角色区分功能（普通用户/管理员），并作为作品发布、点赞、收藏、评论、试衣任务发起的统一行为主体。
- 为支持作者社交数据展示，**users** 表中引入 **followers_count** 与 **following_count** 两个冗余统计字段，用于快速展示用户的粉丝数与关注数，降低列表页聚合统计开销。
- **user_follows** 用于记录用户之间的关注关系，以 **follower_id** 与 **following_id** 两个外键同时关联 **users**，通过唯一约束 (**follower_id**, **following_id**) 防止重复关注，并在用户注销时通过级联删除清理关注关系数据。
- **design_works** 为业务核心实体，记录作品基础信息（标题、描述、状态、发布时间等），通过 **user_id** 关联作品作者；通过 **cover_file_id** 关联封面资源。
- **model_files** 用于统一管理文件资源（名称、URL、大小），在当前模型中主要被作品封面复用，形成“一个文件可被多个作品引用、一个作品最多一个封面文件”的关系。
- **work_tags**、**work_likes**、**work_favorites**、**work_comments** 构成作品互动子模型：标签表达作品的语义分类；点赞与收藏是用户与作品的多对多关系落地；评论表达用户对作品的一对多文本反馈。
- **tryon_tasks** 表示用户针对作品触发的试衣生成任务，记录任务状态、进度、结果地址与异常信息，形成“用户—任务”“作品—任务”的双一对多关系。
- 一致性方面，互动表对作品设置了外键级联删除；同时点赞/收藏通过 (**work_id**, **user_id**) 唯一约束防止重复行为。**design_works** 中的计数字段（点赞数、评论数、收藏数）属于冗余统计字段，用于提高查询性能。

主要联系

- 用户 **users** 与作品 **design_works**：1 对 N（一个用户可发布多个作品）。
- 用户 **users** 与关注关系 **user_follows**：1 对 N（一个用户可发起多条关注关系）；用户 **users** 与关注关系 **user_follows**：1 对 N（一个用户可被多条关注关系指向），从而在实体层面形成用户之间的 M 对 N 关注关系。
- 资源文件 **model_files** 与作品 **design_works**：1 对 N（一个文件可作为多个作品封面）。
- 作品 **design_works** 与标签 **work_tags**：1 对 N。
- 用户 **users** 与点赞 **work_likes**：1 对 N；作品 **design_works** 与点赞 **work_likes**：1 对 N。

- 用户 users 与收藏 work_favorites: 1 对 N; 作品 design_works 与收藏 work_favorites: 1 对 N。
- 用户 users 与评论 work_comments: 1 对 N; 作品 design_works 与评论 work_comments: 1 对 N。
- 用户 users 与试衣任务 tryon_tasks: 1 对 N; 作品 design_works 与试衣任务 tryon_tasks: 1 对 N。

根据以上分析, 系统的 ER 图如图 4.15 所示。

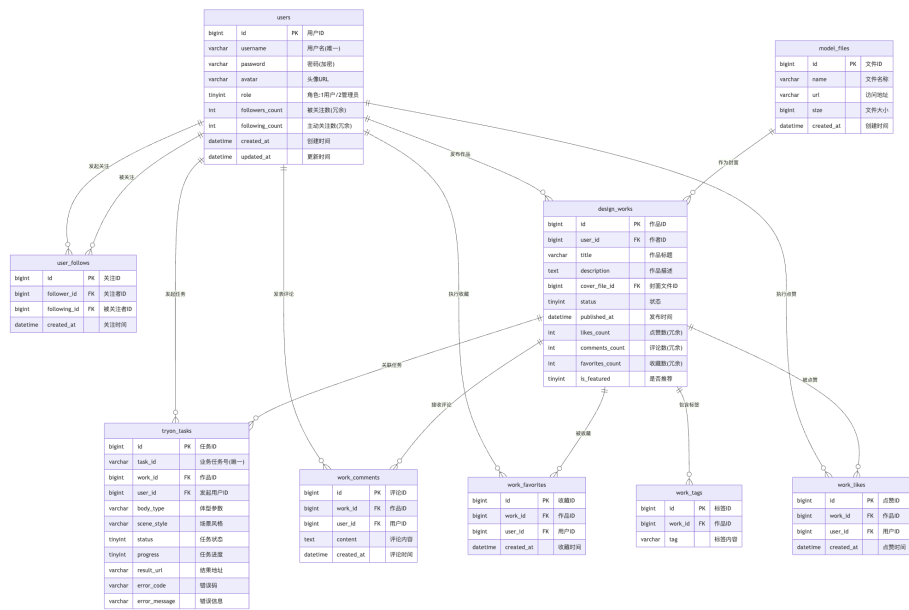


图 4.15 数据库 ER 图

4.3.2 数据库核心数据表

1) users (用户核心表)

users (用户核心表) 用于存储系统用户账号、密码、角色与基础资料, 是全局用户主体表。系统通过 user_follows 建立“用户-用户”多对多关注关系, 并引入 followers_count/following_count 作为冗余计数字段以降低列表页聚合统计开销; 在关注/取关时同步更新被关注者 followers_count 与关注者 following_count。

表 4.1 users 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
username	varchar(50)	NOT NULL, UNIQUE(uk_username)	登录用户名（唯一）
password	varchar(255)	NOT NULL	加密后的密码
avatar	varchar(255)	DEFAULT NULL	头像 URL
role	tinyint(2) UNSIGNED	NOT NULL, DEFAULT 1	角色：1-普通用户，2-管理员
followers_ count	int	NOT NULL, DEFAULT 0	被关注数（冗余统计）
following_ count	int	NOT NULL, DEFAULT 0	主动关注数（冗余统计）
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	注册时间
updated_at	datetime	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	最后更新时间

2) user_follows（用户关注关系表）

user_follows（用户关注关系表）用于记录用户之间的关注关系，以 follower_id 与 following_id 关联 users，支持关注/取关与关注列表/粉丝列表等功能，并通过唯一约束 (follower_id, following_id) 防止重复关注；用户注销时外键设置 ON DELETE CASCADE 以清理关联数据。

表 4.2 user_follows 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
follower_id	bigint(20) UNSIGNED	NOT NULL, KEY idx_follower(follower_id)	关注者 ID
following_id	bigint(20) UNSIGNED	NOT NULL, KEY idx_following(following_id)	被关注者 ID
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	关注时间
-	-	UNIQUE uk_follower_following(follower_id, following_id)	防止重复关注
-	-	FK follower_id -> users(id) ON DELETE CASCADE	关注者外键
-	-	FK following_id -> users(id) ON DELETE CASCADE	被关注者外键

3) model_files (模型/资源文件表)

model_files (模型/资源文件表) 用于统一管理系统上传/生成的文件资源元数据, 包括展示名称、访问 URL、文件大小以及创建/更新时间等信息, 并作为各业务表引用外部资源的统一入口。当前模型中该表主要承载作品封面文件的引用关系, 便于复用同一资源、减少冗余存储, 同时通过名称索引支持后台按名称检索与管理。

表 4.3 model_files 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
name	varchar(255)	NOT NULL, DEFAULT "	展示名称
url	varchar(1024)	NOT NULL	访问 URL
size	bigint(20) UNSIGNED	NOT NULL, DEFAULT 0	文件大小 (byte)
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	datetime	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间
-	-	KEY idx_name(name)	名称索引

4) design_works (作品主表)

design_works (作品主表) 用于存储作品主体信息, 是创意广场的核心业务表, 记录作品标题、描述、发布状态与发布时间等内容, 并通过 user_id 关联作品作者。该表可选关联封面文件 cover_file_id, 并维护点赞数、评论数、收藏数等冗余统计字段与推荐标识, 用于提升广场列表查询与排序展示的性能。

表 4.4 design_works 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	作品 ID
user_id	bigint(20) UNSIGNED	NOT NULL, KEY idx_user(user_id)	作者用户 ID

续表 4-4

字段名	类型	约束/索引	说明
title	varchar(255)	NOT NULL, DEFAULT "	作品标题
description	text	-	作品描述
cover_file_id	bigint(20) UNSIGNED	DEFAULT NULL	封面文件 ID
status	tinyint(3) UNSIGNED	NOT NULL, DEFAULT 1, KEY idx_status(status)	作品状态
published_at	datetime	DEFAULT NULL	发布时间
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	datetime	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间
likes_count	int	NOT NULL, DEFAULT 0	点赞数（冗余统计）
comments_count	int	NOT NULL, DEFAULT 0	评论数（冗余统计）
favorites_count	int	NOT NULL, DEFAULT 0	收藏数（冗余统计）
is_featured	tinyint(1)	NOT NULL, DEFAULT 0	是否推荐

5) work_tags（作品标签表）

work_tags（作品标签表）用于维护作品与标签的关联，将作品与若干语义标签进行绑定，以支撑分类浏览、条件筛选与个性化推荐等功能。表中通过 work_id 快速定位作品标签集合，并为 tag 建立索引以提高按标签聚合与检索的效率。

表 4.5 work_tags 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
work_id	bigint(20) UNSIGNED	NOT NULL, FK->design_works(id), INDEX idx_work_id(work_id)	作品 ID
tag	varchar(32)	NOT NULL, INDEX idx_tag(tag)	标签内容

6) work_likes（作品点赞表）

work_likes（作品点赞表）用于记录用户对作品的点赞行为，形成“用户-作品”的多对多关系落地数据，用于支撑点赞/取消点赞以及点赞列表查询等功能。该表通过唯一约束 (work_id, user_id) 防止重复点赞，并以时间字段记录行为发生时间，便于行为追溯与排序展示。

表 4.6 work_likes 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
work_id	bigint(20) UNSIGNED	NOT NULL, FK->design_works(id)	作品 ID
user_id	bigint(20) UNSIGNED	NOT NULL	点赞用户 ID
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	点赞时间

续表 4-6

字段名	类型	约束/索引	说明
-	-	UNIQUE uk_work_user(work_id, user_id)	防止重复点赞

7) work_favorites (作品收藏表)

work_favorites (作品收藏表) 用于记录用户对作品的收藏行为, 支撑用户收藏夹、收藏/取消收藏与收藏状态判断等功能, 并在数据层面表达“用户-作品”多对多关系。该表同样通过唯一约束 (work_id, user_id) 防止重复收藏, 并使用时间字段记录收藏发生时间, 便于收藏列表按时间排序。

表 4.7 work_favorites 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
work_id	bigint(20) UNSIGNED	NOT NULL, FK->design_works(id)	作品 ID
user_id	bigint(20) UNSIGNED	NOT NULL	收藏用户 ID
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	收藏时间
-	-	UNIQUE uk_work_user(work_id, user_id)	防止重复收藏

8) work_comments (作品评论表)

work_comments (作品评论表) 用于记录用户对作品的文本评论内容, 支撑作品详情页评论展示、评论发布与评论列表分页查询等功能。该表通过 work_id 关联评论所属作品、通过 user_id 标识评论作者, 并对 work_id 建立索引以优化按作品维度的评论检索。

表 4.8 work_comments 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
work_id	bigint(20) UNSIGNED	NOT NULL, FK->design_works(id), INDEX idx_work_id(work_id)	作品 ID
user_id	bigint(20) UNSIGNED	NOT NULL	评论用户 ID
content	text	NOT NULL	评论内容
created_at	datetime	DEFAULT CURRENT_TIMESTAMP	评论时间

9) tryon_tasks (试衣任务表)

tryon_tasks (试衣任务表) 用于记录用户针对作品发起的试衣生成任务及执行结果，支撑异步任务创建、进度查询与结果展示等流程，并通过唯一的业务任务号 task_id 进行幂等与外部调用关联。该表记录输入参考数据、任务状态与进度、结果地址以及异常信息，同时保存开始/完成时间与创建/更新时间，用于任务调度、失败重试与问题定位。

表 4.9 tryon_tasks 表结构

字段名	类型	约束/索引	说明
id	bigint(20) UNSIGNED	PK, AUTO_INCREMENT	主键 ID
task_id	varchar(64)	NOT NULL, UNIQUE uk_task_id(task_id)	业务任务号
work_id	bigint(20) UNSIGNED	NOT NULL, KEY idx_work_id(work_id)	关联作品 ID

续表 4-9

字段名	类型	约束/索引	说明
user_id	bigint(20) UNSIGNED	NOT NULL, KEY idx_user_id(user_id)	发起用户 ID
body_type	varchar(64)	NOT NULL, DEFAULT "	体型参数
scene_style	varchar(64)	NOT NULL, DEFAULT "	场景风格
input_refs	json	NOT NULL	输入参考数据
status	tinyint(3) UNSIGNED	NOT NULL, DEFAULT 1, KEY idx_status(status)	任务状态
progress	tinyint(3) UNSIGNED	NOT NULL, DEFAULT 0	任务进度
result_url	varchar(1024)	NOT NULL, DEFAULT "	结果地址
error_code	varchar(64)	NOT NULL, DEFAULT "	错误码
error_message	varchar(1024)	NOT NULL, DEFAULT "	错误信息
started_at	datetime	DEFAULT NULL	开始时间
finished_at	datetime	DEFAULT NULL	完成时间
created_at	datetime	DEFAULT CURRENT_TIMESTAMP, KEY idx_created_at(created_at)	创建时间
updated_at	datetime	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

5 系统实现

5.1 系统开发环境

本项目采用前后端分离架构，后端基于 **Go + Gin**，前端采用 **Vue 3 + Nuxt 3**。开发环境配置如下：

- (1) **硬件环境**选择 Intel Core i7 及以上级别 CPU、16GB 及以上内存、512GB 及以上 SSD 的开发主机，以保障本地数据库、缓存服务和前端构建任务并行运行的稳定性。
- (2) **操作系统**选择 **Windows 10/11** 作为主要开发平台（兼容 Linux/macOS 开发环境），满足 Go、Node.js、MySQL、Redis 等组件的安装与运行要求。
- (3) **前端技术栈**选择 **Vue 3** 作为前端核心框架，**Nuxt 3** 作为应用框架（支持 SSR/CSR 混合渲染），并结合 TypeScript 进行工程化开发。
- (4) **后端技术栈**选择 **Go 1.25.x** 作为后端开发语言，**Gin** 作为 Web 框架，提供 RESTful API 服务^[15]。
- (5) **数据存储与缓存**选择 **MySQL 8.x** 作为关系型数据库，**Redis 7.x** 作为缓存与状态加速组件，提升高频查询与会话相关操作性能。
- (6) **对象存储服务**选择 **阿里云 OSS** 作为文件存储服务，用于模型文件、作品资源及结果文件的统一管理与访问。
- (7) **项目依赖与构建工具**后端使用 **Go Modules** 进行依赖管理；前端使用 **Node.js 20 LTS + pnpm/npm** 进行依赖安装与构建管理。
- (8) **开发工具与辅助软件**后端开发可使用 **GoLand / VS Code**，前端开发使用 **VS Code (Vue/Nuxt 插件)**；数据库管理使用 **Navicat / DataGrip**；接口联调使用 **Apifox / Postman**；版本管理使用 **Git**。

5.2 系统功能模块实现

本系统采用前后端分离架构实现：前台用户端基于 Vue3 + Nuxt3，负责页面渲染、交互逻辑与接口调用；后端基于 Go + Gin，负责业务处理、权限控制、数据持久化与异步任务调度。各功能模块均通过 RESTful API 对外提供服务，前端通过 JWT 完成登录态管理与受限资源访问。

5.2.1 用户管理模块实现

用户管理模块涉及前台认证与用户信息维护,后端核心对象为 `AuthController`、`User` 数据模型以及 `JWT`/密码工具组件。前端 `Nuxt3` 通过登录页、注册页和个人中心页调用用户接口,实现身份认证与资料维护。

表 5.1 用户管理模块接口

功能说明	接口 API	Http 方法
用户注册	<code>/api/front/register</code>	POST
用户登录	<code>/api/front/login</code>	POST
查询个人信息	<code>/api/front/me</code>	GET
修改个人信息/密码	<code>/api/front/me</code>	POST
上传用户头像	<code>/api/front/me/avatar</code>	POST

根据以上接口,用户模块登录与认证实现过程如下:

- (1) 前端提交用户名与密码到登录接口;
- (2) 后端校验参数完整性,按用户名查询用户记录;
- (3) 使用密码哈希比对算法验证口令正确性;
- (4) 校验通过后生成 `JWT Token`,并返回用户基础信息;
- (5) 前端保存 `Token` (如 `Pinia + LocalStorage/Cookie`),后续请求统一携带 `Authorization: Bearer <token>`;
- (6) 受保护接口通过认证中间件解析 `Token`,将 `userID/role` 写入上下文;
- (7) 前端根据返回结果提示用户“登录成功/失败”,并完成页面跳转与登录态刷新,界面如图 5.1 所示。

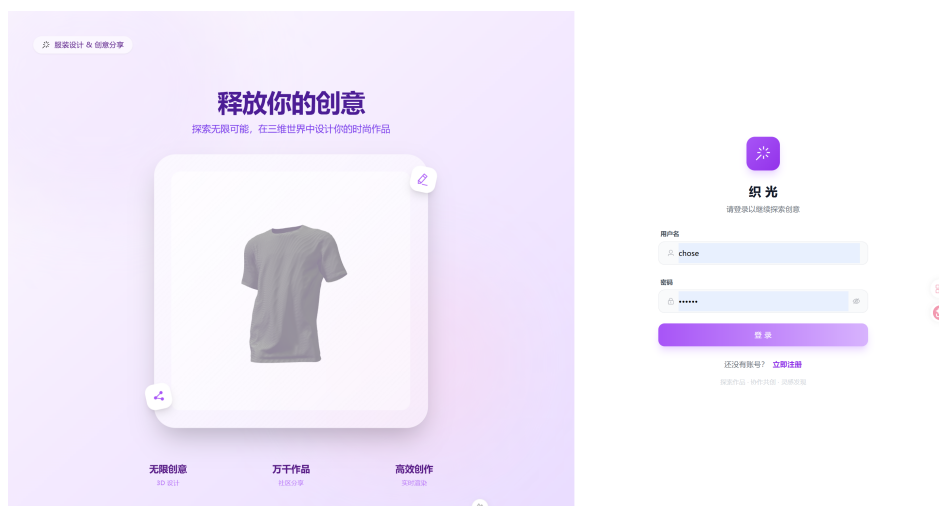


图 5.1 用户登录

5.2.2 3D 服装定制模块实现

3D 服装定制模块用于实现服装参数化设计与模板选择。前端提供参数面板和设计表单，同时通过 Three.js 渲染模型；后端维护作品草稿、定制参数及模型资源引用，实现“创建—编辑—保存”的设计闭环。

表 5.2 3D 服装定制模块接口

功能说明	接口 API	Http 方法
模板资源列表	/api/front/templates	GET
新建定制作品	/api/front/works	POST
更新定制作品	/api/front/works/:id	POST
设置作品标签	/api/front/works/:id/tags	POST

该模块实现过程如下：

- (1) 前端先请求模板列表，加载可选模型资源；
- (2) 用户填写标题、描述、封面并配置 params_json 参数；
- (3) 后端创建作品主记录，并写入初始设计参数；
- (4) 用户二次编辑时，后端按作品状态判断是否允许修改；
- (5) 驳回后重编辑会自动回到草稿态，便于再次提交；
- (6) 标签信息独立维护，支持后续广场检索与分类展示；
- (7) 前端根据保存结果实时刷新设计视图与作品状态，界面如图 5.2 所示。

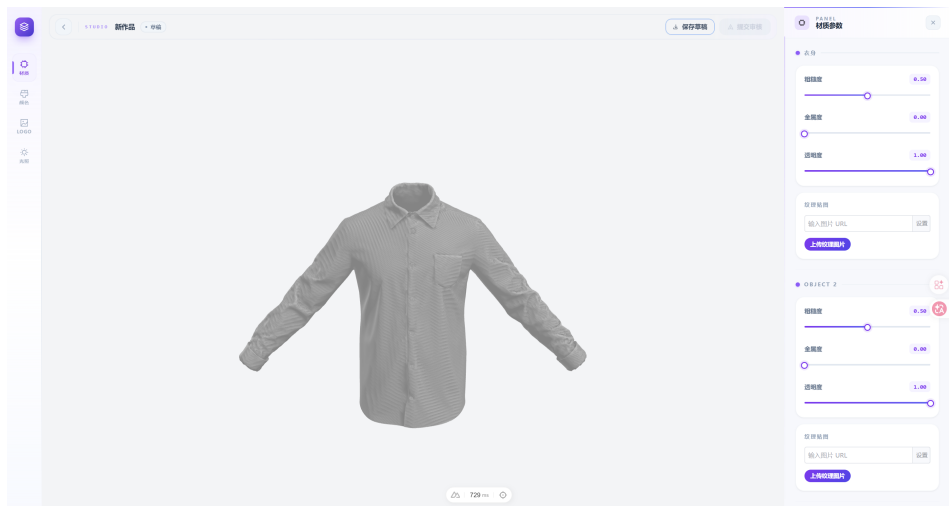


图 5.2 3D 服装定制

5.2.3 AI 虚拟试穿模块实现

AI 虚拟试穿模块采用异步任务架构，服务端由”任务创建接口 + Worker 消费进程”构成。前端 Nuxt3 负责提交试穿参数并轮询任务状态，完成结果展示。本模块参考了 DrapeNet^[24] 的服装生成与自监督拟合思路，结合扩散模型进行真人效果合成，生成图像质量采用结构相似性（SSIM）^[25] 等指标衡量。

表 5.3 AI 虚拟试穿模块接口

功能说明	接口 API	Http 方法
创建试穿任务	/api/front/works/:id/tryon/tasks	POST
查询任务状态	/api/front/works/:id/tryon/tasks/:task_id	GET

该模块实现过程如下：

- (1) 前端提交体型参数、场景风格与参考输入数据；
- (2) 后端校验用户权限、作品归属与任务并发限制；
- (3) 创建试穿任务记录，状态置为“排队中”；
- (4) 任务写入 Redis 队列，由 Worker 异步消费；
- (5) Worker 调用 AI 推理服务，更新执行进度与运行状态；
- (6) 执行成功后写入结果地址，失败则记录错误码并重试/终止；
- (7) 前端轮询任务接口，根据状态与进度更新 UI 并展示试穿结果图，界面如图 5.3、图 5.4所示。

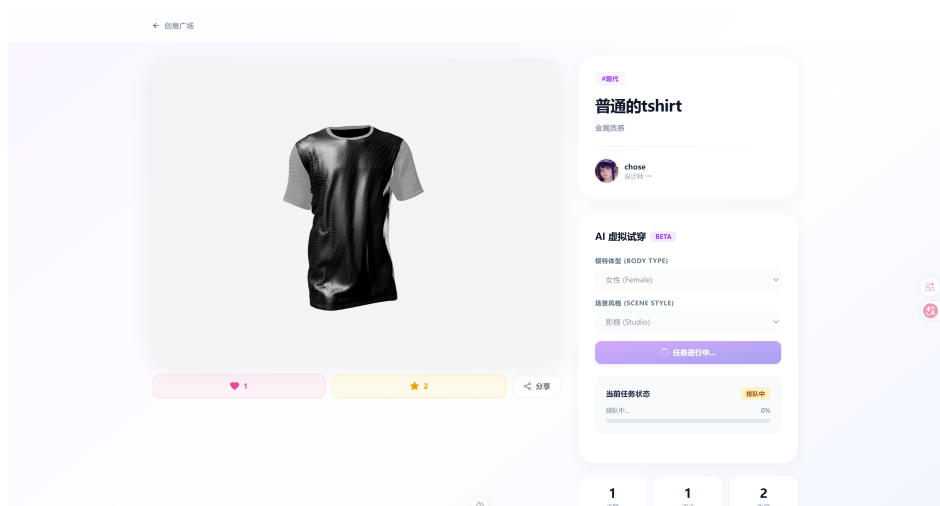


图 5.3 AI 虚拟试穿生成排队

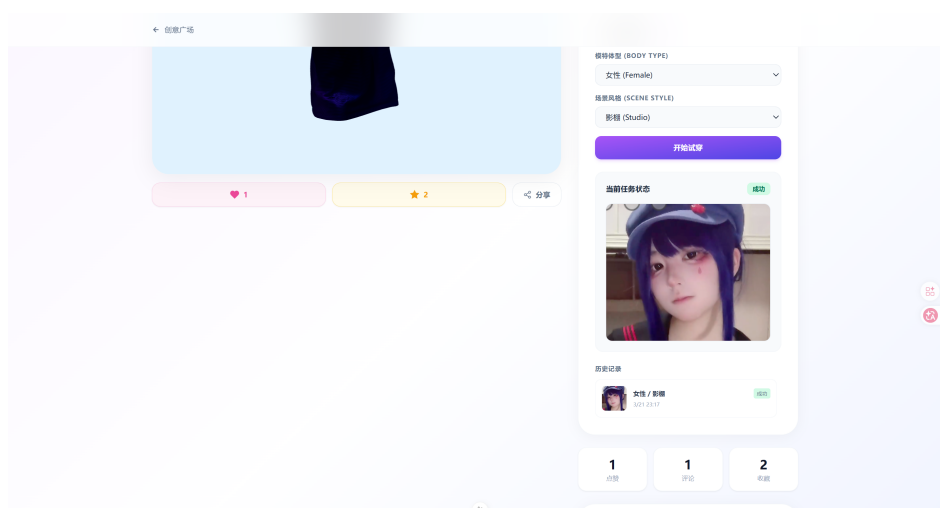


图 5.4 AI 虚拟试穿生成结果

5.2.4 创意广场模块实现

创意广场模块面向作品公开展示与社区互动，支持列表浏览、标签筛选、热度排序、点赞、收藏、评论以及关注创作者。前端对应广场首页与作品详情页，后端支持可选登录态访问和缓存加速。

表 5.4 创意广场模块接口

功能说明	接口 API	Http 方法
广场作品列表	/api/front/square/works	GET
广场作品详情	/api/front/square/works/:id	GET
广场标签列表	/api/front/square/tags	GET
点赞/取消点赞	/api/front/square/works/:id/like	POST
收藏/取消收藏	/api/front/square/works/:id/favorite	POST
发表评论	/api/front/square/works/:id/comment	POST
关注/取消关注作者	/api/front/square/users/:id/follow	POST

该模块实现过程如下：

- (1) 仅展示“已发布”状态作品，保证内容质量与流程一致性；
- (2) 支持按标签、关键字、热度/趋势进行组合检索；
- (3) 未登录用户可浏览，登录用户可返回点赞/收藏状态；
- (4) 点赞与收藏使用关联表记录，并同步维护计数字段；
- (5) 评论写入评论表，作品评论总数同步更新；
- (6) 关注关系写入 user_follows 表，并同步更新用户的 followers_count/following_count 计数；
- (7) 为提升访问性能，列表/详情/标签使用 Redis 缓存；
- (8) 交互动作触发缓存失效，保证数据实时性与一致性，界面如图 5.5 所示。

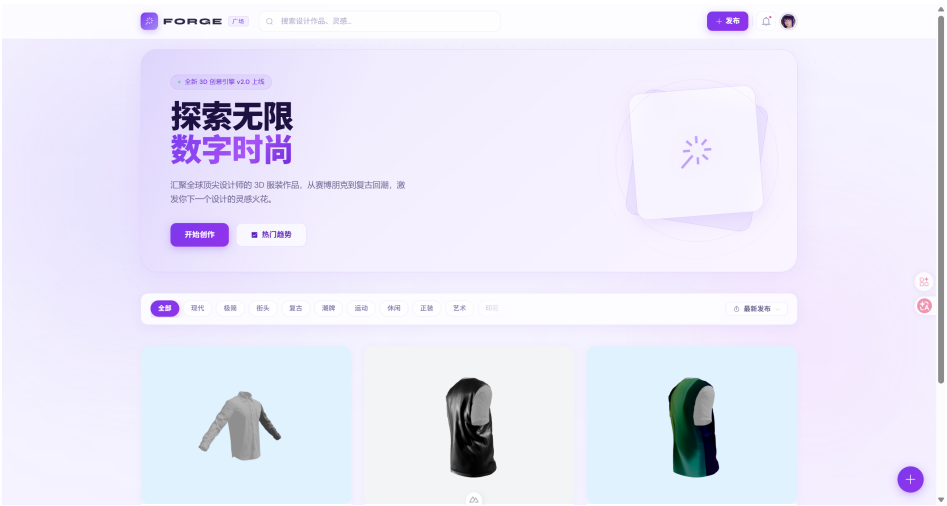


图 5.5 创意广场

5.2.5 作品管理模块实现

作品管理模块负责前台用户对个人作品的全生命周期管理，包括创建、编辑、提审、查看详情、下架和删除，并与审核流程、试穿流程联动。

表 5.5 作品管理模块接口

功能说明	接口 API	Http 方法
作品列表	/api/front/works	GET
作品详情	/api/front/works/:id	GET
创建作品	/api/front/works	POST
修改作品	/api/front/works/:id	POST
提交审核	/api/front/works/:id/submit	POST
下架作品	/api/front/works/:id/unpublish	POST
删除作品	/api/front/works/:id	DELETE

该模块实现过程如下：

- (1) 前端“我的作品”页分页拉取作品列表；
- (2) 用户创建作品后默认进入草稿态；
- (3) 编辑接口按状态机控制可编辑范围；
- (4) 提交审核后状态转为“已提交”，等待管理员审核；

- (5) 详情页聚合展示作品信息、审核记录与试穿历史；
- (6) 下架接口将发布作品恢复为草稿态，支持继续修改；
- (7) 删除接口限制审核中/已发布作品，避免业务流程异常，界面如图 5.6所示。

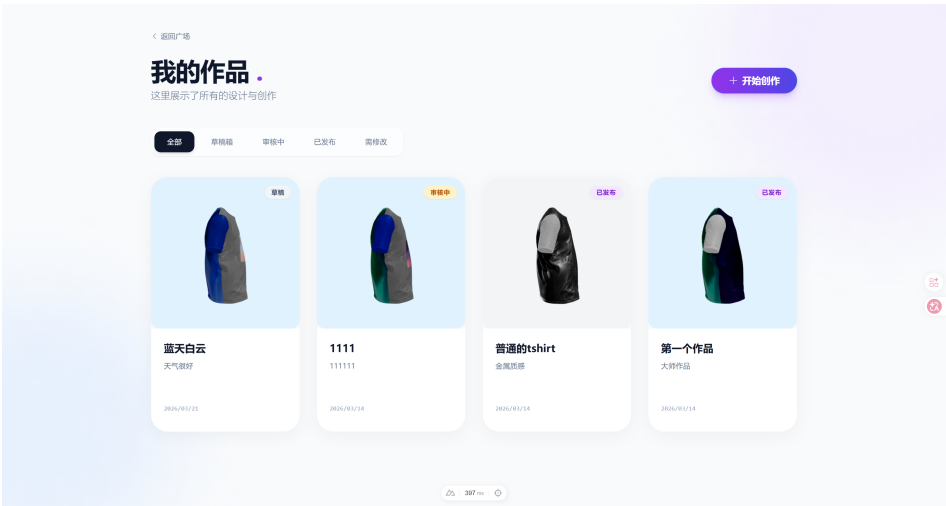


图 5.6 作品管理

5.2.6 后台管理模块实现

后台管理模块用于管理员对平台资源进行统一治理，涵盖管理员登录、用户管理、模型管理和作品审核。后端通过“认证中间件+管理员权限中间件”实现访问控制。

表 5.6 后台管理模块接口

功能说明	接口 API	Http 方法
管理员登录	/api/admin/login	POST
用户管理（查/增/改/删）	/api/admin/users 及子路径	GET/POST/DELETE
用户密码重置	/api/admin/users/:id/password	POST
模型管理（查/增/改/删）	/api/admin/models 及子路径	GET/POST/DELETE
模型文件上传	/api/admin/models/upload	POST
作品审核列表	/api/admin/works	GET
审核处理	/api/admin/works/:id/review	POST

该模块实现过程如下：

- (1) 管理员登录后校验角色为管理员角色；
- (2) 后台接口统一要求 JWT 且通过管理员权限校验；
- (3) 用户管理支持分页检索、角色维护、密码重置；
- (4) 模型管理支持文件上传至对象存储并同步写入数据库；
- (5) 作品审核仅允许“已提交”状态进入审核流程；
- (6) 审核通过将作品发布，审核驳回需填写原因并返回驳回态；
- (7) 前端管理端根据审核结果刷新待审列表与历史记录，界面如图 5.7、图 5.8、图 5.9所示。

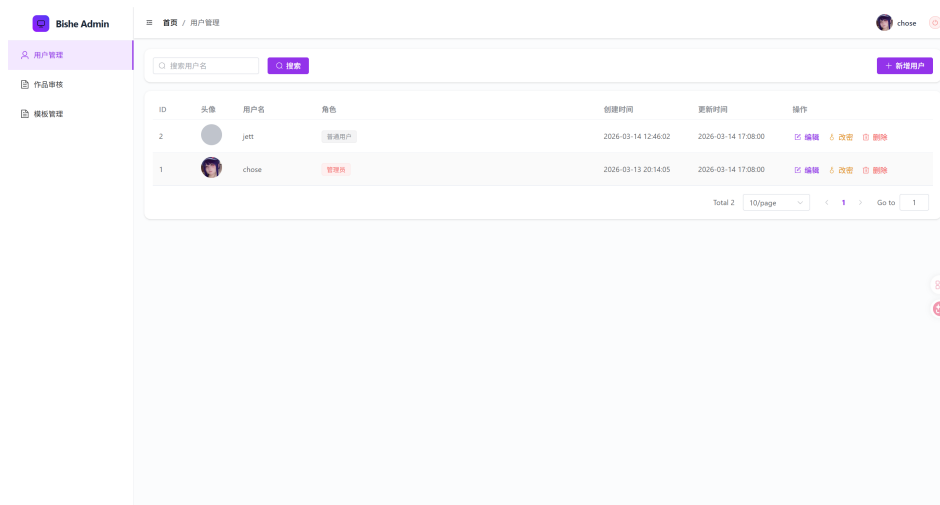


图 5.7 用户管理

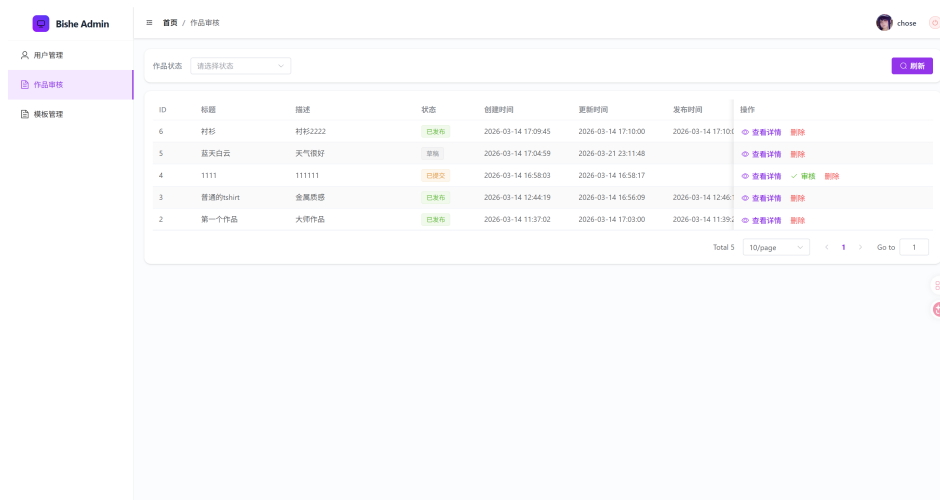


图 5.8 作品审核

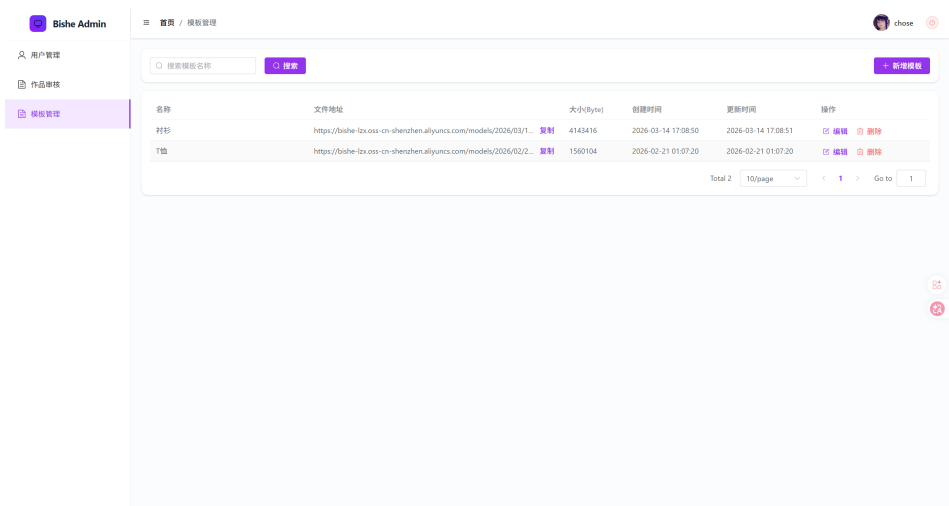


图 5.9 模型管理

6 系统测试

本章主要对系统的核心功能进行黑盒测试，采用“按模块设计用例 + 关键路径优先”的方式覆盖主流程、边界场景与异常处理。测试过程中重点关注：功能正确性、交互完整性、输入校验、权限控制、错误提示与数据一致性。

6.1 功能测试

6.1.1 用户模块测试

用户模块主要覆盖注册登录、个人信息维护、账号安全与会话状态等功能。

(1) 注册与登录

测试用例如表 6.1 所示。

表 6.1 注册与登录测试用例

用例 ID	前置条件	测试步骤	期望结果
001	未登录状态；用户名为未注册状态。	1. 进入注册页，输入合法信息并提交； 2. 注册成功后返回登录页，使用新账号登录； 3. 退出登录后再次登录，验证会话恢复与登录态保持逻辑。	1. 注册信息校验正确（必填项提示明确、格式错误提示准确）； 2. 注册成功后账号可正常登录；登录失败时提示原因（密码错误/账号不存在等）； 3. 登录后进入正确的首页/个人中心，用户信息与权限展示正常。

注册与登录功能测试过程中，系统能够对账号不存在、密码错误等异常输入给出明确的错误提示，并在认证通过后正确跳转并展示登录态下的页面内容；其中，图 6.1 展示了登录失败时的提示信息，图 6.2 展示了登录成功后的页面与会话状态。

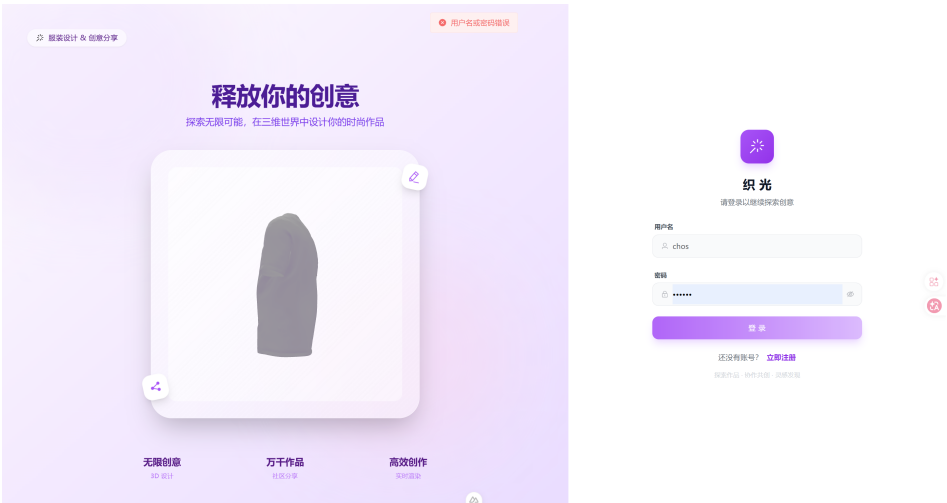


图 6.1 注册与登录失败提示



图 6.2 注册与登录成功页面

(2) 个人信息与资料维护

测试用例如表 6.2所示。

表 6.2 个人信息与资料维护测试用例

用例 ID	前置条件	测试步骤	期望结果
002	已登录。	1. 进入个人中心，修改昵称、头像、密码等资料并保存； 2. 刷新页面或重新登录，检查资料是否持久化； 3. 测试修改密码是否生效； 4. 退出登录，验证会话失效与受限资源访问。	1. 保存后即时生效且刷新后不丢失； 2. 对不合法输入进行拦截或提示，避免写入异常数据； 3. 退出登录后 token/session 失效，访问受保护页面会被拦截并重定向到登录页。

个人信息展示与修改密码校验效果分别如图 6.3、图 6.4所示。

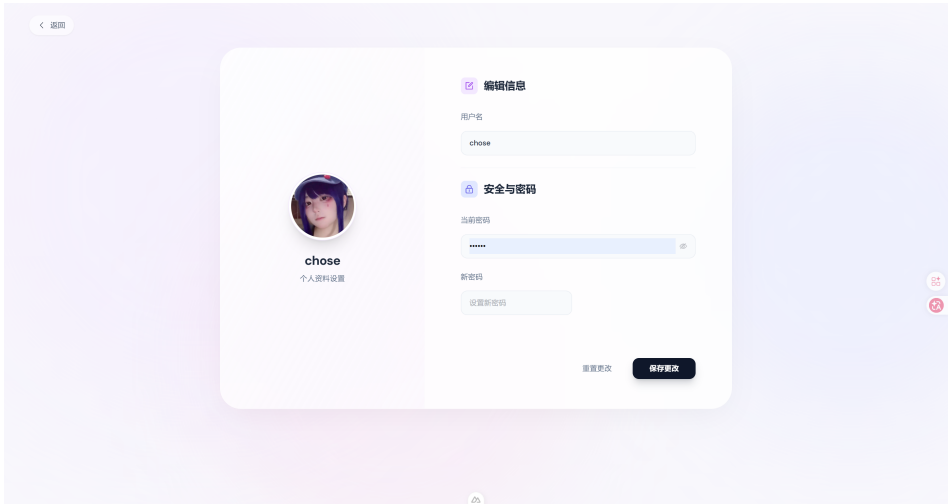


图 6.3 个人信息展示

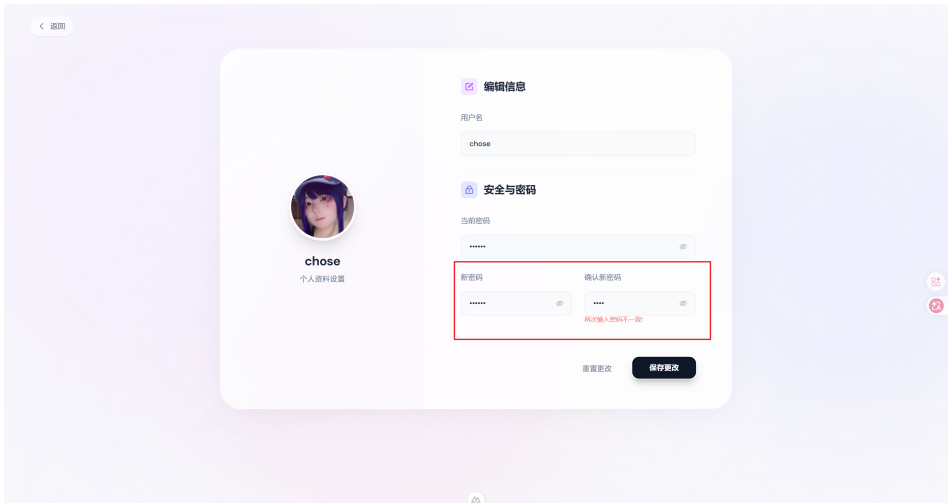


图 6.4 修改密码校验

6.1.2 3D 服装定制模块测试

3D 服装定制模块主要覆盖“选择款式—参数调整—3D 预览—保存”的完整链路，并验证渲染交互与数据保存的正确性。

(1) 进入定制与资源加载

测试用例如表 6.3所示。

表 6.3 进入定制与资源加载测试用例

用例 ID	前置条件	测试步骤	期望结果
003	已登录（若该模块需要登录）；网络稳定。	1. 从首页/功能入口进入 3D 定制页面； 2. 观察模型、材质、纹理加载过程与加载提示。	1. 进入页面后可预览 3D 服装； 2. 加载完成后模型显示完整，无明显缺面/贴图错误。

模型广场展示效果如图 6.5所示。

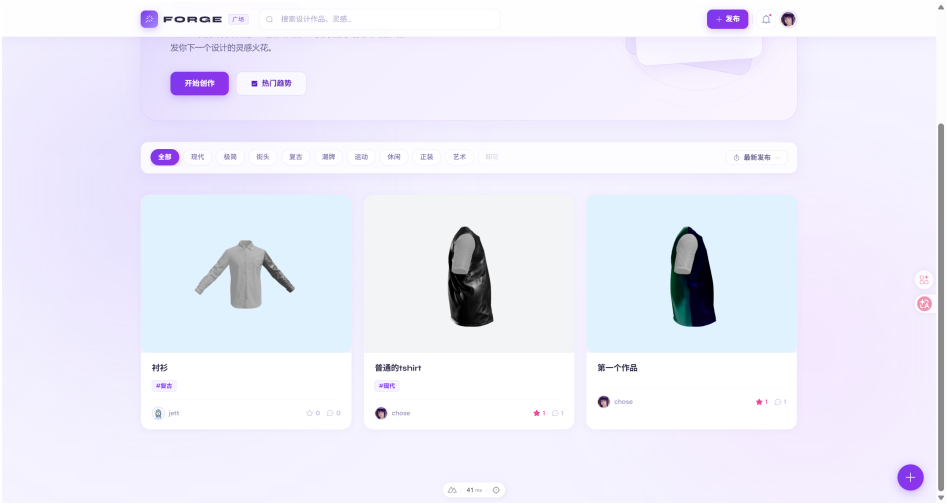


图 6.5 模型广场展示

(2) 款式与属性配置

测试用例如表 6.4所示。

表 6.4 款式与属性配置测试用例

用例 ID	前置条件	测试步骤	期望结果
004	3D 模型已加载完 成。	1. 选择不同服装款式； 2. 调整颜色、面料/材质、型，显示效果与选择项一 图案等属性，观察 3D 视 图实时变化。	1. 属性变更即时反映到模 致； 2. 参数边界有合理限制 (最小/最大值、步进、非 法输入拦截)。

模型选择与编辑成功效果分别如图 6.6、图 6.7所示。

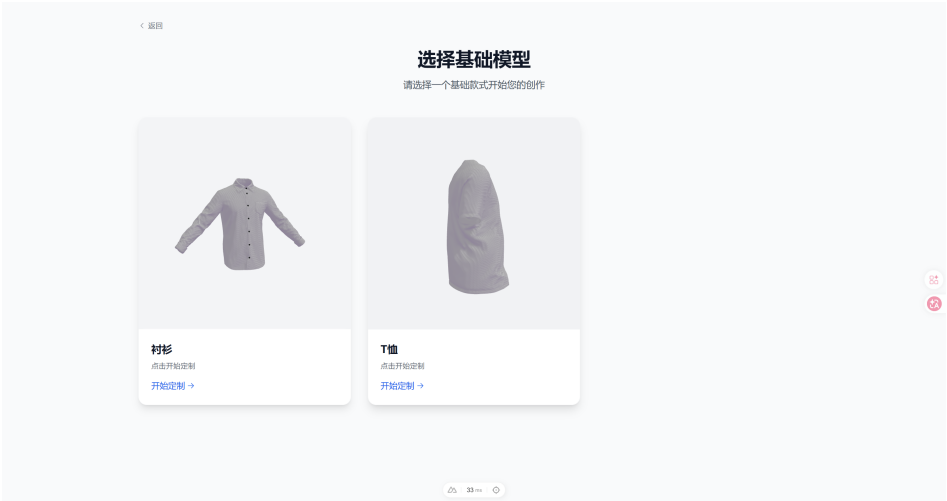


图 6.6 模型选择展示

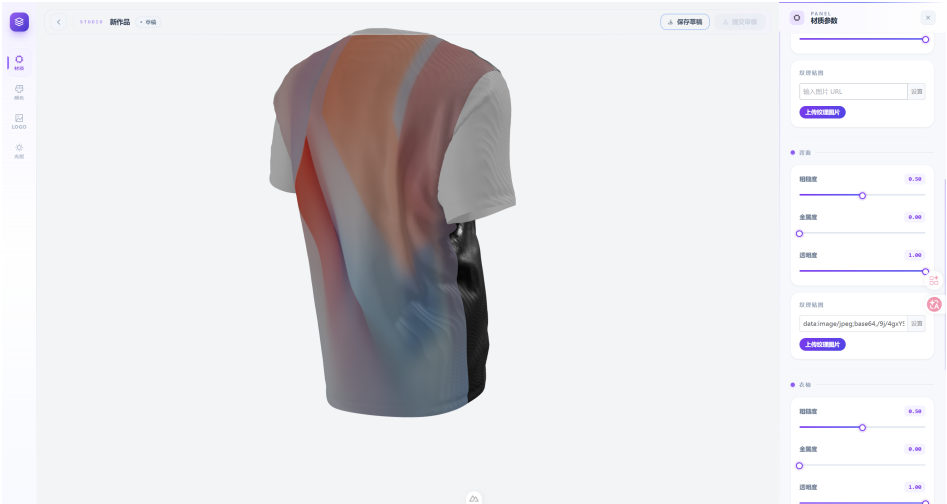


图 6.7 模型编辑成功展示

(3) 3D 交互与预览验证

测试用例如表 6.5所示。

表 6.5 3D 交互与预览验证测试用例

用例 ID	前置条件	测试步骤	期望结果
005	模型可交互。	1. 测试旋转、缩放、平移、重置视角等交互； 2. 切换不同视角，检查渲染是否稳定、帧率是否可接受； 3. 在不同分辨率下查看布局是否错位。	1. 交互响应顺畅；重置视角可恢复默认状态； 2. 关键组件（按钮/面板）不遮挡核心操作，布局自适应正常。

模型交互展示如图 6.8所示。



图 6.8 模型交互展示

(4) 方案保存与后续流转

测试用例如表 6.6所示。

表 6.6 方案保存与后续流转测试用例

用例 ID	前置条件	测试步骤	期望结果
006	已完成一组定制参数。	1. 点击保存，存储草稿并提交审核； 2. 进入“我的作品”查看 并正确回显； 保存记录是否存在，参数是否一致； 3. 删除/编辑保存记录，检查更新是否正确。	1. 保存成功提示明确；保存后数据可在列表中查询 2. 编辑/删除操作生效且不影响其他记录。

作品状态展示如图 6.9所示。

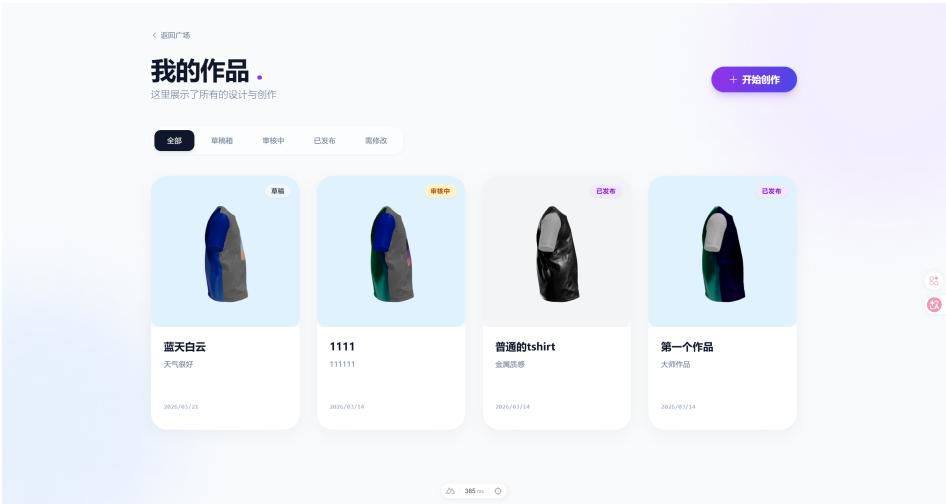


图 6.9 作品状态展示

6.1.3 AI 虚拟试穿模块测试

AI 虚拟试穿模块主要覆盖“输入准备—提交生成—结果展示—历史管理”的流程，并重点验证任务状态、异常提示与结果可用性。

(1) 试穿任务创建

测试用例如表 6.7所示。

表 6.7 试穿任务创建测试用例

用例 ID	前置条件	测试步骤	期望结果
007	已登录；具备可选服装。	1. 进入作品详情页面，选择模特性别及场景； 2. 点击“开始试穿/生成”提交任务。	1. 提交后显示任务状态（排队），并防止重复提交造成多任务堆积（如按钮置灰）。

AI 试穿创建任务页面如图 6.10所示。

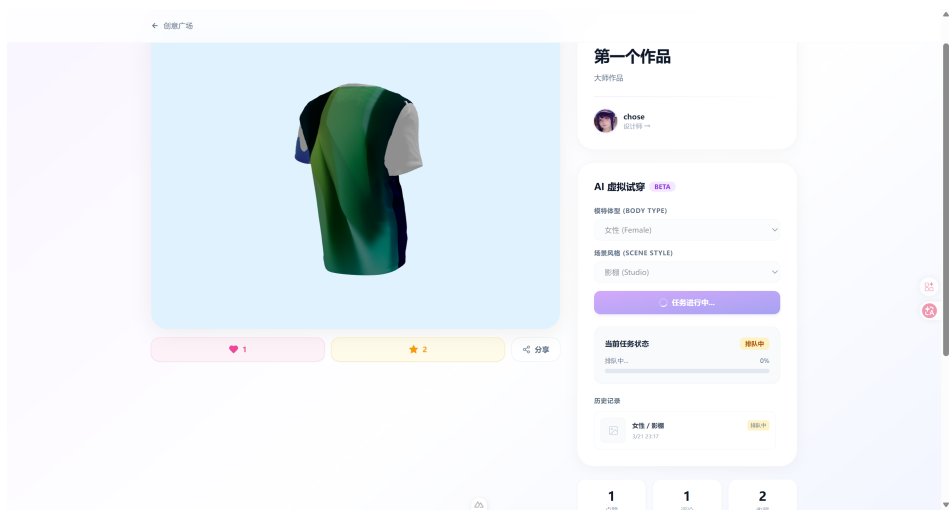


图 6.10 AI 试穿创建任务展示

(2) 生成过程与结果展示
测试用例如表 6.8所示。

表 6.8 生成过程与结果展示测试用例

用例 ID	前置条件	测试步骤	期望结果
008	任务已提交。	1. 观察生成进度提示与历史记录展示； 2. 生成完成后查看试穿图； 3. 测试下载/再次生成等操作。	1. 生成完成后自动刷新或提供可操作入口，不出现“生成完成但页面不更新”的问题； 2. 结果可正常预览与下载；失败时返回可理解的错误原因与建议。

AI 生成结果如图 6.11所示。

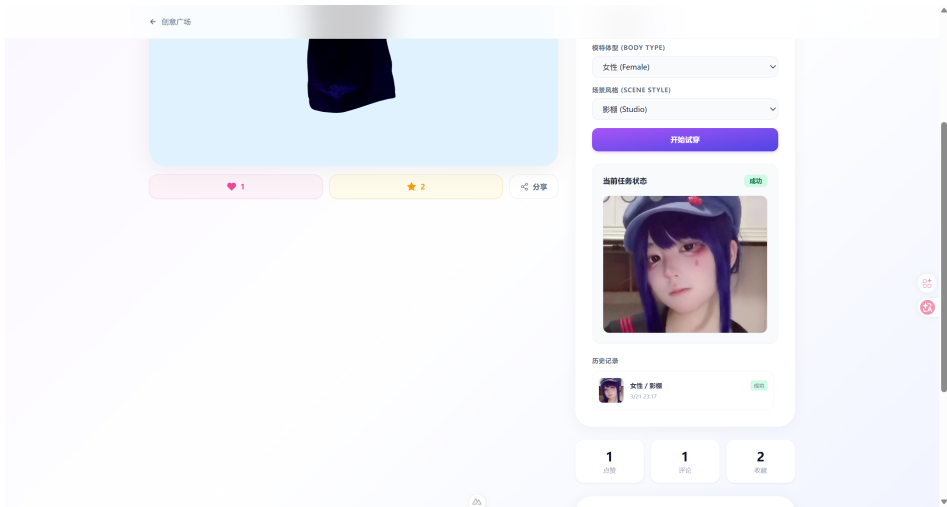


图 6.11 AI 试穿生成结果

6.1.4 创意广场模块测试

创意广场模块主要覆盖内容浏览、搜索筛选、互动行为等功能，验证信息流展示与交互一致性。

(1) 内容浏览与详情查看

测试用例如表 6.9所示。

表 6.9 内容浏览与详情查看测试用例

用例 ID	前置条件	测试步骤	期望结果
009	已登录。	1. 进入创意广场，浏览推荐/最新等列表； 2. 分页加载，检查加载提示与重复数据问题； 3. 进入任意作品详情，查看图片/3D 展示（如有）、作者信息、标签等。	1. 列表加载顺序与规则正确；详情信息与列表项一致； 2. 图片/资源加载失败有占位与重试，不影响主流看图片/3D 展示（如有）、作者信息、标签等。

创意广场展示如图 6.12所示。

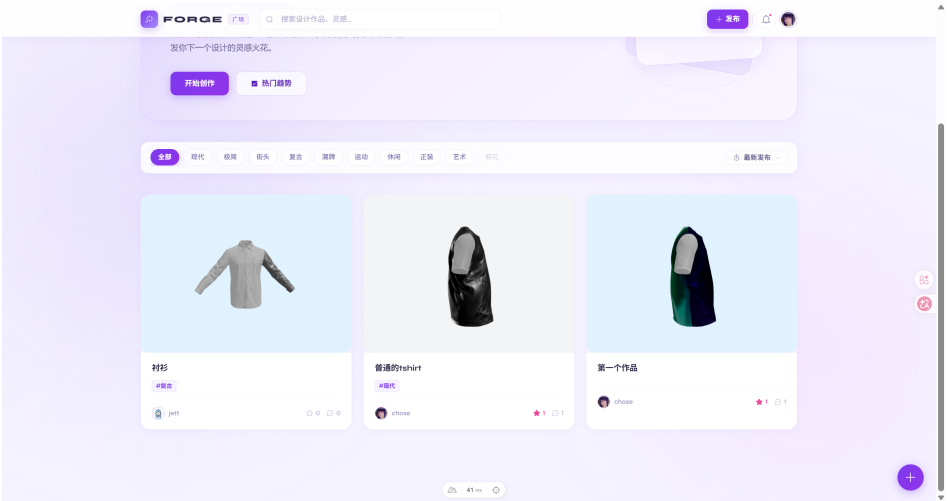


图 6.12 创意广场展示

(2) 搜索、筛选与排序

测试用例如表 6.10所示。

表 6.10 搜索、筛选与排序测试用例

用例 ID	前置条件	测试步骤	期望结果
010	已登录；进入创意广场列表页。	1. 使用关键词搜索作品/用户； 2. 使用标签/风格/时间等筛选条件组合查询； 3. 清空筛选条件，检查列表是否恢复默认。	1. 搜索结果与筛选条件一致； 2. 条件切换后列表更新及时，不出现条件叠加错误。

搜索加载展示如图 6.13所示。



图 6.13 搜索加载展示

(3) 互动与反馈

测试用例如表 6.11所示。

表 6.11 互动与反馈测试用例

用例 ID	前置条件	测试步骤	期望结果
011	已登录；处于作品列表/作品详情或作者主页页面。	1. 点赞/收藏/评论/分享/关注作者（按系统实现）； 2. 取消点赞/取消关注等反向操作，验证状态回滚。	1. 互动状态即时更新并在刷新后保持一致； 2. 不允许的操作有明确提示（未登录、无权限、内容违规等）。

作品交互与作者主页页面分别如图 6.14、图 6.15所示。

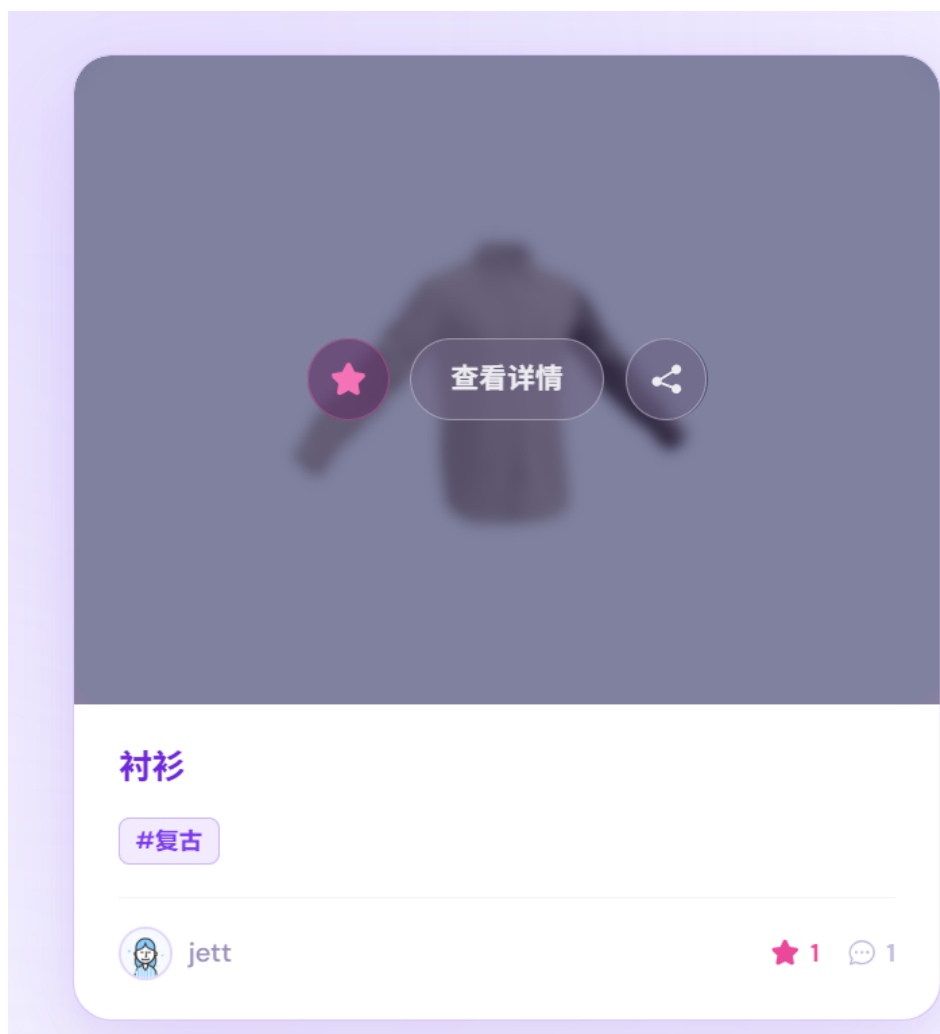


图 6.14 作品交互展示

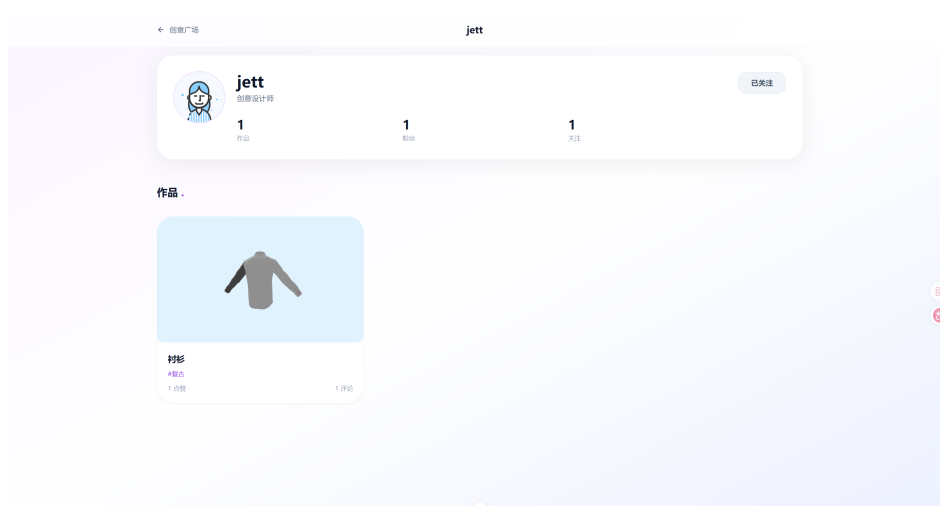


图 6.15 作者主页展示

6.1.5 作品管理模块测试

作品管理模块面向个人创作资产，主要覆盖作品创建、编辑、发布状态管理等流程，验证数据一致性与权限控制。

(1) 作品创建与保存

测试用例如表 6.12所示。

表 6.12 作品创建与保存测试用例

用例 ID	前置条件	测试步骤	期望结果
012	已登录。	1. 进入“我的作品”，新建作品； 2. 保存为草稿，提交审核，验证发布流程是否正确。	1. 必填字段校验准确；保存成功后列表中可见该作品； 2. 回显内容与保存内容一致（含图片、标签、关联资源）。

“我的作品”展示如图 6.16所示。

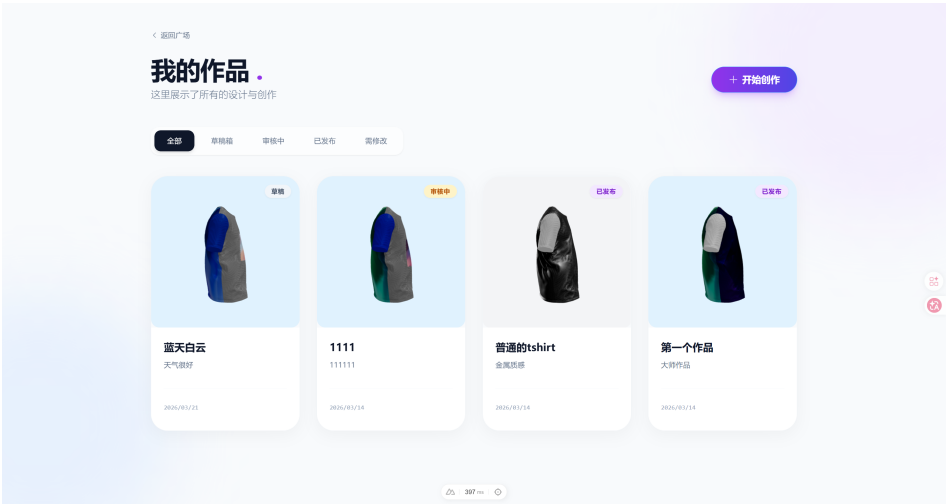


图 6.16 我的作品展示

(2) 作品编辑、发布与撤回

测试用例如表 6.13所示。

表 6.13 作品编辑、发布与撤回测试用例

用例 ID	前置条件	测试步骤	期望结果
013	作品已存在。	1. 编辑作品信息并保存， 2. 发布作品到创意广场 3. 撤回/下架作品（如有）， 验证广场端不可见且个人端状态正确。	1. 发布状态切换正确；不同状态下可执行的操作按钮符合权限（如草稿可编辑，已发布可下架等）。

作品状态展示如图 6.17所示。

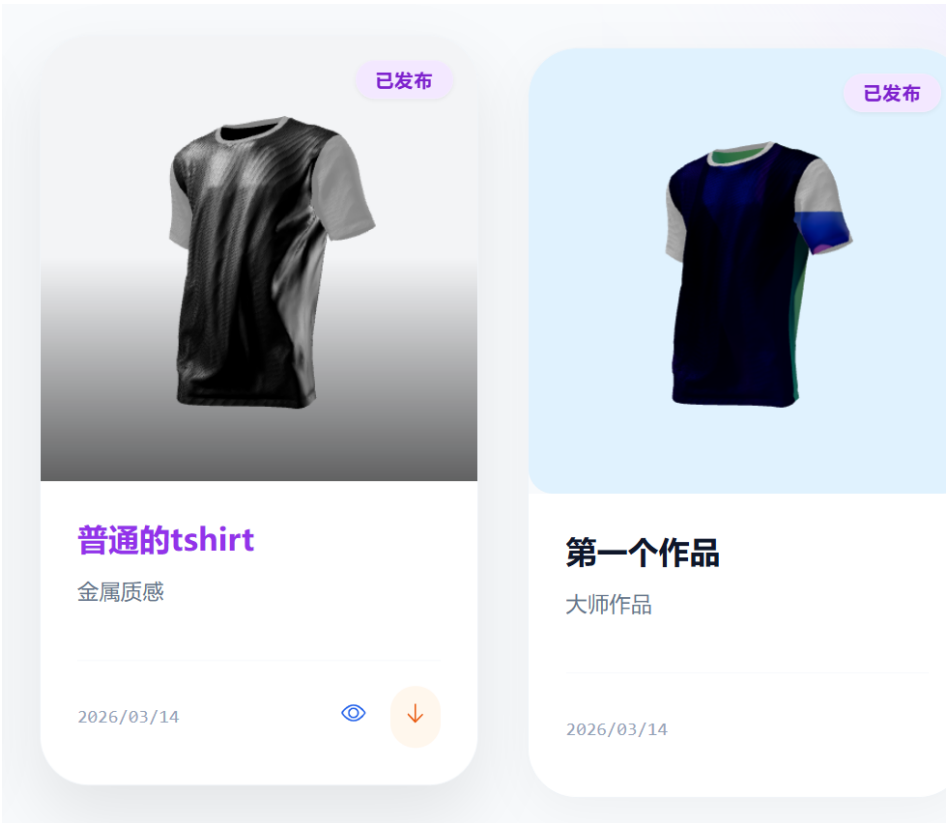


图 6.17 作品状态展示

6.1.6 后台管理模块测试

后台管理模块主要覆盖管理员登录、用户管理、作品审核、模型管理等功能，验证管理操作的安全性及可追溯性。

(1) 管理员登录与权限控制

测试用例如表 6.14所示。

表 6.14 管理员登录与权限控制测试用例

用例 ID	前置条件	测试步骤	期望结果
014	管理员账号已配置。	1. 进入后台登录页，使用管理员账号登录； 2. 使用普通用户账号尝试访问后台路由/页面，验证访问限制； 3. 退出登录后再次访问后台页面，验证会话失效。	1. 仅管理员可访问后台；非管理员访问会被拦截并提示无权限或跳转登录。

后台登录拦截与登录成功页面分别如图 6.18、图 6.19所示。

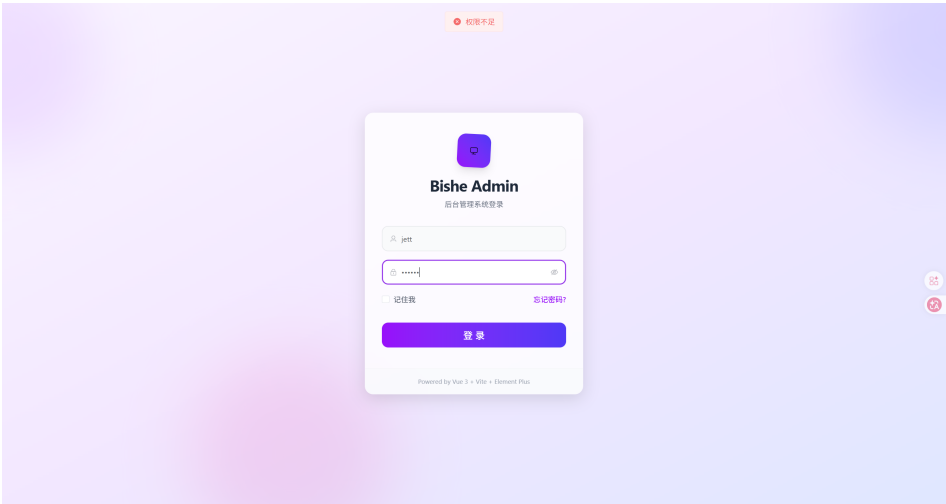


图 6.18 后台登录拦截展示

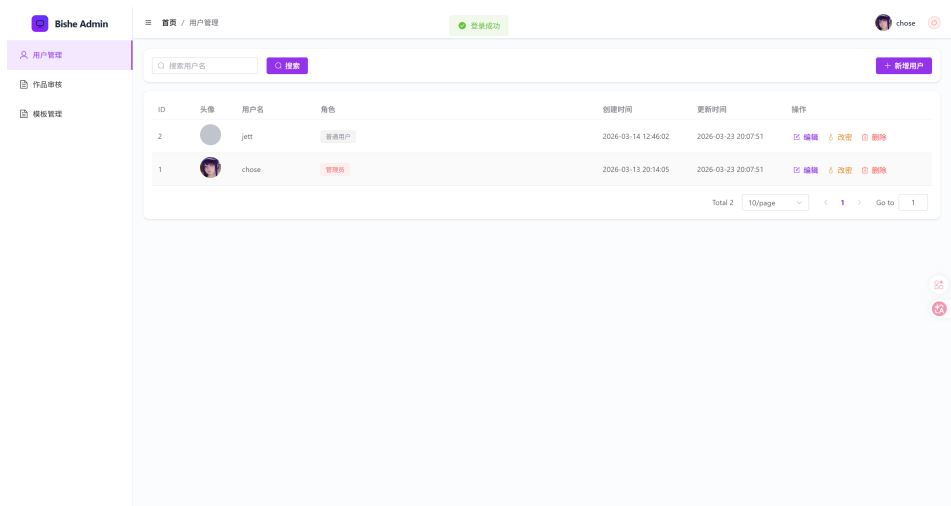


图 6.19 后台登录成功展示

(2) 用户管理

测试用例如表 6.15所示。

表 6.15 用户管理测试用例

用例 ID	前置条件	测试步骤	期望结果
015	已登录后台管理端，具备管理员权限。	1. 查询用户列表； 2. 执行修改信息/密码/删除等管理动作，观察实际效果。	1. 管理操作有二次确认与结果提示；列表状态刷新正确。

用户列表与修改用户信息页面分别如图 6.20、图 6.21所示。

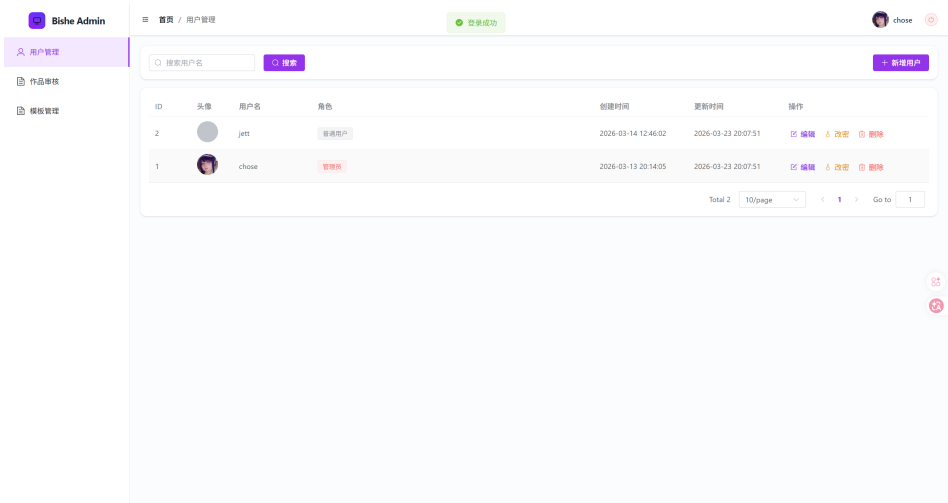


图 6.20 用户列表展示

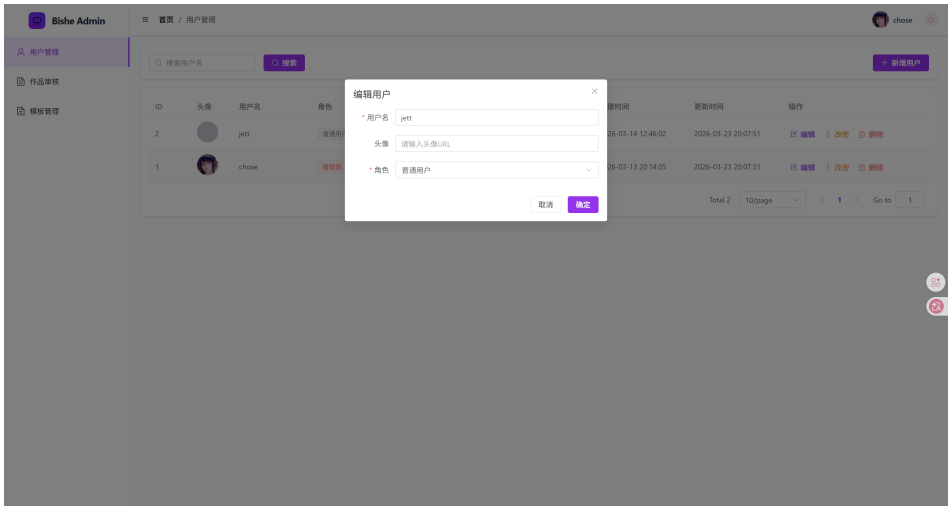


图 6.21 修改用户信息展示

(3) 作品审核

测试用例如表 6.16所示。

表 6.16 作品审核测试用例

用例 ID	前置条件	测试步骤	期望结果
016	已登录后台管理端；存在待审核作品。	1. 审核作品内容：通过、驳回、下架，检查前台展示变化。	1. 审核状态与前台可见性一致；驳回原因可记录并对作者可见（如系统支持）。

审核作品页面如图 6.22所示。



图 6.22 审核作品展示

(4) 模型管理

测试用例如表 6.17所示。

表 6.17 模型管理测试用例

用例 ID	前置条件	测试步骤	期望结果
017	已登录后台管理端；具备模型管理权限。	1. 查询模型列表； 2. 上传模型信息。	1. 模型列表显示上传的模型信息； 2. 上传模型信息成功后，模型列表中可见该模型。

模型列表与上传模型页面分别如图 6.23、图 6.24所示。

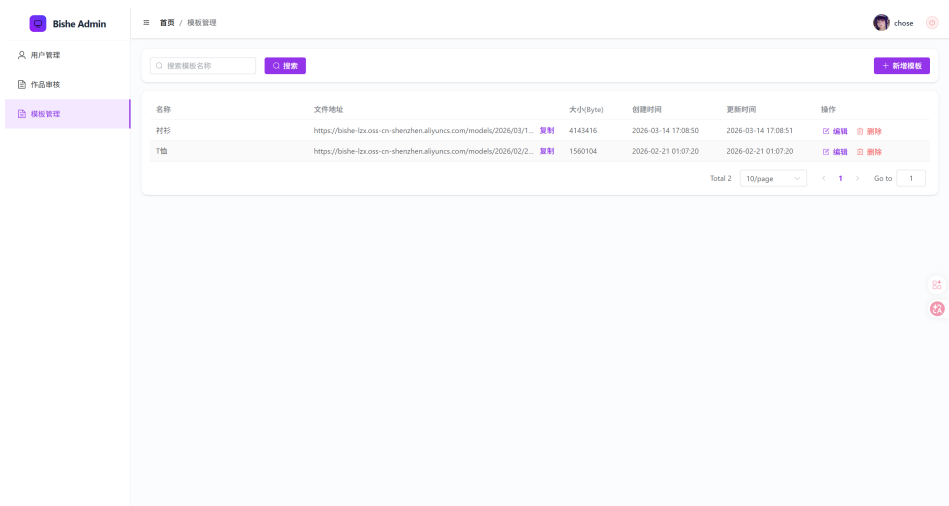


图 6.23 模型列表展示

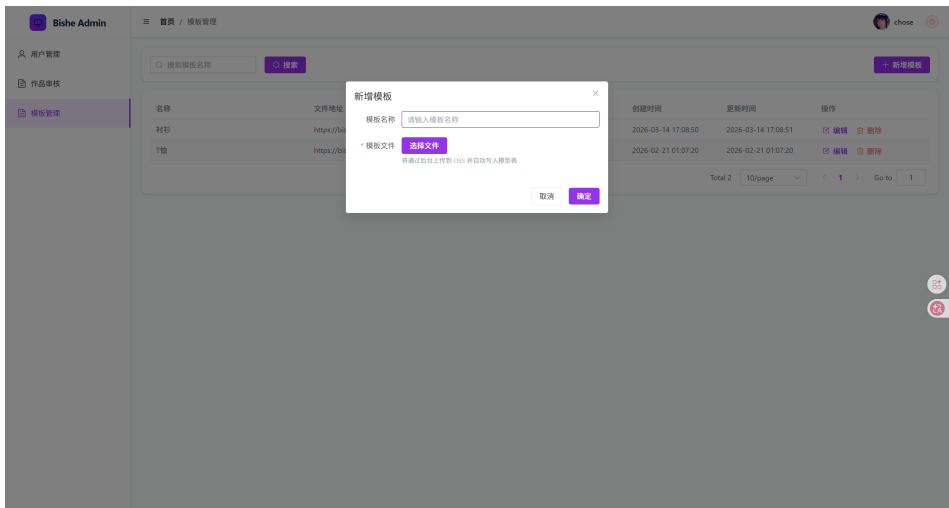


图 6.24 上传模型展示

7 总结与展望

7.1 总结

本课题围绕“融合 3D 定制与 AI 虚拟试穿的服装创意分享平台”的设计与实现展开研究，面向普通用户提供从创意生成、3D 服装定制、AI 试穿展示到作品发布分享的一体化流程，并配套实现后台管理系统以支撑平台内容治理与资源维护。通过对需求分析、系统设计、核心功能实现与测试验证的完整闭环，平台能够满足用户个性化创作与社区分享的实际需求。

本文的主要工作与成果可归纳如下。

- (1) 完成系统需求分析与总体架构设计。结合平台“创作—展示—传播”的业务特点，对普通用户与管理员两类角色进行功能划分，明确用户模块、3D 服装定制模块、AI 虚拟试穿模块、创意广场模块、作品管理模块与后台管理模块等核心功能，并在此基础上完成系统整体架构与数据流转设计，为后续开发提供清晰边界与实现路径。
- (2) 实现 3D 服装定制与可视化交互能力。定制模块支持服装款式选择与属性调整（如颜色、材质、纹理等），并提供旋转、缩放、视角重置等交互操作，使用户能够在设计阶段获得直观、可控的视觉反馈，降低设计门槛并提升创作体验。
- (3) 实现 AI 虚拟试穿能力并与作品资产联动。平台支持用户在作品详情中发起试穿任务，展示任务状态与生成结果，提供下载与再次生成等操作；同时将试穿结果纳入作品资产体系，形成“定制—试穿—发布”的闭环链路，提升作品表达的真实感与传播价值。
- (4) 实现创意广场与作品管理功能，形成内容发布与互动体系。创意广场提供作品浏览、搜索筛选与互动能力（点赞、收藏、关注等），作品管理模块支持作品创建、编辑、提审、发布/下架等状态流转，保证内容发布流程可控并支持社区侧展示与传播。
- (5) 实现后台管理系统，支撑资源与内容治理。后台管理模块提供管理员登录与权限控制，支持用户管理、作品审核与模型管理等操作，实现对平台内容与资源的统一维护，为平台运行提供基础保障。

7.2 展望

尽管平台已实现论文目标并能够稳定完成主要业务流程，但在模型资源丰富度、试穿效果质量、性能与工程化能力等方面仍有提升空间，后续可从以下方向进一步完善。

- (1) 提升 3D 定制能力与资源体系。扩充服装模型与面料材质库，支持更多服装品类与部件组合；引入更细粒度的参数化版型与布料物理模拟，提高定制的真实性与可玩性；在移动端与低性能设备上进一步优化渲染资源加载与交互性能。
- (2) 提升 AI 试穿效果与稳定性。进一步优化人像预处理与服装约束策略，提升对不同姿态、光照、遮挡场景的鲁棒性；引入更精细的控制条件（如人体关键点、分割掩码、深度信息等）提升服装贴合与纹理一致性；完善任务队列与失败重试机制，提升生成链路在高并发场景下的稳定性。
- (3) 完善内容生态与个性化推荐。结合用户兴趣与互动行为构建推荐策略，提升创意广场内容分发效率；增加作品标签体系与主题活动机制，促进优质内容沉淀；完善评论审核与举报流程，提升社区治理能力。
- (4) 强化安全与工程化能力。进一步细化权限模型与接口访问控制，完善日志审计与异常监控；在部署层面引入更标准化的 CI/CD、自动化回归测试与性能压测流程；针对关键接口与资源加载链路进行缓存与限流优化，提升系统可用性与可维护性。

参考文献

- [1] 吴一文, 任武, 鲁毅, 温文怡. 生成式人工智能 (AIGC) 技术研究综述[J]. 中国图象图形学报, 2023, 28(10):3001–3015.
- [2] Han X, Wu Z, Wu Z, Yu R, Davis L S. VITON: An Image-Based Virtual Try-On Network [C]. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018: 7543–7552.
- [3] Wang B, Zheng H, Liang X, Chen Y, Lin L, Yang M. Toward Characteristic-Preserving Image-Based Virtual Try-On Network[C]. In: Proceedings of the European Conference on Computer Vision (ECCV), 2018: 589–604.
- [4] Choi S, Park S, Lee M, Choo J. VITON-HD: High-Resolution Virtual Try-On via Misalignment-Aware Normalization[C]. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021: 14131–14140.
- [5] 曹思祺, 郑德付, 刘沛津, 江涛. 基于深度学习的图像虚拟试衣技术综述[J]. 计算机应用, 2023, 43(4):993–1006.
- [6] LeCun Y, Bengio Y, Hinton G. Deep Learning[J]. Nature, 2015, 521(7553):436–444.
- [7] Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y. Generative Adversarial Networks[J]. Communications of the ACM, 2020, 63(11):139–144.
- [8] Gundogdu E, Constantin V, Seifoddini A, Dang M, Salzmann M, Fua P. GarNet: A Two-Stream Network for Fast and Accurate 3D Cloth Draping[C]. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019: 8739–8748.
- [9] Yildirim E, Sarafianos N, Katircioglu I, Black M J, Tung T. Detailed Garment Recovery from a Single-View Image[C]. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023: 22974–22983.
- [10] You E, Vue.js Contributors. Vue.js 3: The Progressive JavaScript Framework[EB/OL]. 2020. <https://vuejs.org>.
- [11] Cabello R, Three.js Contributors. Three.js: JavaScript 3D Library[EB/OL]. 2010. <https://threejs.org>.
- [12] 周进, 杨晓峰, 熊明福. 基于 Three.js 的服装 Web3D 可视化系统设计与实现[J]. 计算机工程与设计, 2022, 43(7):2003–2010.
- [13] Loper M, Mahmood N, Romero J, Pons-Moll G, Black M J. SMPL: A Skinned Multi-Person Linear Model[J]. ACM Transactions on Graphics, 2015, 34(6):248:1–248:16.
- [14] Gin Contributors. Gin Web Framework[EB/OL]. 2014. <https://gin-gonic.com>.
- [15] Fielding R T. Architectural Styles and the Design of Network-Based Software Architectures [D]. Irvine: University of California, Irvine, 2000.

- [16] Redis Ltd. Redis: The Open Source, In-Memory Data Store[EB/OL]. 2009. <https://redis.io>.
- [17] Pons-Moll G, Pujades S, Hu S, Black M J. ClothCap: Seamless 4D Clothing Capture and Retargeting[J]. ACM Transactions on Graphics, 2017, 36(4):73:1–73:15.
- [18] Ho J, Jain A, Abbeel P. Denoising Diffusion Probabilistic Models[C]. In: Advances in Neural Information Processing Systems (NeurIPS), volume 33, 2020: 6840–6851.
- [19] Rombach R, Blattmann A, Lorenz D, Esser P, Ommer B. High-Resolution Image Synthesis with Latent Diffusion Models[C]. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022: 10684–10695.
- [20] Zhang L, Rao A, Agrawala M. Adding Conditional Control to Text-to-Image Diffusion Models [C]. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023: 3836–3847.
- [21] Cao Z, Hidalgo G, Simon T, Wei S, Sheikh Y. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021, 43(1):172–186.
- [22] Li P, Xu Y, Wei Y, Yang Y. Self-Correction for Human Parsing[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 44(6):3260–3271.
- [23] Zhao F, Xia Z, Meng K, Jiang N, Lu C, Dong J. M3D-VTON: A Monocular-to-3D Virtual Try-On Network[C]. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021: 13239–13249.
- [24] De Luigi L, Li R, Guillard B, Salzmann M, Fua P. DrapeNet: Garment Generation and Self-Supervised Draping[C]. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023: 1451–1460.
- [25] Wang Z, Bovik A C, Sheikh H R, Simoncelli E P. Image Quality Assessment: From Error Visibility to Structural Similarity[J]. IEEE Transactions on Image Processing, 2004, 13(4):600–612.

致谢

大学的时光一眨眼就过去了，回想起四年的点点滴滴却又是那么漫长，充实。这一路上，我遇到了很多朋友，以及照料我的老师们，感谢他们的支持与指导，才能让我走到今天，真的不甚感激。即将离开校园，也意味着我们不再是学生了，也要成为大家口中的牛马了，但是这路上的经历我都会牢牢记住，哪天累了回想起来，也会会心一笑。唉，真是携来百侣曾游，忆往昔峥嵘岁月稠啊。最后，再次感谢我的老师，朋友们，谢谢你们！

附录

本附录 A 到附录 F 主要说明 5.2 系统功能模块实现的关键源码，包括用户管理、3D 服装定制、AI 虚拟试穿、创意广场、作品管理、后台管理的关键代码。

附录 A 用户管理模块

前端代码

```
<template>
```

```
  <!-- Right Panel: Login Form -->
```

```
    <div class="w-full lg:w-2/5 flex flex-col justify-center items-center p-8 lg:p-16 bg-white relative">
```

```
      <div class="w-full max-w-[400px]">
```

```
        <!-- Header -->
```

```
        <div class="text-center mb-6">
```

```
          <div
```

```
            class="inline-flex justify-center items-center w-[64px] h-[64px] bg-gradient-to-br from-purple-500 to-purple-700 rounded" />
```

```
            <el-icon color="white" size="32">
```

```
              <MagicStick />
```

```
            </el-icon>
```

```
          </div>
```

```
          <h2 class="text-2xl font-bold text-gray-900 mb-2">织 光</h2>
```

```
          <p class="text-gray-500 text-sm">请登录以继续探索创意</p>
```

```
        </div>
```

```
  <!-- Form -->
```

```
  <el-form ref="formRef" :model="form" :rules="rules" size="large" class="space-y-5" @submit="login">
```

```
    <div class="space-y-1">
```

```
      <label class="text-xs font-semibold text-gray-600 ml-1">用户名</label>
```

```
      <el-form-item prop="username" class="!mb-0">
```

```
        <el-input v-model="form.username" placeholder="请输入用户名" :prefix-icon="User" />
```

```
        class="!h-11 custom-input-style" />
```

```
      </el-form-item>
```

```
    </div>
```

```
    <div class="space-y-1">
```

```
      <label class="text-xs font-semibold text-gray-600 ml-1">密码</label>
```

```
      <el-form-item prop="password" class="!mb-0">
```

```
        <el-input v-model="form.password" type="password" placeholder="请输入密码" :prefix-icon="Lock" />
```

```

        show-password class="!h-11 custom-input-style" />
      </el-form-item>
    </div>

    <el-button type="primary"
      class="w-full !h-11 !bg-gradient-to-r !from-[#a855f7] !to-[#d8b4fe] !border-none hover:!color-white"
      style="border-radius: 12px;" :loading="loading" @click="handleSubmit(formRef)">
      {{ isLogin ? '登 录' : '注 册' }}
    </el-button>
  </el-form>

  <div class="text-center mt-6">
    <p class="text-sm text-gray-500">
      {{ isLogin ? "还没有账号? " : "已有账号? " }}
      <button class="text-purple-600 font-bold hover:underline ml-1" @click="toggleMode">
        {{ isLogin ? '立即注册' : '去登录' }}
      </button>
    </p>
  </div>

  <div class="mt-4 text-center">
    <p class="text-xs text-gray-300">探索作品 · 协作共创 · 灵感发现</p>
  </div>
</div>
</div>
</div>
</template>

<script setup lang="ts">
import { User, Lock, MagicStick, EditPen, Share } from '@element-plus/icons-vue'
import { ElMessage } from 'element-plus'
import type { FormInstance, FormRules } from 'element-plus'

```

```
import { useUserStore } from '~/stores/user'
import ModelPreview from '~/components/ModelPreview.vue'

const isLogin = ref(true)
const loading = ref(false)
const formRef = ref<FormInstance>()
const userStore = useUserStore()

const form = reactive({
  username: "",
  password: ""
})

const rules = reactive<FormRules>({
  username: [
    { required: true, message: '请输入用户名', trigger: 'change' },
    { min: 3, message: '用户名长度不能少于3位', trigger: 'blur' }
  ],
  password: [
    { required: true, message: '请输入密码', trigger: 'change' },
    { min: 6, message: '密码长度不能少于6位', trigger: 'blur' }
  ]
})

const toggleMode = () => {
  isLogin.value = !isLogin.value
  formRef.value?.resetFields()
}

const handleSubmit = async (formEl: FormInstance | undefined) => {
  if (!formEl) return
```

```

await formEl.validate(async (valid, fields) => {
  if (valid) {
    loading.value = true
    try {
      const url = isLogin.value ? '/api/login' : '/api/register'
      const res: any = await useRequest(url, { ...form })
      userStore.setAuth({ token: res.token, user: res.user })
      if (res.success) {
        ElMessage.success(`${isLogin.value ? '登录' : '注册'} 成功!`)
        navigateTo('/home')
      } else {
        ElMessage.error(res.message)
      }
    } catch (error) {
      ElMessage.error('请求失败')
    } finally {
      loading.value = false
    }
  }
})
}
</script>

```

后端代码

```
package model
```

```

import (
  "database/sql"
  "fmt"
  "strings"
  "time"

```

```
"github.com/chosetop/bishe_back/internal/config"
```

```
)
```

```
type User struct {
```

```
    ID      int    `json:"id"`
```

```
    Username string `json:"username"`
```

```
    Password string `json:"- "` // 不返回密码
```

```
    Avatar  string `json:"avatar"`
```

```
    Role    int    `json:"role"`
```

```
    CreatedAt time.Time `json:"created_at"`
```

```
    UpdatedAt time.Time `json:"updated_at"`
```

```
}
```

```
// CreateUser 创建用户
```

```
func CreateUser(username, passwordHash string) error {
```

```
    stmt, err := config.DB.Prepare("INSERT INTO users(username, password) VALUES(?, ?)")
```

```
    if err != nil {
```

```
        return err
```

```
    }
```

```
    defer stmt.Close()
```

```
    _, err = stmt.Exec(username, passwordHash)
```

```
    return err
```

```
}
```

```
// GetUserByUsername 根据用户名获取用户
```

```
func GetUserByUsername(username string) (*User, error) {
```

```
    row := config.DB.QueryRow("SELECT id, username, password, COALESCE(avatar, ''), role, created_at
```

```
    var user User
```

```
    err := row.Scan(&user.ID, &user.Username, &user.Password, &user.Avatar, &user.Role, &user.CreatedAt)
```

```
    if err != nil {
```

```
if err == sql.ErrNoRows {
    return nil, nil // 用户不存在
}
return nil, err
}
return &user, nil
}

// GetUserByID 根据ID获取用户
func GetUserByID(id int) (*User, error) {
    row := config.DB.QueryRow("SELECT id, username, COALESCE(avatar, ''), role, created_at, updated_at FROM users WHERE id = ?", id)

    var user User
    err := row.Scan(&user.ID, &user.Username, &user.Avatar, &user.Role, &user.CreatedAt, &user.UpdatedAt)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, nil
        }
        return nil, err
    }
    return &user, nil
}

func CountUsers(username string) (int, error) {
    var (
        where string
        args []any
    )
    if username != "" {
        where = " WHERE username LIKE ?"
        args = append(args, "%"+username+"%")
    }
}
```

```
row := config.DB.QueryRow("SELECT COUNT(1) FROM users"+where, args...)
var total int
if err := row.Scan(&total); err != nil {
    return 0, err
}
return total, nil
}
```

```
func ListUsers(username string, limit, offset int) ([]User, error) {
    var (
        where string
        args []any
    )
    if username != "" {
        where = " WHERE username LIKE ?"
        args = append(args, "%"+username+"%")
    }
    args = append(args, limit, offset)
```

```
rows, err := config.DB.Query(
    "SELECT id, username, COALESCE(avatar, ''), role, created_at, updated_at FROM users"+where+" ORDER BY id",
    args...,
)
if err != nil {
    return nil, err
}
defer rows.Close()
```

```
list := make([]User, 0, limit)
for rows.Next() {
    var u User
```

```
if err := rows.Scan(&u.ID, &u.Username, &u.Avatar, &u.Role, &u.CreatedAt, &u.UpdatedAt); err != nil {
    return nil, err
}
list = append(list, u)
}
if err := rows.Err(); err != nil {
    return nil, err
}
return list, nil
}
```

```
func GetUserAuthByID(id int) (*User, error) {
    row := config.DB.QueryRow("SELECT id, username, password, COALESCE(avatar, ''), role, created_at FROM users WHERE id = ?", id)

    var user User
    err := row.Scan(&user.ID, &user.Username, &user.Password, &user.Avatar, &user.Role, &user.CreatedAt)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, nil
        }
        return nil, err
    }
    return &user, nil
}
```

```
func UpdateUserFields(id int, username *string, avatar *string, role *int) (int64, error) {
    fields := make([]string, 0, 3)
    args := make([]any, 0, 3)

    if username != nil {
        fields = append(fields, "username = ?")
        args = append(args, *username)
    }
}
```

```
}  
if avatar != nil {  
    fields = append(fields, "avatar = ?")  
    args = append(args, *avatar)  
}  
if role != nil {  
    fields = append(fields, "role = ?")  
    args = append(args, *role)  
}  
if len(fields) == 0 {  
    return 0, nil  
}  
  
args = append(args, id)  
query := fmt.Sprintf("UPDATE users SET %s WHERE id = ?", strings.Join(fields, ", "))  
res, err := config.DB.Exec(query, args...)  
if err != nil {  
    return 0, err  
}  
return res.RowsAffected()  
}  
  
func UpdateUserPassword(id int, passwordHash string) (int64, error) {  
    res, err := config.DB.Exec("UPDATE users SET password = ? WHERE id = ?", passwordHash, id)  
    if err != nil {  
        return 0, err  
    }  
    return res.RowsAffected()  
}  
  
func DeleteUser(id int) (int64, error) {  
    res, err := config.DB.Exec("DELETE FROM users WHERE id = ?", id)
```

```
if err != nil {  
    return 0, err  
}  
return res.RowsAffected()  
}
```

附录 B 3D 服装定制模块

前端代码

// 定制页

```
<template>

  <div class="flex h-screen w-full bg-gray-100 overflow-hidden">

    <!-- Left: Feature Navigation -->

    <SideNav v-model:active="activePanel" />


    <!-- Center: 3D Viewer -->

    <div class="flex-1 relative flex flex-col min-w-0">

      <!-- Top Bar -->

      <DesignTopBar
        :work-title="workTitle"
        :work-status="workStatus"
        :model-name="modelName"
        :can-undo="canUndo"
        :can-redo="canRedo"
        :saving="saving"
        :submitting="submitting"
        :is-read-only="isReadOnly"
        :current-work-id="currentWorkId"
        :show-undo-redo="true"
        @back="router.back()"
        @undo="undo"
        @redo="redo"
        @save="handleSave"
        @submit="handleSubmit"
      />


      <!-- Viewer -->

      <div class="flex-1 relative bg-gradient-to-b from-gray-50 to-gray-100">

        <Viewer />

      </div>

    </div>

  </div>
```

```
</div>
</div>

<!-- Right: Properties Panel -->
<transition name="slide-fade">
  <PropertiesPanel v-if="activePanel" :active="activePanel" @close="activePanel = ''" />
</transition>

<!-- Save Dialog -->
<WorkSaveDialog
  v-model="saveDialogVisible"
  :title="workForm.title"
  :description="workForm.description"
  :tags="workForm.tags"
  @confirm="confirmSave"
/>
</div>
</template>

<script setup lang="ts">
import { ref, computed, onMounted, reactive } from 'vue'
import { useRouter, useRoute } from 'vue-router'
import { ElMessage, ElMessageBox } from 'element-plus'
import SideNav from '../..../components/design/SideNav.vue'
import PropertiesPanel from '../..../components/design/PropertiesPanel.vue'
import Viewer from '../..../components/design/Viewer.vue'
import DesignTopBar from '../..../components/design/DesignTopBar.vue'
import WorkSaveDialog from '../..../components/design/WorkSaveDialog.vue'
import { useDesignState } from '../..../composables/useDesignState'
// useWorks is auto-imported

const router = useRouter()
```

```
const route = useRoute()
const { undo, redo, canUndo, canRedo, setConfig, config } = useDesignState()
const { fetchWorkDetail, updateWorkDraft, submitWorkReview } = useWorks()

const activePanel = ref('material')
const saving = ref(false)
const submitting = ref(false)
const saveDialogVisible = ref(false)
const currentWorkId = ref<number | null>(null)
const workTitle = ref("")
const workStatus = ref<number | null>(null)

const workForm = reactive({
  title: "",
  description: "",
  tags: [] as string[]
})

const isReadOnly = computed(() => {
  if (!currentWorkId.value) return false
  return ![1, 4].includes(workStatus.value ?? 1)
})

const modelNames: Record<string, string> = {
  tshirt: '标准 T恤',
  hoodie: '连帽卫衣'
}

const modelName = computed(() => modelNames[config.modelId] || 'Unknown Model')

onMounted(async () => {
  const idParam = route.params.id as string
```

```
const workId = Number(idParam)
if (!isNaN(workId)) {
  currentWorkId.value = workId
  await loadWork(workId)
} else {
  ElMessage.error('无效的作品ID')
  router.push('/design')
}
})

const loadWork = async (id: number) => {
  try {
    const res = await fetchWorkDetail(id)
    if (res.success) {
      const { work, version } = res
      workTitle.value = work.title
      workStatus.value = work.status
      workForm.title = work.title
      workForm.description = work.description
      workForm.tags = Array.isArray(work.tags) ? [...work.tags] : []

      if (version?.params_json) {
        setConfig(version.params_json)
      }
    }
  } catch (error) {
    ElMessage.error('加载作品失败')
    console.error(error)
    router.push('/works')
  }
}
```

```
const handleSave = () => {
  saveDialogVisible.value = true
}

const confirmSave = async (formData: { title: string; description: string; tags: string[] }) => {
  if (!currentWorkId.value) return
  saving.value = true
  try {
    const res = await updateWorkDraft(currentWorkId.value, {
      title: formData.title,
      description: formData.description,
      tags: formData.tags,
      cover_file_id: 100,
      model_file_id: 120,
      params_json: config
    })
    if (res.success) {
      ElMessage.success('保存成功')
      workTitle.value = res.work.title
      workForm.title = formData.title
      workForm.description = formData.description
      workForm.tags = [...formData.tags]
      saveDialogVisible.value = false
    }
  } catch (error) {
    ElMessage.error('保存失败')
    console.error(error)
  } finally {
    saving.value = false
  }
}
```

```

const handleSubmit = async () => {
  if (!currentWorkId.value) return
  try {
    await ElMessageBox.confirm('确定要提交审核吗?提交后将无法修改。','提示', {
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning'
    })
    submitting.value = true
    await submitWorkReview(currentWorkId.value)
    ElMessage.success('提交成功')
    workStatus.value = 2
    router.push('/works')
  } catch {
    // Cancelled or error
  } finally {
    submitting.value = false
  }
}
</script>

```

// 关键组件

```

<template>
  <div ref="container" class="w-full h-full relative overflow-hidden bg-gray-50">
    <div v-if="showLoading" class="absolute inset-0 flex items-center justify-center bg-white/80 z-10 bac
    <div class="flex flex-col items-center gap-2">
      <div class="w-8 h-8 border-4 border-purple-500 border-t-transparent rounded-full animate-spin"></div>
      <span class="text-sm text-gray-500 font-medium">加载中...</span>
    </div>
  </div>
</div>
</template>

```

```
<script setup lang="ts">
import { ref, computed, onMounted, onBeforeUnmount, watch, toRaw } from 'vue'
import { DesignService } from '../services/DesignService'
import { useDesignState } from '../composables/useDesignState'
import type { LogoConfig } from '../types/design'

const container = ref<HTMLElement | null>(null)
const loading = ref(true)
const designService = ref<DesignService | null>(null)
const { config, ensureMaterial, updateLogo, isLoading } = useDesignState()

const showLoading = computed(() => loading.value || isLoading.value)

onMounted(async () => {
  if (!container.value) return

  // Initialize Service
  const service = new DesignService(container.value)
  designService.value = service

  // Initial Load
  await loadModel()

  // Initial Lighting
  service.setLightPreset(config.scene.lightPreset)

  // Initial Materials
  // We need to wait for model to load to know mesh names,
  // but we can also just try to apply what we have.
  applyAllMaterials()
```

```
// Initial Logos
config.logos.forEach(logo => {
  service.addLogo(toRaw(logo))
  renderedLogoIds.add(logo.id)
})

// Add Drag Listeners
if (container.value) {
  container.value.addEventListener('mousedown', onMouseDown)
  window.addEventListener('mousemove', onMouseMove)
  window.addEventListener('mouseup', onMouseUp)
}
})

onBeforeUnmount(() => {
  if (designService.value) {
    designService.value.dispose()
  }
  if (container.value) {
    container.value.removeEventListener('mousedown', onMouseDown)
  }
  window.removeEventListener('mousemove', onMouseMove)
  window.removeEventListener('mouseup', onMouseUp)
})

const draggingLogoId = ref<string | null>(null)

const onMouseDown = (e: MouseEvent) => {
  if (!designService.value) return
  const id = designService.value.pickLogo(e.clientX, e.clientY)
  if (id) {
    draggingLogoId.value = id
  }
}
```

```
    if (designService.value.controls) designService.value.controls.enabled = false
  }
}
```

```
const onMouseMove = (e: MouseEvent) => {
  if (draggingLogoId.value && designService.value) {
    // Find intersection on model
    const hit = designService.value.getIntersection(e.clientX, e.clientY)
    if (hit) {
      const currentLogo = config.logos.find(l => l.id === draggingLogoId.value)
      if (currentLogo) {
        // Update position directly
        updateLogo(draggingLogoId.value, {
          transform: {
            ...currentLogo.transform,
            position: hit.point
          },
          meshName: hit.meshName
        })
      }
    }
  }
}
```

```
const onMouseUp = () => {
  if (draggingLogoId.value) {
    draggingLogoId.value = null
    if (designService.value && designService.value.controls) {
      designService.value.controls.enabled = true
    }
  }
}
```

```
// --- Watchers ---

watch(() => config.modelId, async () => {
  await loadModel()
})

watch(() => config.scene.lightPreset, (preset) => {
  designService.value?.setLightPreset(preset)
})

watch(() => config.materials, (materials) => {
  if (!designService.value) return
  // Apply all changes. DesignService.updateMaterial is efficient enough.
  Object.entries(materials).forEach(([meshName, matConfig]) => {
    designService.value?.updateMaterial(meshName, toRaw(matConfig))
  })
}, { deep: true })

const renderedLogoIds = new Set<string>()

watch(() => config.logos, async (newLogos) => {
  if (!designService.value) return

  const newIds = new Set(newLogos.map(l => l.id))

  // Remove deleted
  for (const id of renderedLogoIds) {
    if (!newIds.has(id)) {
      designService.value.removeLogo(id)
    }
  }
}
```

```
// Detect if we have new logos (which might require texture loading)
const hasNewLogos = newLogos.some(l => !renderedLogoIds.has(l.id))

// Update our set synchronously to prevent race conditions
renderedLogoIds.clear()
newIds.forEach(id => renderedLogoIds.add(id))

if (hasNewLogos) {
  loading.value = true
}

// Add/Update new
const updatePromises = newLogos.map(logo => {
  return designService.value?.addLogo(toRaw(logo))
})

await Promise.all(updatePromises)

if (hasNewLogos) {
  loading.value = false
}

}, { deep: true })

// --- Methods ---

const loadModel = async () => {
  if (!designService.value) return
  loading.value = true
  await designService.value.loadModel(config.modelUrl)
```

```
// Initialize materials in state for the new model
const meshNames = designService.value.getMeshNames()
meshNames.forEach(name => {
  ensureMaterial(name)
})

applyAllMaterials()
loading.value = false
}

const applyAllMaterials = () => {
  if (!designService.value) return
  Object.entries(config.materials).forEach(([name, mat]) => {
    designService.value?.updateMaterial(name, toRaw(mat))
  })
}

</script>

//关键3D类
import * as THREE from 'three'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { DecalGeometry } from 'three/examples/jsm/geometries/DecalGeometry.js'
import type { MaterialConfig, LogoConfig, LightPreset } from '../types/design'
import { LightManager } from './LightManager'

export class DesignService {
  private container: HTMLElement
  private renderer: THREE.WebGLRenderer
  private scene: THREE.Scene
```

```
private camera: THREE.PerspectiveCamera
private controls: OrbitControls
private lightManager: LightManager
private currentModel: THREE.Group | null = null
private raycaster = new THREE.Raycaster()
private logos: Map<string, THREE.Mesh> = new Map() // Map logo ID to Mesh
private textureLoader = new THREE.TextureLoader()
private textureCache: Map<string, THREE.Texture> = new Map()
private rafId: number = 0

constructor(container: HTMLElement) {
  this.container = container

  // Renderer
  this.renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true })
  this.renderer.setSize(container.clientWidth, container.clientHeight)
  this.renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
  this.renderer.shadowMap.enabled = true
  this.renderer.outputColorSpace = THREE.SRGBColorSpace
  this.renderer.toneMapping = THREE.ACESFilmicToneMapping
  container.appendChild(this.renderer.domElement)

  // Scene
  this.scene = new THREE.Scene()

  // Camera
  this.camera = new THREE.PerspectiveCamera(45, container.clientWidth / container.clientHeight, 0.1,
  this.camera.position.set(0, 0.5, 2.5)

  // Controls
  this.controls = new OrbitControls(this.camera, this.renderer.domElement)
  this.controls.enableDamping = true
```

```
this.controls.dampingFactor = 0.05
this.controls.minDistance = 1
this.controls.maxDistance = 5

// Light Manager
this.lightManager = new LightManager(this.scene)
this.lightManager.setPreset('indoor')

// Animation Loop
this.animate = this.animate.bind(this)
this.animate()

// Resize Handler
this.onResize = this.onResize.bind(this)
window.addEventListener('resize', this.onResize)
}

async loadModel(url: string) {
  if (this.currentModel) {
    this.scene.remove(this.currentModel)
    this.disposeModel(this.currentModel)
    this.currentModel = null
  }

  const loader = new GLTFLoader()
  try {
    const gltf = await loader.loadAsync(url)
    this.currentModel = gltf.scene

    // Center and Scale
    const box = new THREE.Box3().setFromObject(this.currentModel)
    const center = box.getCenter(new THREE.Vector3())
```

```
const size = box.getSize(new THREE.Vector3())
const maxDim = Math.max(size.x, size.y, size.z)
const scale = 1.5 / maxDim

this.currentModel.scale.setScalar(scale)
this.currentModel.position.sub(center.multiplyScalar(scale))
this.currentModel.position.y -= 0.2 // Adjust height

this.currentModel.traverse((child) => {
  if ((child as THREE.Mesh).isMesh) {
    child.castShadow = true
    child.receiveShadow = true
  }
})

this.scene.add(this.currentModel)
} catch (error) {
  console.error("Failed to load model:", error)
}

return this.currentModel
}

updateMaterial(meshName: string, config: MaterialConfig) {
  if (!this.currentModel) return

  this.currentModel.traverse((child) => {
    if ((child as THREE.Mesh).isMesh) {
      // If meshName is provided, only update that mesh. Otherwise update all.
      // For simplicity, we assume strict matching or specific generic names logic in caller.
      // Here we check if the mesh name includes the target name (e.g. 'tshirt' includes 'body'?)
      // Or exact match.
```

```
// Let's use exact match or 'all'
if (meshName !== 'all' && child.name !== meshName) return

const mesh = child as THREE.Mesh
// Ensure material is unique to this mesh if not already
if (!Array.isArray(mesh.material)) {
  // Clone to avoid sharing if needed, but usually we want shared for same parts
}

const materials = Array.isArray(mesh.material) ? mesh.material : [mesh.material]

materials.forEach(m => {
  if (m instanceof THREE.MeshStandardMaterial) {
    m.color.set(config.color)
    m.roughness = config.roughness
    m.metalness = config.metalness

    if (config.emissive) m.emissive.set(config.emissive)
    if (config.opacity !== undefined) {
      m.opacity = config.opacity
      m.transparent = config.opacity < 1
    }

    if (config.map) {
      this.textureLoader.load(config.map, (tex) => {
        tex.colorSpace = THREE.SRGBColorSpace
        tex.flipY = false
        m.map = tex
        m.needsUpdate = true
      })
    } else if (config.map === null) {
      m.map = null
    }
  }
})
```

```
    }

    m.needsUpdate = true
  }
})
}
})
}

// Method to identify mesh names for the UI
getMeshNames(): string[] {
  const names: string[] = []
  if (this.currentModel) {
    this.currentModel.traverse((child) => {
      if ((child as THREE.Mesh).isMesh) {
        names.push(child.name)
      }
    })
  }
  return names
}

async addLogo(logo: LogoConfig) {
  if (!this.currentModel) return

  // Optimization: Check if update is needed
  const existingMesh = this.logos.get(logo.id)
  if (existingMesh) {
    const prevConfig = existingMesh.userData.config as LogoConfig
    // Simple deep compare to avoid unnecessary updates
    if (prevConfig && JSON.stringify(prevConfig) === JSON.stringify(logo)) {
      return
    }
  }
}
```

```
    }  
  }  
  
  // Load texture (with caching)  
  let texture = this.textureCache.get(logo.url)  
  if (!texture) {  
    try {  
      texture = await this.textureLoader.loadAsync(logo.url)  
      texture.colorSpace = THREE.SRGBColorSpace  
      this.textureCache.set(logo.url, texture)  
    } catch (e) {  
      console.error('Failed to load texture', e)  
      return  
    }  
  }  
  
  // Remove existing mesh(es) for this ID  
  // Crucial: do this AFTER await (if any) to prevent race conditions leaving ghosts  
  // And ensure we remove ALL instances to clean up any past ghosts  
  this.removeLogo(logo.id)  
  
  // Create Decal Material  
  // We can potentially reuse material if texture hasn't changed, but creating new one is safer for now  
  const material = new THREE.MeshPhongMaterial({  
    map: texture,  
    transparent: true,  
    depthTest: true,  
    depthWrite: false,  
    polygonOffset: true,  
    polygonOffsetFactor: -4,  
    shininess: 30,  
    specular: 0x111111
```

```
}}
```

```
// Find target mesh
```

```
let targetMesh: THREE.Mesh | null = null
```

```
this.currentModel.traverse((child) => {
```

```
  if ((child as THREE.Mesh).isMesh && child.name === logo.meshName) {
```

```
    targetMesh = child as THREE.Mesh
```

```
  }
```

```
}}
```

```
// Fallback: use the first mesh if no name match
```

```
if (!targetMesh) {
```

```
  this.currentModel.traverse((child) => {
```

```
    if (!targetMesh && (child as THREE.Mesh).isMesh) {
```

```
      targetMesh = child as THREE.Mesh
```

```
    }
```

```
  })
```

```
}
```

```
if (targetMesh) {
```

```
  const position = new THREE.Vector3(logo.transform.position.x, logo.transform.position.y, logo.transform.position.z)
```

```
  const orientation = new THREE.Euler(0, 0, logo.transform.rotation)
```

```
  const size = new THREE.Vector3(logo.transform.scale, logo.transform.scale, logo.transform.scale)
```

```
  const geometry = new DecalGeometry(targetMesh, position, orientation, size)
```

```
  const mesh = new THREE.Mesh(geometry, material)
```

```
  mesh.name = `logo-${logo.id}`
```

```
  mesh.userData.config = JSON.parse(JSON.stringify(logo)) // Store deep copy of config
```

```
  this.scene.add(mesh)
```

```
  this.logos.set(logo.id, mesh)
```

```
}
```

```
}

removeLogo(id: string) {
  // Robust removal: keep removing until no object with this name exists
  let mesh: THREE.Object3D | undefined
  while ((mesh = this.scene.getObjectByName(`logo-${id}`))) {
    this.scene.remove(mesh)
    if ((mesh as THREE.Mesh).isMesh) {
      const m = mesh as THREE.Mesh
      m.geometry.dispose()
      if (Array.isArray(m.material)) {
        m.material.forEach(mat => mat.dispose())
      } else {
        (m.material as THREE.Material).dispose()
      }
    }
  }
  this.logos.delete(id)
}

setLightPreset(preset: LightPreset) {
  this.lightManager.setPreset(preset)
}

private animate() {
  this.rafId = requestAnimationFrame(this.animate)
  this.controls.update()
  this.renderer.render(this.scene, this.camera)
}

private onResize() {
  if (!this.container) return
```

```
const width = this.container.clientWidth
const height = this.container.clientHeight
this.camera.aspect = width / height
this.camera.updateProjectionMatrix()
this.renderer.setSize(width, height)
}

private disposeModel(model: THREE.Group) {
  model.traverse((child) => {
    if ((child as THREE.Mesh).isMesh) {
      const mesh = child as THREE.Mesh
      mesh.geometry.dispose()
      const mats = Array.isArray(mesh.material) ? mesh.material : [mesh.material]
      mats.forEach(m => m.dispose())
    }
  })
}

dispose() {
  cancelAnimationFrame(this.rafId)
  this.renderer.dispose()
  this.renderer.forceContextLoss()
  window.removeEventListener('resize', this.onResize)

  // Dispose scene
  this.scene.traverse((child) => {
    if ((child as THREE.Mesh).isMesh) {
      (child as THREE.Mesh).geometry.dispose()
    }
  })
}
```

```

// Helper to get intersection for placement
getIntersection(clientX: number, clientY: number) {
  if (!this.currentModel) return null

  const rect = this.renderer.domElement.getBoundingClientRect()
  const x = ((clientX - rect.left) / rect.width) * 2 - 1
  const y = -((clientY - rect.top) / rect.height) * 2 + 1

  this.raycaster.setFromCamera({ x, y }, this.camera)

  const intersects = this.raycaster.intersectObject(this.currentModel, true)

  if (intersects.length > 0) {
    const hit = intersects[0]
    return {
      point: hit.point,
      normal: hit.face?.normal?.clone().transformDirection(hit.object.matrixWorld) || new THREE.Vector3(),
      meshName: hit.object.name
    }
  }
  return null
}

pickLogo(clientX: number, clientY: number): string | null {
  const rect = this.renderer.domElement.getBoundingClientRect()
  const x = ((clientX - rect.left) / rect.width) * 2 - 1
  const y = -((clientY - rect.top) / rect.height) * 2 + 1

  this.raycaster.setFromCamera({ x, y }, this.camera)

  const logoMeshes = Array.from(this.logos.values())
  const intersects = this.raycaster.intersectObjects(logoMeshes, false)

```

```
if (intersects.length > 0) {  
    // Name is logo-{id}  
    return intersects[0].object.name.replace('logo-', "  
}  
return null  
}  
}
```

后端代码

```
package front
```

```
import (  
    "context"  
    "crypto/rand"  
    "encoding/hex"  
    "encoding/json"  
    "fmt"  
    "net/http"  
    "os"  
    "strconv"  
    "time"  
  
    "github.com/chosetop/bishe_back/internal/config"  
    "github.com/chosetop/bishe_back/internal/model"  
    "github.com/gin-gonic/gin"  
)
```

```
const tryOnQueueKey = "queue:tryon:tasks"
```

```
type CreateWorkInput struct {  
    Title    string    `json:"title" binding:"required"`
```

```

Description string    `json:"description"`
CoverFileID *int64    `json:"cover_file_id"`
ParamsJSON json.RawMessage `json:"params_json" binding:"required"`
ModelFileID *int64    `json:"model_file_id"`
Tags []string    `json:"tags"`
}

```

```

func CreateWork(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
}

```

```

var input CreateWorkInput
if err := c.ShouldBindJSON(&input); err != nil {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": err.Error()})
    return
}
if len(input.ParamsJSON) == 0 {
    c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "params_json不能为空"})
    return
}

```

```

workID, err := model.CreateDesignWork(int64(userID), input.Title, input.Description, input.CoverFileID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建作品失败"})
}

```

```
    return
}

versionID, err := model.CreateDesignVersion(workID, 1, input.ParamsJSON, input.ModelFileID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建版本失败"})
    return
}

_, err = model.UpdateDesignWorkFields(workID, nil, nil, nil, nil, &versionID, nil)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "更新作品失败"})
    return
}

if len(input.Tags) > 0 {
    model.SetWorkTags(workID, input.Tags)
}

work, _ := model.GetDesignWorkByID(workID)
version, _ := model.GetDesignVersionByID(versionID)
c.JSON(http.StatusOK, gin.H{"success": true, "work": work, "version": version})
}

type UpdateWorkInput struct {
    Title      *string    `json:"title"`
    Description *string    `json:"description"`
    CoverFileID *int64     `json:"cover_file_id"`
    ParamsJSON json.RawMessage `json:"params_json"`
    ModelFileID *int64     `json:"model_file_id"`
    Tags       []string   `json:"tags"`
}
```

```

type CreateTryOnTaskInput struct {
    BodyType string    `json:"body_type" binding:"required"`
    SceneStyle string    `json:"scene_style" binding:"required"`
    InputRefs json.RawMessage `json:"input_refs"`
}

func UpdateWork(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }

    workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || workID <= 0 {
        c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
        return
    }

    work, err := model.GetDesignWorkByID(workID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
        return
    }
    if work == nil || work.UserID != int64(userID) {
        c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    }
}

```

```
    return
}
if work.Status == model.DesignStatusSubmitted || work.Status == model.DesignStatusApproved || work.
c.JSON(http.StatusOK, gin.H{"success": false, "message": "当前状态不可编辑"})
return
}

var input UpdateWorkInput
if err := c.ShouldBindJSON(&input); err != nil {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": err.Error()})
    return
}

var status *int
if work.Status == model.DesignStatusRejected {
    s := model.DesignStatusDraft
    status = &s
}

_, err = model.UpdateDesignWorkFields(workID, input.Title, input.Description, input.CoverFileID, statu
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "更新作品失败"})
    return
}

var versionID int64
if work.CurrentVersionID != nil {
    versionID = *work.CurrentVersionID
} else {
    latest, err := model.GetLatestDesignVersionByWorkID(workID)
    if err != nil || latest == nil {
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取版本失败"})
```

```

    return
}
versionID = latest.ID
_, _ = model.UpdateDesignWorkFields(workID, nil, nil, nil, &versionID, nil)
}

clearSubmittedAt := work.Status == model.DesignStatusRejected || len(input.ParamsJSON) > 0 || input.M
if clearSubmittedAt {
    _, err = model.UpdateDesignVersionFields(versionID, input.ParamsJSON, input.ModelFileID, true)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "更新版本失败"})
        return
    }
}

work, _ = model.GetDesignWorkByID(workID)
var version *model.DesignVersion
if work != nil && work.CurrentVersionID != nil {
    version, _ = model.GetDesignVersionByID(*work.CurrentVersionID)
}

if input.Tags != nil {
    model.SetWorkTags(workID, input.Tags)
}

c.JSON(http.StatusOK, gin.H{"success": true, "work": work, "version": version})
}

func SubmitWork(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
    }
}

```

```
    return
}
userID, ok := userIDVal.(int)
if !ok || userID <= 0 {
    c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
    return
}

workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
if err != nil || workID <= 0 {
    c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
    return
}

work, err := model.GetDesignWorkByID(workID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}
if work == nil || work.UserID != int64(userID) {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    return
}
if work.Status != model.DesignStatusDraft && work.Status != model.DesignStatusRejected {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "当前状态不可提交"})
    return
}
if work.CurrentVersionID == nil {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "缺少版本信息"})
    return
}
```

```

status := model.DesignStatusSubmitted
now := time.Now()
_, err = model.UpdateDesignWorkFields(workID, nil, nil, nil, &status, nil, nil)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "提交失败"})
    return
}
_, err = model.UpdateDesignVersionSubmittedAt(*work.CurrentVersionID, now)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "提交失败"})
    return
}

work, _ = model.GetDesignWorkByID(workID)
version, _ := model.GetDesignVersionByID(*work.CurrentVersionID)
c.JSON(http.StatusOK, gin.H{"success": true, "work": work, "version": version})
}

func ListWorks(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }

    page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
    pageSize, _ := strconv.Atoi(c.DefaultQuery("page_size", "20"))

```

```
if page <= 0 {
    page = 1
}
if pageSize <= 0 {
    pageSize = 20
}
if pageSize > 100 {
    pageSize = 100
}
offset := (page - 1) * pageSize

var status *int
if s := c.Query("status"); s != "" {
    if v, err := strconv.Atoi(s); err == nil && v > 0 {
        status = &v
    }
}
uid := int64(userID)
total, err := model.CountDesignWorks(&uid, status)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}
list, err := model.ListDesignWorks(&uid, status, pageSize, offset)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}

type WorkListItem struct {
    Work    model.DesignWork    `json:"work"`
    Version *model.DesignVersion `json:"version"``
```

```

    Model *model.ModelFile    `json:"model"`
}
items := make([]WorkListItem, 0, len(list))
for _, w := range list {
    var version *model.DesignVersion
    var modelFile *model.ModelFile
    if w.CurrentVersionID != nil {
        version, _ = model.GetDesignVersionByID(*w.CurrentVersionID)
    } else {
        version, _ = model.GetLatestDesignVersionByWorkID(w.ID)
    }
    if version != nil && version.ModelFileID != nil {
        if m, err := model.GetModelFileByID(int(*version.ModelFileID)); err == nil {
            modelFile = m
        }
    }
    items = append(items, WorkListItem{
        Work:    w,
        Version:  version,
        Model:    modelFile,
    })
}

c.JSON(http.StatusOK, gin.H{
    "success": true,
    "data": gin.H{
        "list":    items,
        "total":   total,
        "page":    page,
        "page_size": pageSize,
    },
})

```

```
}

func GetWork(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }

    workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || workID <= 0 {
        c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
        return
    }

    work, err := model.GetDesignWorkByID(workID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
        return
    }
    if work == nil || work.UserID != int64(userID) {
        c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
        return
    }

    var version *model.DesignVersion
    if work.CurrentVersionID != nil {
```

```

    version, _ = model.GetDesignVersionByID(*work.CurrentVersionID)
}
reviews, _ := model.ListDesignReviewsByWorkID(workID)
tryOnHistory, _ := model.ListTryOnTasksByWorkID(workID, 10)
var latestTask *model.TryOnTask
if len(tryOnHistory) > 0 {
    latestTask = &tryOnHistory[0]
}

c.JSON(http.StatusOK, gin.H{
    "success": true,
    "work":    work,
    "version": version,
    "reviews": reviews,
    "tryon": gin.H{
        "latest_task": latestTask,
        "history_tasks": tryOnHistory,
    },
})
}

func CreateTryOnTask(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
}

```

```
workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
if err != nil || workID <= 0 {
    c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
    return
}

work, err := model.GetDesignWorkByID(workID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}
if work == nil || work.UserID != int64(userID) {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    return
}
if work.CurrentVersionID == nil {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "缺少版本信息"})
    return
}

var input CreateTryOnTaskInput
if err := c.ShouldBindJSON(&input); err != nil {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": err.Error()})
    return
}

if len(input.InputRefs) == 0 {
    input.InputRefs = json.RawMessage("[]")
}

workActiveCount, err := model.CountActiveTryOnTasksByWorkID(workID)
```

```
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建任务失败"})
    return
}

if workActiveCount > 0 {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "当前作品已有进行中的试穿任务"})
    return
}

userActiveCount, err := model.CountActiveTryOnTasksByUserID(int64(userID))
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建任务失败"})
    return
}

userLimit := getEnvIntWithDefault("TRYON_USER_ACTIVE_LIMIT", 3)
if userActiveCount >= userLimit {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "进行中的试穿任务过多,请稍后再试"})
    return
}

taskID, err := generateTryOnTaskID(workID, int64(userID))
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建任务失败"})
    return
}

_, err = model.CreateTryOnTask(taskID, workID, *work.CurrentVersionID, int64(userID), input.BodyTy
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "创建任务失败"})
    return
}
```

```

if config.RedisClient != nil {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()
    if err := config.RedisClient.LPush(ctx, tryOnQueueKey, taskID).Err(); err != nil {
        _, _ = model.FailTryOnTask(taskID, "enqueue_failed", err.Error())
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "任务入队失败"})
        return
    }
}

task, _ := model.GetTryOnTaskByTaskID(taskID)
c.JSON(http.StatusOK, gin.H{"success": true, "task": task})
}

func GetTryOnTask(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }

    workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || workID <= 0 {
        c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
        return
    }
}

```

```
work, err := model.GetDesignWorkByID(workID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}
if work == nil || work.UserID != int64(userID) {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    return
}

taskID := c.Param("task_id")
if taskID == "" {
    c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "task_id不能为空"})
    return
}

task, err := model.GetTryOnTaskByTaskID(taskID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取任务失败"})
    return
}
if task == nil || task.WorkID != workID || task.UserID != int64(userID) {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "任务不存在"})
    return
}

c.JSON(http.StatusOK, gin.H{
    "success":    true,
    "task":       task,
    "thumbnail_url": task.ResultURL,
})
}
```

```
func generateTryOnTaskID(workID, userID int64) (string, error) {  
    buf := make([]byte, 8)  
    if _, err := rand.Read(buf); err != nil {  
        return "", err  
    }  
    return fmt.Sprintf("tryon_%d_%d_%d_%s", workID, userID, time.Now().UnixNano(), hex.EncodeToString(buf))  
}
```

```
func getEnvIntWithDefault(key string, defaultValue int) int {  
    raw := os.Getenv(key)  
    if raw == "" {  
        return defaultValue  
    }  
    val, err := strconv.Atoi(raw)  
    if err != nil || val <= 0 {  
        return defaultValue  
    }  
    return val  
}
```

```
func UnpublishWork(c *gin.Context) {  
    userIDVal, ok := c.Get("userID")  
    if !ok {  
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})  
        return  
    }  
    userID, ok := userIDVal.(int)  
    if !ok || userID <= 0 {  
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})  
        return  
    }  
}
```

```
workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
if err != nil || workID <= 0 {
    c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
    return
}

work, err := model.GetDesignWorkByID(workID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
    return
}
if work == nil || work.UserID != int64(userID) {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    return
}
if work.Status != model.DesignStatusPublished {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "当前状态不可下架"})
    return
}

affected, err := model.UnpublishDesignWork(workID)
if err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "下架失败"})
    return
}
if affected == 0 {
    c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
    return
}

work, _ = model.GetDesignWorkByID(workID)
```

```
c.JSON(http.StatusOK, gin.H{"success": true, "work": work})
}

func DeleteWork(c *gin.Context) {
    userIDVal, ok := c.Get("userID")
    if !ok {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }
    userID, ok := userIDVal.(int)
    if !ok || userID <= 0 {
        c.JSON(http.StatusUnauthorized, gin.H{"success": false, "message": "未登录"})
        return
    }

    workID, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || workID <= 0 {
        c.JSON(http.StatusBadRequest, gin.H{"success": false, "message": "ID不合法"})
        return
    }

    work, err := model.GetDesignWorkByID(workID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "获取作品失败"})
        return
    }
    if work == nil || work.UserID != int64(userID) {
        c.JSON(http.StatusOK, gin.H{"success": false, "message": "作品不存在"})
        return
    }
    if work.Status == model.DesignStatusSubmitted || work.Status == model.DesignStatusPublished {
        c.JSON(http.StatusOK, gin.H{"success": false, "message": "当前状态不可删除"})
    }
}
```

```
return
}

if err := model.DeleteDesignWorkCascade(workID); err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"success": false, "error": "删除失败"})
    return
}
c.JSON(http.StatusOK, gin.H{"success": true, "message": "删除成功"})
}
```

附录 C AI 虚拟试穿模块

前端代码

```
<template>
```

```
<div class="bg-white rounded-[2rem] border border-slate-100 shadow-[0_4px_24px_rgba(0,0,0,0.04)] p-6">
```

```
<h2 class="text-lg font-black text-slate-900 mb-6 flex items-center gap-2">
```

```
  AI 虚拟试穿
```

```
<span class="px-2.5 py-0.5 rounded-full bg-purple-100 text-purple-600 text-xs font-bold">BETA</span>
```

```
</h2>
```

```
<div v-if="!userStore.token" class="text-center py-6">
```

```
<p class="text-sm text-slate-500 mb-4">登录后即可体验 AI 虚拟试穿功能</p>
```

```
<button @click="router.push('/login')" class="px-6 py-2 rounded-xl bg-purple-50 text-purple-600 text-sm">
```

```
  去登录
```

```
</button>
```

```
</div>
```

```
<div v-else>
```

```
<!-- Configuration Form -->
```

```
<div class="space-y-4 mb-6">
```

```
<div>
```

```
<label class="block text-xs font-bold text-slate-500 mb-2 uppercase tracking-wider">模特体型 (Body Type)</label>
```

```
<select v-model="form.body_type" :disabled="isBusy" class="w-full px-4 py-2.5 rounded-xl bg-slate-50">
```

```
<option value="female">女性 (Female)</option>
```

```
<option value="male">男性 (Male)</option>
```

```
</select>
```

```
</div>
```

```
<div>
```

```
<label class="block text-xs font-bold text-slate-500 mb-2 uppercase tracking-wider">场景风格 (Scene Style)</label>
```

```
<select v-model="form.scene_style" :disabled="isBusy" class="w-full px-4 py-2.5 rounded-xl bg-slate-50">
```

```
<option value="studio">影棚 (Studio)</option>
```

```
<option value="street">街拍 (Street)</option>
```

```
<option value="nature">自然 (Nature)</option>
```

```

    </select>
  </div>

  <button
    @click="startTryon"
    :disabled="isBusy"
    class="w-full flex items-center justify-center gap-2 py-3 rounded-xl bg-gradient-to-br from-purple-
  >
    <span v-if="isBusy" class="w-4 h-4 rounded-full border-2 border-white/30 border-t-white animate
    <span>{{ isBusy ? '任务进行中...' : '开始试穿' }}</span>
  </button>
</div>

<!-- Current Task Status -->
<div v-if="latestTask" class="bg-slate-50 rounded-2xl p-5 mb-6 border border-slate-100">
  <div class="flex items-center justify-between mb-3">
    <span class="text-sm font-bold text-slate-800">当前任务状态</span>
    <span class="px-2.5 py-1 rounded-lg text-xs font-bold" :class="statusConfig[latestTask.status]?.cla
      {{ statusConfig[latestTask.status]?.label || '未知状态' }}
    </span>
  </div>
</div>

<!-- Progress Bar for Queued/Running -->
<div v-if="[1, 2].includes(latestTask.status)" class="space-y-2">
  <div class="flex justify-between text-xs text-slate-500">
    <span>{{ latestTask.status === 1 ? '排队中...' : 'AI 正在生成中...' }}</span>
    <span>{{ latestTask.progress || 0 }}%</span>
  </div>
  <div class="w-full h-2 bg-slate-200 rounded-full overflow-hidden">
    <div class="h-full bg-purple-500 transition-all duration-500" :style="{ width: `${latestTask.progr
  </div>
</div>

```

```

<!-- Error Message -->
<div v-if="latestTask.status === 4" class="mt-2 p-3 rounded-xl bg-red-50 text-red-600 text-xs">
  {{ latestTask.error_message || '任务执行失败，请稍后重试' }}
</div>

<!-- Success Result -->
<div v-if="latestTask.status === 3 && resultImage" class="mt-4 relative rounded-xl overflow-hidden">
  
</div>
</div>

<!-- History Tasks -->
<div v-if="historyTasks.length > 0">
  <h3 class="text-xs font-bold text-slate-500 mb-3 uppercase tracking-wider">历史记录</h3>
  <div class="space-y-3 max-h-64 overflow-y-auto pr-2 custom-scrollbar">
    <div v-for="task in historyTasks" :key="task.id" class="flex gap-3 p-3 rounded-xl bg-white border">
      <div class="w-12 h-12 rounded-lg bg-slate-50 overflow-hidden shrink-0 flex items-center justify-center">
        
        <svg v-else width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
          <rect x="3" y="3" width="18" height="18" rx="2" ry="2"/><circle cx="8.5" cy="8.5" r="1.5"/>
        </svg>
      </div>
      <div class="flex-1 min-w-0 flex flex-col justify-center">
        <div class="flex items-center justify-between mb-1">
          <span class="text-xs font-bold text-slate-700 truncate">
            {{ task.body_type === 'female' ? '女性' : '男性' }} / {{ task.scene_style === 'studio' ? '影棚' : '户外' }}
          </span>
          <span class="text-[10px] px-1.5 py-0.5 rounded text-slate-500" :class="statusConfig[task.status]">
            {{ statusConfig[task.status]?.label }}
          </span>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

      <span class="text-[10px] text-slate-400">{{ formatTime(task.created_at) }}</span>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</template>

```

```

<script setup lang="ts">
import { ref, computed, onMounted, onUnmounted, watch } from 'vue'
import { useRouter } from 'vue-router'
import { useUserStore } from '~/stores/user'
import { useWorks } from '~/composables/useWorks'
import type { TryonTask } from '~/composables/useWorks'
import { ElMessage } from 'element-plus'

const props = defineProps<{
  workId: number
  initData?: {
    latest_task: TryonTask | null
    history_tasks: TryonTask[]
  }
}>()

const router = useRouter()
const userStore = useUserStore()
const { createTryonTask, getTryonTask } = useWorks()

const form = ref({
  body_type: 'female',
  scene_style: 'studio'

```

```
}}
```

```
const latestTask = ref<TryonTask & { thumbnail_url?: string } | null>(null)
```

```
const historyTasks = ref<(TryonTask & { thumbnail_url?: string })[]>([])
```

```
const pollTimer = ref<number | null>(null)
```

```
const isBusy = computed(() => {
```

```
  return latestTask.value ? [1, 2].includes(latestTask.value.status) : false
```

```
})
```

```
const resultImage = computed(() => {
```

```
  if (!latestTask.value) return null
```

```
  return latestTask.value.thumbnail_url || latestTask.value.result_url
```

```
})
```

```
const statusConfig: Record<number, { label: string; class: string }> = {
```

```
  1: { label: '排队中', class: 'bg-amber-100 text-amber-700' },
```

```
  2: { label: '处理中', class: 'bg-blue-100 text-blue-700' },
```

```
  3: { label: '成功', class: 'bg-emerald-100 text-emerald-700' },
```

```
  4: { label: '失败', class: 'bg-red-100 text-red-700' },
```

```
  5: { label: '已取消', class: 'bg-slate-200 text-slate-600' }
```

```
}
```

```
const formatTime = (timeStr: string) => {
```

```
  if (!timeStr) return "
```

```
  const date = new Date(timeStr)
```

```
  return `${date.getMonth() + 1}/${date.getDate()} ${date.getHours().toString().padStart(2, '0')}:${date.g
```

```
}
```

```
const startPolling = () => {
```

```
  stopPolling()
```

```
  const fetchStatus = async () => {
```

```

if (!latestTask.value?.task_id) return
try {
  const res = await getTryonTask(props.workId, latestTask.value.task_id) as any
  if (res.success && res.task) {
    latestTask.value = { ...res.task, thumbnail_url: res.thumbnail_url }
    if (![1, 2].includes(latestTask.value!.status)) {
      stopPolling()
      if (latestTask.value!.status === 3) {
        ElMessage.success('试穿完成! ')
        updateHistory(latestTask.value!)
      } else if (latestTask.value!.status === 4) {
        ElMessage.error(latestTask.value!.error_message || '试穿失败')
        updateHistory(latestTask.value!)
      }
    }
  }
} catch (e) {
  console.error('Polling error:', e)
}

fetchStatus()
pollTimer.value = window.setInterval(fetchStatus, 2000)
}

const stopPolling = () => {
  if (pollTimer.value) {
    clearInterval(pollTimer.value)
    pollTimer.value = null
  }
}

```

```

const updateHistory = (task: TryonTask & { thumbnail_url?: string }) => {
  const index = historyTasks.value.findIndex(t => t.task_id === task.task_id)
  if (index >= 0) {
    historyTasks.value[index] = task
  } else {
    historyTasks.value.unshift(task)
  }
}

const startTryon = async () => {
  if (!userStore.token) {
    ElMessage.warning('请先登录')
    return
  }

  try {
    const res = await createTryonTask(props.workId, {
      body_type: form.value.body_type,
      scene_style: form.value.scene_style,
      input_refs: []
    }) as any

    if (res.success && res.task) {
      latestTask.value = res.task
      startPolling()
      ElMessage.success('已加入试穿队列')
    } else {
      ElMessage.error(res.message || '创建任务失败')
    }
  } catch (e: any) {
    const msg = e.response?._data?.message || e.response?.data?.message || e.message || '网络请求失败'
    ElMessage.error(msg)
  }
}

```

```

    }
  }

const viewHistoryTask = (task: TryonTask & { thumbnail_url?: string }) => {
  if (isBusy.value) {
    ElMessage.warning('当前有任务正在进行中')
    return
  }
  latestTask.value = task
}

watch(() => props.initialData, (newData) => {
  if (newData) {
    if (newData.latest_task) {
      latestTask.value = newData.latest_task
      if ([1, 2].includes(latestTask.value!.status)) {
        startPolling()
      }
    }
    if (newData.history_tasks) {
      historyTasks.value = [...newData.history_tasks]
    }
  }
}, { immediate: true, deep: true })

onUnmounted(() => {
  stopPolling()
})
</script>

<style scoped>
.custom-scrollbar::-webkit-scrollbar {

```

```
width: 4px;
}
.custom-scrollbar::-webkit-scrollbar-track {
  background: transparent;
}
.custom-scrollbar::-webkit-scrollbar-thumb {
  background: #cbd5e1;
  border-radius: 4px;
}
.custom-scrollbar::-webkit-scrollbar-thumb:hover {
  background: #94a3b8;
}
</style>
```

后端代码

package model

```
import (
    "database/sql"
    "encoding/json"
    "fmt"
    "strings"
    "time"

    "github.com/chosetop/bishe_back/internal/config"
)
```

```
const (
    TryOnTaskStatusQueued    = 1
    TryOnTaskStatusRunning  = 2
    TryOnTaskStatusSuccess   = 3
)
```

```

TryOnTaskStatusFailed    = 4
TryOnTaskStatusCancelled = 5
)

```

```

type TryOnTask struct {
    ID          int64      `json:"id"`
    TaskID      string     `json:"task_id"`
    WorkID      int64      `json:"work_id"`
    VersionID   int64      `json:"version_id"`
    UserID      int64      `json:"user_id"`
    BodyType    string     `json:"body_type"`
    SceneStyle  string     `json:"scene_style"`
    InputRefs   json.RawMessage `json:"input_refs"`
    Status      int        `json:"status"`
    Progress    int        `json:"progress"`
    ResultURL   string     `json:"result_url"`
    ErrorCode   string     `json:"error_code"`
    ErrorMessage string     `json:"error_message"`
    StartedAt   *time.Time `json:"started_at"`
    FinishedAt  *time.Time `json:"finished_at"`
    CreatedAt   time.Time  `json:"created_at"`
    UpdatedAt   time.Time  `json:"updated_at"`
}

```

```

type tryOnTaskScanner interface {
    Scan(dest ...any) error
}

```

```

func scanTryOnTask(scanner tryOnTaskScanner) (*TryOnTask, error) {
    var t TryOnTask
    var inputRefsRaw []byte
    var startedAt sql.NullTime

```

```
var finishedAt sql.NullTime
if err := scanner.Scan(
    &t.ID,
    &t.TaskID,
    &t.WorkID,
    &t.VersionID,
    &t.UserID,
    &t.BodyType,
    &t.SceneStyle,
    &inputRefsRaw,
    &t.Status,
    &t.Progress,
    &t.ResultURL,
    &t.ErrorCode,
    &t.ErrorMessage,
    &startedAt,
    &finishedAt,
    &t.CreatedAt,
    &t.UpdatedAt,
); err != nil {
    return nil, err
}
if inputRefsRaw != nil {
    t.InputRefs = json.RawMessage(inputRefsRaw)
}
if startedAt.Valid {
    val := startedAt.Time
    t.StartedAt = &val
}
if finishedAt.Valid {
    val := finishedAt.Time
    t.FinishedAt = &val
}
```

```

    }
    return &t, nil
}

func CreateTryOnTask(taskID string, workID, versionID, userID int64, bodyType, sceneStyle string, inputRefs []string) (int64, error) {
    if len(inputRefs) == 0 {
        inputRefs = json.RawMessage("[]")
    }
    res, err := config.DB.Exec(
        `INSERT INTO tryon_tasks(task_id, work_id, version_id, user_id, body_type, scene_style, input_refs, status, progress,
        VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)`,
        taskID, workID, versionID, userID, bodyType, sceneStyle, inputRefs, TryOnTaskStatusQueued, 0,
    )
    if err != nil {
        return 0, err
    }
    return res.LastInsertId()
}

func GetTryOnTaskByID(id int64) (*TryOnTask, error) {
    row := config.DB.QueryRow(
        `SELECT id, task_id, work_id, version_id, user_id, body_type, scene_style, input_refs, status, progress,
        result_url, error_code, error_message, started_at, finished_at, created_at, updated_at
        FROM tryon_tasks
        WHERE id = ?`,
        id,
    )
    task, err := scanTryOnTask(row)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, nil
        }
    }
}

```

```
    return nil, err
}
return task, nil
}
```

```
func GetTryOnTaskByTaskID(taskID string) (*TryOnTask, error) {
    row := config.DB.QueryRow(
        `SELECT id, task_id, work_id, version_id, user_id, body_type, scene_style, input_refs, status, progress,
           result_url, error_code, error_message, started_at, finished_at, created_at, updated_at
        FROM tryon_tasks
        WHERE task_id = ?`,
        taskID,
    )
    task, err := scanTryOnTask(row)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, nil
        }
        return nil, err
    }
    return task, nil
}
```

```
func ListTryOnTasksByWorkID(workID int64, limit int) ([]TryOnTask, error) {
    if limit <= 0 {
        limit = 20
    }
    if limit > 100 {
        limit = 100
    }
    rows, err := config.DB.Query(
        `SELECT id, task_id, work_id, version_id, user_id, body_type, scene_style, input_refs, status, progress,
```

```

        result_url, error_code, error_message, started_at, finished_at, created_at, updated_at
    FROM tryon_tasks
    WHERE work_id = ?
    ORDER BY id DESC
    LIMIT ?`,
    workID, limit,
)
if err != nil {
    return nil, err
}
defer rows.Close()

list := make([]TryOnTask, 0, limit)
for rows.Next() {
    task, err := scanTryOnTask(rows)
    if err != nil {
        return nil, err
    }
    list = append(list, *task)
}
if err := rows.Err(); err != nil {
    return nil, err
}
return list, nil
}

```

```

func CountActiveTryOnTasksByWorkID(workID int64) (int, error) {
    var total int
    err := config.DB.QueryRow(
        `SELECT COUNT(1)
        FROM tryon_tasks
        WHERE work_id = ? AND status IN (?, ?)`,

```

```
workID, TryOnTaskStatusQueued, TryOnTaskStatusRunning,  
)}.Scan(&total)  
return total, err  
}
```

```
func CountActiveTryOnTasksByUserID(userID int64) (int, error) {  
    var total int  
    err := config.DB.QueryRow(  
        `SELECT COUNT(1)  
        FROM tryon_tasks  
        WHERE user_id = ? AND status IN (?, ?)`,  
        userID, TryOnTaskStatusQueued, TryOnTaskStatusRunning,  
    )}.Scan(&total)  
    return total, err  
}
```

```
func ListTimedOutRunningTryOnTasks(timeoutBefore time.Time, limit int) ([]TryOnTask, error) {  
    if limit <= 0 {  
        limit = 20  
    }  
    if limit > 200 {  
        limit = 200  
    }  
    rows, err := config.DB.Query(  
        `SELECT id, task_id, work_id, version_id, user_id, body_type, scene_style, input_refs, status, progress,  
        result_url, error_code, error_message, started_at, finished_at, created_at, updated_at  
        FROM tryon_tasks  
        WHERE status = ? AND started_at IS NOT NULL AND started_at < ?  
        ORDER BY started_at ASC  
        LIMIT ?`,  
        TryOnTaskStatusRunning, timeoutBefore, limit,  
    )
```

```
if err != nil {
    return nil, err
}
defer rows.Close()

list := make([]TryOnTask, 0, limit)
for rows.Next() {
    task, err := scanTryOnTask(rows)
    if err != nil {
        return nil, err
    }
    list = append(list, *task)
}
if err := rows.Err(); err != nil {
    return nil, err
}
return list, nil
}

func UpdateTryOnTaskProgress(taskID string, progress int) (int64, error) {
    if progress < 0 {
        progress = 0
    }
    if progress > 100 {
        progress = 100
    }
    res, err := config.DB.Exec(
        `UPDATE tryon_tasks
        SET progress = ?
        WHERE task_id = ? AND status = ?`,
        progress, taskID, TryOnTaskStatusRunning,
    )
}
```

```
if err != nil {  
    return 0, err  
}  
return res.RowsAffected()  
}
```

```
func StartTryOnTask(taskID string) (int64, error) {  
    now := time.Now()  
    res, err := config.DB.Exec(  
        `UPDATE tryon_tasks  
        SET status = ?, started_at = ?, error_code = "", error_message = "  
        WHERE task_id = ? AND status = ?`,  
        TryOnTaskStatusRunning, now, taskID, TryOnTaskStatusQueued,  
    )  
    if err != nil {  
        return 0, err  
    }  
    return res.RowsAffected()  
}
```

```
func RequeueTryOnTask(taskID string) (int64, error) {  
    res, err := config.DB.Exec(  
        `UPDATE tryon_tasks  
        SET status = ?, progress = 0, started_at = NULL, finished_at = NULL  
        WHERE task_id = ? AND status = ?`,  
        TryOnTaskStatusQueued, taskID, TryOnTaskStatusRunning,  
    )  
    if err != nil {  
        return 0, err  
    }  
    return res.RowsAffected()  
}
```

```

func CompleteTryOnTask(taskID, resultURL string) (int64, error) {
    now := time.Now()
    res, err := config.DB.Exec(
        `UPDATE tryon_tasks
        SET status = ?, progress = 100, result_url = ?, error_code = "", error_message = "", finished_at = ?
        WHERE task_id = ? AND status = ?`,
        TryOnTaskStatusSuccess, resultURL, now, taskID, TryOnTaskStatusRunning,
    )
    if err != nil {
        return 0, err
    }
    return res.RowsAffected()
}

```

```

func FailTryOnTask(taskID, errorCode, errorMessage string) (int64, error) {
    now := time.Now()
    res, err := config.DB.Exec(
        `UPDATE tryon_tasks
        SET status = ?, error_code = ?, error_message = ?, finished_at = ?
        WHERE task_id = ? AND status IN (?, ?)`,
        TryOnTaskStatusFailed, errorCode, errorMessage, now, taskID, TryOnTaskStatusQueued, TryOnTaskStatusRunning,
    )
    if err != nil {
        return 0, err
    }
    return res.RowsAffected()
}

```

```

func CancelTryOnTask(taskID string) (int64, error) {
    now := time.Now()
    res, err := config.DB.Exec(

```

```
`UPDATE tryon_tasks
    SET status = ?, finished_at = ?
    WHERE task_id = ? AND status IN (?, ?)`,
TryOnTaskStatusCancelled, now, taskID, TryOnTaskStatusQueued, TryOnTaskStatusRunning,
)
if err != nil {
    return 0, err
}
return res.RowsAffected()
}
```

```
func UpdateTryOnTaskStatus(taskID string, toStatus int, fromStatuses []int) (int64, error) {
    if len(fromStatuses) == 0 {
        return 0, nil
    }
    holders := make([]string, 0, len(fromStatuses))
    args := make([]any, 0, len(fromStatuses)+2)
    args = append(args, toStatus, taskID)
    for _, s := range fromStatuses {
        holders = append(holders, "?")
        args = append(args, s)
    }
    query := fmt.Sprintf("UPDATE tryon_tasks SET status = ? WHERE task_id = ? AND status IN (%s)", strings.Join(holders, ","))
    res, err := config.DB.Exec(query, args...)
    if err != nil {
        return 0, err
    }
    return res.RowsAffected()
}
```

附录 D 创意广场模块

前端代码

// 广场页

```
<template>
```

```
  <div class="sq-root">
```

```
    <!-- Ambient Background -->
```

```
    <div class="sq-ambient" aria-hidden="true">
```

```
      <div class="sq-orb sq-orb--violet"></div>
```

```
      <div class="sq-orb sq-orb--pink"></div>
```

```
      <div class="sq-dots"></div>
```

```
    </div>
```

```
    <!-- Header -->
```

```
    <header class="sq-header" :class="{ 'sq-header--raised': isScrolled }">
```

```
      <div class="sq-header__inner">
```

```
        <!-- Logo -->
```

```
        <div class="sq-logo" @click="navigateTo('/')">
```

```
          <div class="sq-logo__mark">
```

```
            <el-icon :size="17">
```

```
              <MagicStick />
```

```
            </el-icon>
```

```
          </div>
```

```
          <span class="sq-logo__text">FORGE</span>
```

```
          <span class="sq-logo__badge">广场</span>
```

```
        </div>
```

```
    <!-- Search -->
```

```
    <div class="sq-search">
```

```
      <el-icon class="sq-search__icon">
```

```
        <Search />
```

```
      </el-icon>
```

```
      <input v-model="searchQuery" type="text" placeholder="搜索设计作品、灵感..." class="sq-
```

```
</div>
```

```
<!-- Actions -->
```

```
<div class="sq-header__actions">
```

```
  <button class="sq-btn-icon sq-btn-icon--mobile-only">
```

```
    <el-icon :size="19">
```

```
      <Search />
```

```
    </el-icon>
```

```
  </button>
```

```
<button @click="navigateTo('/design')" class="sq-btn-publish">
```

```
  <el-icon>
```

```
    <Plus />
```

```
  </el-icon>
```

```
  <span>发布</span>
```

```
</button>
```

```
<div class="sq-divider-v"></div>
```

```
<button class="sq-btn-icon sq-btn-icon--notify">
```

```
  <el-icon :size="19">
```

```
    <Bell />
```

```
  </el-icon>
```

```
  <span class="sq-notify-dot"></span>
```

```
</button>
```

```
<el-dropdown trigger="click" popper-class="custom-dropdown">
```

```
  <div class="sq-avatar-wrap">
```

```
    
```

```
  </div>
```

```
  <template #dropdown>
```

```

<el-dropdown-menu class="!rounded-2xl !p-2 !min-w-[160px]">
  <el-dropdown-item class="!rounded-lg !mb-1" @click="navigateTo('/works')">
    <el-icon>
      <Folder />
    </el-icon>我的作品
  </el-dropdown-item>
  <el-dropdown-item class="!rounded-lg !mb-1" @click="navigateTo('/user/profile')">
    <el-icon>
      <User />
    </el-icon>个人主页
  </el-dropdown-item>
  <el-dropdown-item class="!rounded-lg !mb-1">
    <el-icon>
      <Star />
    </el-icon>我的收藏
  </el-dropdown-item>
  <div class="h-px bg-white/10 my-1"></div>
  <el-dropdown-item class="!rounded-lg !text-red-400"
    @click="handleLogout">退出登录</el-dropdown-item>
</el-dropdown-menu>
</template>
</el-dropdown>
</div>
</div>
</header>

<main class="sq-main">
  <!-- Hero -->
  <section class="sq-hero">
    <div class="sq-hero__glow"></div>
    <div class="sq-hero__content">
      <div class="sq-hero__badge">

```

```

    <span class="sq-hero__badge-dot"></span>
    全新 3D 创意引擎 v2.0 上线
</div>
<h1 class="sq-hero__title">
    探索无限<br />
    <em class="sq-hero__title-em">数字时尚</em>
</h1>
<p class="sq-hero__desc">
    汇聚全球顶尖设计师的 3D 服装作品,从赛博朋克到复古回潮,激发你下一个设计的灵感
</p>
<div class="sq-hero__actions">
    <button @click="navigateTo('/design')" class="sq-hero-btn sq-hero-btn--primary">
        开始创作
    </button>
    <button class="sq-hero-btn sq-hero-btn--ghost">
        <el-icon>
            <TrendCharts />
        </el-icon>
        热门趋势
    </button>
</div>
</div>
<div class="sq-hero__visual">
    <div class="sq-hero__card sq-hero__card--back"></div>
    <div class="sq-hero__card sq-hero__card--front">
        <el-icon :size="72" class="sq-hero__visual-icon">
            <MagicStick />
        </el-icon>
    </div>
    <div class="sq-hero__ring sq-hero__ring--1"></div>
    <div class="sq-hero__ring sq-hero__ring--2"></div>
</div>

```

```

</section>

<!-- Filter Bar -->
<div class="sq-filter-wrap">
  <div class="sq-filter">
    <!-- Tags -->
    <div class="sq-tags">
      <button v-for="tag in tags" :key="tag" @click="activeTag = tag" class="sq-tag"
        :class="{ 'sq-tag--active': activeTag === tag }">{{ tag }}</button>
    </div>

    <!-- Tools -->
    <div class="sq-filter__tools">
      <el-dropdown trigger="click" @command="(c: string) => sortBy = c">
        <button class="sq-sort-btn">
          <el-icon v-if="sortBy === 'latest'">
            <Timer />
          </el-icon>
          <el-icon v-else-if="sortBy === 'hot'">
            <Trophy />
          </el-icon>
          <el-icon v-else>
            <TrendCharts />
          </el-icon>
          {{ sortLabel }}
          <el-icon class="sq-sort-btn__arrow">
            <ArrowDown />
          </el-icon>
        </button>
        <template #dropdown>
          <el-dropdown-menu class="!rounded-xl !p-1.5 !min-w-[140px]">
            <el-dropdown-item command="latest"

```

```

      :class="{ '!text-violet-400 !bg-violet-900/30 !font-bold': sortBy === 'latest' }"
      class="!rounded-lg">最新发布</el-dropdown-item>
    <el-dropdown-item command="hot"
      :class="{ '!text-violet-400 !bg-violet-900/30 !font-bold': sortBy === 'hot' }"
      class="!rounded-lg">热门推荐</el-dropdown-item>
    <el-dropdown-item command="trend"
      :class="{ '!text-violet-400 !bg-violet-900/30 !font-bold': sortBy === 'trend' }"
      class="!rounded-lg">人气最高</el-dropdown-item>
  </el-dropdown-menu>
</template>
</el-dropdown>

<div class="sq-divider-v"></div>

<div class="sq-view-toggle">
  <button class="sq-view-btn" :class="{ 'sq-view-btn--active': viewMode === 'grid' }"
    @click="viewMode = 'grid'">
    <el-icon :size="16">
      <Grid />
    </el-icon>
  </button>
  <button class="sq-view-btn" :class="{ 'sq-view-btn--active': viewMode === 'list' }"
    @click="viewMode = 'list'">
    <el-icon :size="16">
      <List />
    </el-icon>
  </button>
</div>
</div>
</div>
</div>

```

```

<!-- Works Grid -->
<transition-group name="sq-fade" tag="div" class="sq-grid"
  :class="viewMode === 'grid' ? 'sq-grid--grid' : 'sq-grid--list'">
  <div v-for="(work, index) in works" :key="work.id" class="sq-card"
    :style="{ animationDelay: `${index * 55}ms` }">
    <!-- Image -->
    <div class="sq-card__image">
      <ModelPreview v-if="work.params_json?.modelUrl" :model-url="work.params_json.mod
        :params="work.params_json"
        class="w-full h-full transition-transform duration-700 group-hover:scale-105" />
      <div v-else class="sq-card__placeholder">
        <el-icon :size="52">
          <MagicStick />
        </el-icon>
      </div>
    <!-- Overlay -->
    <div class="sq-card__overlay">
      <button class="sq-card__action" :class="{ 'sq-card__action--fav': work.is_favorited }"
        @click.stop="toggleFavorite(work)" title="收藏">
        <el-icon :size="18">
          <StarFilled v-if="work.is_favorited" />
          <Star v-else />
        </el-icon>
      </button>
      <button class="sq-card__action sq-card__action--main"
        @click.stop="router.push(`/square/${work.id}`)">查看详情</button>
      <button class="sq-card__action" title="分享">
        <el-icon :size="18">
          <Share />
        </el-icon>
      </button>
    </div>
  </div>

```

```
</div>
```

```
<div v-if="work.is_featured" class="sq-card__featured">精选</div>
```

```
</div>
```

```
<!-- Body -->
```

```
<div class="sq-card__body">
```

```
<h3 class="sq-card__title">{{ work.title }}</h3>
```

```
<div class="sq-card__tags">
```

```
<span v-for="tag in work.tags.slice(0, 3)" :key="tag" class="sq-card__tag">#{{ tag }}</span>
```

```
</div>
```

```
<div class="sq-card__footer">
```

```
<div class="sq-card__author">
```

```

```

```
<span class="sq-card__author-name">{{ work.author }}</span>
```

```
</div>
```

```
<div class="sq-card__stats">
```

```
<button class="sq-stat" :class="{ 'sq-stat--liked': work.is_liked }"
```

```
@click="toggleLike(work)">
```

```
<el-icon :size="14">
```

```
<component :is="work.is_liked ? 'StarFilled' : 'Star'" />
```

```
</el-icon>
```

```
<span>{{ work.likes }}</span>
```

```
</button>
```

```
<button class="sq-stat">
```

```
<el-icon :size="14">
```

```
<ChatDotRound />
```

```
</el-icon>
```

```
<span>{{ work.comments }}</span>
```

```
</button>
```

```
</div>
```

```
</div>
```

```

    </div>
  </div>
</transition-group>

<!-- Load More -->
<div v-if="!loading && works.length < total" class="sq-load-more">
  <button @click="loadMore" :disabled="loadingMore" class="sq-load-btn">
    <span v-if="loadingMore" class="sq-load-spinner"></span>
    {{ loadingMore ? '加载中...' : '探索更多灵感' }}
  </button>
</div>
</main>

<!-- Mobile FAB -->
<button @click="navigateTo('/design')" class="sq-fab md:hidden">
  <el-icon :size="22">
    <Plus />
  </el-icon>
</button>
</div>
</template>

<script setup lang="ts">
import {
  Search, Bell, Plus, User, ArrowDown, MagicStick, Trophy, Timer,
  TrendCharts, Grid, List, Star, StarFilled, Share, ChatDotRound, Folder
} from '@element-plus/icons-vue'
import { useUserStore } from '~/stores/user'

const userStore = useUserStore()
const router = useRouter()

```

```
// State
const searchQuery = ref('')
const activeTag = ref('全部')
const viewMode = ref<'grid' | 'list'>('grid')
const sortBy = ref('latest')
const loadingMore = ref(false)
const isScrolled = ref(false)

// Lifecycle
onMounted(() => {
  window.addEventListener('scroll', handleScroll)
  fetchSquareWorks(true)
})

onUnmounted(() => {
  window.removeEventListener('scroll', handleScroll)
})

const handleScroll = () => {
  isScrolled.value = window.scrollY > 20
}

watch([activeTag, sortBy], () => fetchSquareWorks(true))

let searchTimer: ReturnType<typeof setTimeout>
watch(searchQuery, () => {
  clearTimeout(searchTimer)
  searchTimer = setTimeout(() => fetchSquareWorks(true), 400)
})

// Constants
const tags = ['全部', '夹克', '连衣裙', 'T恤', '运动装', '正装', '休闲', '概念设计', '国潮', '赛博朋克', '虚拟展']
```

```
interface SquareWork {
  id: number
  title: string
  author: string
  avatar: string
  likes: number
  comments: number
  favorites: number
  is_liked: boolean
  is_favorited: boolean
  is_featured: boolean
  tags: string[]
  params_json: Record<string, any> | null
}

const works = ref<SquareWork[]>([])
const total = ref(0)
const currentPage = ref(1)
const pageSize = ref(12)
const loading = ref(false)

const fetchSquareWorks = async (reset = false) => {
  if (reset) currentPage.value = 1
  loading.value = true
  try {
    const res = await useRequest('/api/square/works', {
      page: currentPage.value,
      page_size: pageSize.value,
      tag: activeTag.value === '全部' ? undefined : activeTag.value,
      sort: sortBy.value,
      keyword: searchQuery.value || undefined
    })
  }
}
```

```
    }, { method: 'GET' }) as any
    if (reset) {
      works.value = res.data.list
    } else {
      works.value.push(...res.data.list)
    }
    total.value = res.data.total
  } catch (e) {
    console.error(e)
  } finally {
    loading.value = false
  }
}

const toggleLike = async (work: SquareWork) => {
  try {
    const res = await useRequest(`/api/square/works/${work.id}/like`, {}, { method: 'POST' }) as any
    work.is_liked = res.data.is_liked
    work.likes = res.data.likes
  } catch (e) {
    console.error(e)
  }
}

const toggleFavorite = async (work: SquareWork) => {
  try {
    const res = await useRequest(`/api/square/works/${work.id}/favorite`, {}, { method: 'POST' }) as any
    work.is_favorited = res.data.is_favorited
    work.favorites = res.data.favorites
  } catch (e) {
    console.error(e)
  }
}
```

```
}

// Computed
const sortLabel = computed(() => {
  switch (sortBy.value) {
    case 'latest': return '最新发布'
    case 'hot': return '热门推荐'
    case 'trend': return '人气最高'
    default: return '排序'
  }
})

// Methods
const handleLogout = () => {
  userStore.logout()
  ElMessage.success('已退出登录')
  router.push('/')
}

const loadMore = async () => {
  if (works.value.length >= total.value) return
  currentPage.value++
  loadingMore.value = true
  await fetchSquareWorks(false)
  loadingMore.value = false
}

const navigateTo = (path: string) => {
  router.push(path)
}
</script>
```

```
<style scoped>
```

```
@import url('https://fonts.googleapis.com/css2?family=Syne:wght@600;700;800&family=DM+Sans:wght@400;500;600;700;800;900&display=block');
```

```
/* —— Root —— */
```

```
.sq-root {  
  min-height: 100vh;  
  background: #F5F3FF;  
  font-family: 'DM Sans', system-ui, sans-serif;  
  color: #2E2A48;  
  position: relative;  
  overflow-x: hidden;  
  padding-bottom: 5rem;  
}
```

```
/* —— Ambient —— */
```

```
.sq-ambient {  
  position: fixed;  
  inset: 0;  
  pointer-events: none;  
  z-index: 0;  
  overflow: hidden;  
}
```

```
.sq-orb {  
  position: absolute;  
  border-radius: 50%;  
  filter: blur(100px);  
}
```

```
.sq-orb--violet {  
  width: 55vw;  
  height: 55vw;
```

```
top: -18%;
left: -12%;
background: radial-gradient(circle, rgba(167, 139, 250, 0.22) 0%, transparent 70%);
animation: orb-drift 14s ease-in-out infinite;
}

.sq-orb--pink {
width: 45vw;
height: 45vw;
bottom: -8%;
right: -8%;
background: radial-gradient(circle, rgba(232, 121, 249, 0.13) 0%, transparent 70%);
animation: orb-drift 18s ease-in-out infinite reverse;
animation-delay: -7s;
}

@keyframes orb-drift {

0%,
100% {
transform: translate(0, 0) scale(1);
}

33% {
transform: translate(3%, 5%) scale(1.06);
}

66% {
transform: translate(-2%, 2%) scale(0.97);
}
}
```

```
.sq-dots {  
    position: absolute;  
    inset: 0;  
    background-image: radial-gradient(circle, rgba(109, 40, 217, 0.07) 1px, transparent 1px);  
    background-size: 32px 32px;  
    mask-image: radial-gradient(ellipse 80% 50% at 50% 0%, black 20%, transparent 75%);  
}
```

```
/* —— Header —— */
```

```
.sq-header {  
    position: fixed;  
    top: 0;  
    width: 100%;  
    z-index: 50;  
    background: rgba(255, 255, 255, 0.78);  
    backdrop-filter: blur(20px);  
    -webkit-backdrop-filter: blur(20px);  
    border-bottom: 1px solid rgba(139, 92, 246, 0.1);  
    transition: background 0.3s, border-color 0.3s, box-shadow 0.3s;  
}
```

```
.sq-header--raised {  
    background: rgba(255, 255, 255, 0.95);  
    border-bottom-color: rgba(139, 92, 246, 0.18);  
    box-shadow: 0 2px 24px rgba(109, 40, 217, 0.07);  
}
```

```
.sq-header__inner {  
    max-width: 1300px;  
    margin: 0 auto;  
    padding: 0 1.5rem;  
    height: 66px;
```

```
display: flex;
align-items: center;
gap: 1.25rem;
}

/* Logo */
.sq-logo {
display: flex;
align-items: center;
gap: 0.6rem;
cursor: pointer;
user-select: none;
flex-shrink: 0;
}

.sq-logo__mark {
width: 33px;
height: 33px;
border-radius: 9px;
background: linear-gradient(140deg, #7C3AED 0%, #A78BFA 100%);
display: flex;
align-items: center;
justify-content: center;
color: #fff;
box-shadow: 0 4px 14px rgba(124, 58, 237, 0.28);
transition: box-shadow 0.3s, transform 0.3s;
flex-shrink: 0;
}

.sq-logo:hover .sq-logo__mark {
box-shadow: 0 6px 20px rgba(124, 58, 237, 0.42);
transform: scale(1.07);
```

```
}

.sq-logo__text {
  font-family: 'Syne', sans-serif;
  font-weight: 800;
  font-size: 1.05rem;
  letter-spacing: 0.14em;
  color: #2E2A48;
  display: none;
}

.sq-logo__badge {
  font-size: 0.68rem;
  color: #7C3AED;
  letter-spacing: 0.06em;
  padding: 1px 7px;
  border: 1px solid rgba(124, 58, 237, 0.22);
  border-radius: 4px;
  background: rgba(124, 58, 237, 0.06);
  display: none;
}

@media (min-width: 580px) {

  .sq-logo__text,
  .sq-logo__badge {
    display: block;
  }
}

/* Search */
.sq-search {
```

```
flex: 1;
max-width: 460px;
display: none;
position: relative;
align-items: center;
}
```

```
@media (min-width: 768px) {
  .sq-search {
    display: flex;
  }
}
```

```
.sq-search__icon {
  position: absolute;
  left: 13px;
  color: #A8A4C0;
  font-size: 14px;
  transition: color 0.25s;
  pointer-events: none;
}
```

```
.sq-search:focus-within .sq-search__icon {
  color: #7C3AED;
}
```

```
.sq-search__input {
  width: 100%;
  height: 40px;
  padding: 0 1rem 0 2.6rem;
  background: rgba(255, 255, 255, 0.7);
  border: 1px solid rgba(139, 92, 246, 0.15);
}
```

```
border-radius: 11px;
color: #2E2A48;
font-family: 'DM Sans', sans-serif;
font-size: 0.875rem;
outline: none;
transition: all 0.25s;
}

.sq-search__input::placeholder {
  color: #B0ACCC;
}

.sq-search__input:hover {
  border-color: rgba(124, 58, 237, 0.3);
  background: #fff;
}

.sq-search__input:focus {
  border-color: rgba(124, 58, 237, 0.5);
  background: #fff;
  box-shadow: 0 0 0 3px rgba(124, 58, 237, 0.1);
}

/* Header Actions */
.sq-header__actions {
  display: flex;
  align-items: center;
  gap: 0.45rem;
  margin-left: auto;
}

.sq-btn-icon {
```

```
width: 37px;
height: 37px;
border-radius: 10px;
border: 1px solid rgba(139, 92, 246, 0.12);
background: rgba(255, 255, 255, 0.6);
color: #8B82B8;
display: flex;
align-items: center;
justify-content: center;
cursor: pointer;
transition: all 0.2s;
position: relative;
}

.sq-btn-icon:hover {
  background: #fff;
  color: #4C1D95;
  border-color: rgba(124, 58, 237, 0.28);
  box-shadow: 0 2px 8px rgba(109, 40, 217, 0.1);
}

.sq-btn-icon--mobile-only {
  display: flex;
}

@media (min-width: 768px) {
  .sq-btn-icon--mobile-only {
    display: none;
  }
}

.sq-btn-icon--notify .sq-notify-dot {
```

```
position: absolute;
top: 7px;
right: 7px;
width: 6px;
height: 6px;
background: #EC4899;
border-radius: 50%;
border: 1.5px solid #F5F3FF;
}

.sq-btn-publish {
display: none;
align-items: center;
gap: 0.35rem;
padding: 0 1rem;
height: 37px;
background: linear-gradient(135deg, #7C3AED 0%, #9333EA 100%);
color: #fff;
font-family: 'DM Sans', sans-serif;
font-weight: 600;
font-size: 0.85rem;
border-radius: 10px;
border: none;
cursor: pointer;
transition: all 0.2s;
box-shadow: 0 4px 14px rgba(124, 58, 237, 0.28);
letter-spacing: 0.01em;
flex-shrink: 0;
}

.sq-btn-publish:hover {
background: linear-gradient(135deg, #6D28D9 0%, #7C3AED 100%);
```

```
    box-shadow: 0 6px 20px rgba(124, 58, 237, 0.4);
    transform: translateY(-1px);
}
```

```
@media (min-width: 580px) {
    .sq-btn-publish {
        display: flex;
    }
}
```

```
.sq-divider-v {
    width: 1px;
    height: 18px;
    background: rgba(109, 40, 217, 0.12);
    display: none;
    flex-shrink: 0;
}
```

```
@media (min-width: 580px) {
    .sq-divider-v {
        display: block;
    }
}
```

```
.sq-avatar-wrap {
    width: 34px;
    height: 34px;
    border-radius: 50%;
    border: 1.5px solid rgba(124, 58, 237, 0.25);
    padding: 2px;
    cursor: pointer;
    transition: border-color 0.2s;
```

```
    flex-shrink: 0;
}

.sq-avatar-wrap:hover {
    border-color: rgba(124, 58, 237, 0.55);
}
```

```
.sq-avatar {
    width: 100%;
    height: 100%;
    border-radius: 50%;
    object-fit: cover;
}
```

```
/* —— Main —— */
```

```
.sq-main {
    position: relative;
    z-index: 1;
    max-width: 1300px;
    margin: 0 auto;
    padding: 86px 1.5rem 60px;
    display: flex;
    flex-direction: column;
    gap: 2.5rem;
}
```

```
/* —— Hero —— */
```

```
.sq-hero {
    position: relative;
    display: flex;
    align-items: center;
    justify-content: space-between;
```

```
gap: 2.5rem;
padding: 3rem 3.5rem;
border-radius: 2rem;
background: linear-gradient(140deg, #EDE9FE 0%, #F5F0FF 50%, #FAF5FF 100%);
border: 1px solid rgba(167, 139, 250, 0.3);
overflow: hidden;
box-shadow: 0 8px 40px rgba(124, 58, 237, 0.08);
}
```

```
.sq-hero__glow {
  position: absolute;
  top: -40%;
  left: -5%;
  width: 60%;
  height: 180%;
  background: linear-gradient(90deg, rgba(167, 139, 250, 0.2) 0%, transparent 60%);
  transform: skewX(-18deg);
  pointer-events: none;
}
```

```
.sq-hero__content {
  position: relative;
  z-index: 1;
  max-width: 580px;
}
```

```
.sq-hero__badge {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.28rem 0.85rem;
  border-radius: 999px;
```

```
background: rgba(124, 58, 237, 0.08);
border: 1px solid rgba(124, 58, 237, 0.2);
font-size: 0.73rem;
font-weight: 500;
color: #6D28D9;
letter-spacing: 0.04em;
margin-bottom: 1.4rem;
}
```

```
.sq-hero__badge-dot {
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: #22C55E;
  animation: blink-dot 2.2s ease-in-out infinite;
}
```

```
@keyframes blink-dot {
```

```
  0%,
  100% {
    opacity: 1;
  }
```

```
  50% {
    opacity: 0.35;
  }
}
```

```
.sq-hero__title {
  font-family: 'Syne', sans-serif;
  font-weight: 800;
```

```
font-size: clamp(2.4rem, 4.8vw, 3.75rem);
line-height: 1.06;
color: #1E1040;
letter-spacing: -0.025em;
margin: 0 0 1.2rem;
}

.sq-hero__title-em {
  font-style: normal;
  background: linear-gradient(128deg, #7C3AED 0%, #A855F7 60%, #6D28D9 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.sq-hero__desc {
  font-size: 0.97rem;
  line-height: 1.75;
  color: #6B6490;
  max-width: 460px;
  margin: 0 0 2rem;
}

.sq-hero__actions {
  display: flex;
  flex-wrap: wrap;
  gap: 0.85rem;
}

.sq-hero-btn {
  padding: 0.72rem 1.7rem;
  border-radius: 11px;
```

```
font-family: 'DM Sans', sans-serif;
font-weight: 600;
font-size: 0.88rem;
cursor: pointer;
transition: all 0.25s;
border: none;
display: inline-flex;
align-items: center;
gap: 0.45rem;
letter-spacing: 0.015em;
}

.sq-hero-btn--primary {
  background: linear-gradient(135deg, #7C3AED, #9333EA);
  color: #fff;
  box-shadow: 0 4px 16px rgba(124, 58, 237, 0.3);
}

.sq-hero-btn--primary:hover {
  background: linear-gradient(135deg, #6D28D9, #7C3AED);
  box-shadow: 0 6px 22px rgba(124, 58, 237, 0.42);
  transform: translateY(-2px);
}

.sq-hero-btn--ghost {
  background: rgba(255, 255, 255, 0.65);
  color: #5B21B6;
  border: 1px solid rgba(124, 58, 237, 0.2);
  backdrop-filter: blur(8px);
}

.sq-hero-btn--ghost:hover {
```

```
background: #fff;
border-color: rgba(124, 58, 237, 0.38);
color: #4C1D95;
box-shadow: 0 4px 14px rgba(124, 58, 237, 0.1);
}
```

```
/* Hero Visual */
.sq-hero__visual {
  display: none;
  position: relative;
  z-index: 1;
  width: 270px;
  height: 270px;
  flex-shrink: 0;
}
```

```
@media (min-width: 768px) {
  .sq-hero__visual {
    display: block;
  }
}
```

```
.sq-hero__card {
  position: absolute;
  border-radius: 22px;
  border: 1px solid rgba(167, 139, 250, 0.35);
}
```

```
.sq-hero__card--back {
  inset: 14px;
  background: rgba(167, 139, 250, 0.12);
  transform: rotate(9deg);
}
```

```
    animation: card-back 7.5s ease-in-out infinite;
  }

.sq-hero__card--front {
  inset: 14px;
  background: rgba(255, 255, 255, 0.55);
  backdrop-filter: blur(14px);
  transform: rotate(-6deg);
  animation: card-front 6s ease-in-out infinite;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 8px 32px rgba(124, 58, 237, 0.1);
}

.sq-hero__visual-icon {
  color: rgba(124, 58, 237, 0.45);
  filter: drop-shadow(0 0 16px rgba(124, 58, 237, 0.3));
}

@keyframes card-back {

  0%,
  100% {
    transform: rotate(9deg) translateY(0);
  }

  50% {
    transform: rotate(11deg) translateY(-14px);
  }
}
```

```
@keyframes card-front {  
  
  0%,  
  100% {  
    transform: rotate(-6deg) translateY(0);  
  }  
  
  50% {  
    transform: rotate(-4deg) translateY(-20px);  
  }  
}
```

```
.sq-hero__ring {  
  position: absolute;  
  border-radius: 50%;  
  pointer-events: none;  
  border: 1px solid rgba(124, 58, 237, 0.1);  
}
```

```
.sq-hero__ring--1 {  
  width: 310px;  
  height: 310px;  
  top: -20px;  
  left: -20px;  
  animation: ring-spin 22s linear infinite;  
}
```

```
.sq-hero__ring--2 {  
  width: 375px;  
  height: 375px;  
  top: -52px;  
  left: -52px;
```

```
border-color: rgba(232, 121, 249, 0.07);
animation: ring-spin 34s linear infinite reverse;
}
```

```
@keyframes ring-spin {
  from {
    transform: rotate(0deg);
  }

  to {
    transform: rotate(360deg);
  }
}
```

```
/* —— Filter Bar —— */
.sq-filter-wrap {
  position: sticky;
  top: 66px;
  z-index: 40;
  background: rgba(255, 255, 255, 0.82);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  border-radius: 14px;
  border: 1px solid rgba(167, 139, 250, 0.18);
  padding: 0.8rem 1.1rem;
  box-shadow: 0 4px 20px rgba(109, 40, 217, 0.06);
}
```

```
.sq-filter {
  display: flex;
  flex-direction: column;
  gap: 0.8rem;
```

```
}

@media (min-width: 768px) {
  .sq-filter {
    flex-direction: row;
    align-items: center;
    justify-content: space-between;
  }
}

/* Tags */
.sq-tags {
  display: flex;
  align-items: center;
  gap: 0.45rem;
  overflow-x: auto;
  padding-bottom: 2px;
  -ms-overflow-style: none;
  scrollbar-width: none;
  mask-image: linear-gradient(to right, black 86%, transparent 100%);
}

.sq-tags::-webkit-scrollbar {
  display: none;
}

.sq-tag {
  padding: 0.36rem 0.9rem;
  border-radius: 999px;
  font-size: 0.78rem;
  font-weight: 500;
  font-family: 'DM Sans', sans-serif;
```

```
white-space: nowrap;
cursor: pointer;
transition: all 0.2s;
border: 1px solid rgba(167, 139, 250, 0.2);
background: rgba(255, 255, 255, 0.6);
color: #7C6FAA;
letter-spacing: 0.01em;
}

.sq-tag:hover {
  color: #6D28D9;
  border-color: rgba(124, 58, 237, 0.35);
  background: rgba(124, 58, 237, 0.06);
}

.sq-tag--active {
  background: linear-gradient(135deg, #7C3AED, #9333EA);
  border-color: transparent;
  color: #fff;
  font-weight: 600;
  box-shadow: 0 3px 12px rgba(124, 58, 237, 0.28);
}

.sq-tag--active:hover {
  background: linear-gradient(135deg, #6D28D9, #7C3AED);
  box-shadow: 0 4px 16px rgba(124, 58, 237, 0.38);
  color: #fff;
  border-color: transparent;
}

/* Filter Tools */
.sq-filter__tools {
```

```
display: flex;
align-items: center;
gap: 0.45rem;
flex-shrink: 0;
}

.sq-sort-btn {
display: flex;
align-items: center;
gap: 0.45rem;
padding: 0.38rem 0.85rem;
border-radius: 9px;
border: 1px solid rgba(167, 139, 250, 0.2);
background: rgba(255, 255, 255, 0.7);
color: #6B6490;
font-family: 'DM Sans', sans-serif;
font-size: 0.8rem;
font-weight: 600;
cursor: pointer;
transition: all 0.2s;
letter-spacing: 0.01em;
}

.sq-sort-btn:hover {
border-color: rgba(124, 58, 237, 0.35);
color: #5B21B6;
background: #fff;
box-shadow: 0 2px 8px rgba(109, 40, 217, 0.08);
}

.sq-sort-btn__arrow {
color: rgba(124, 58, 237, 0.3);
```

```
    font-size: 11px;
}

.sq-view-toggle {
    display: flex;
    background: rgba(255, 255, 255, 0.7);
    border: 1px solid rgba(167, 139, 250, 0.2);
    border-radius: 9px;
    padding: 3px;
    gap: 2px;
}

.sq-view-btn {
    width: 30px;
    height: 30px;
    border-radius: 7px;
    border: none;
    background: transparent;
    color: #B0A8D0;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.2s;
}

.sq-view-btn:hover {
    color: #7C3AED;
}

.sq-view-btn--active {
    background: rgba(124, 58, 237, 0.1);
```

```
    color: #6D28D9;
}

/* —— Grid —— */
.sq-grid {
    display: grid;
    gap: 1.375rem;
}

.sq-grid--grid {
    grid-template-columns: 1fr;
}

@media (min-width: 580px) {
    .sq-grid--grid {
        grid-template-columns: repeat(2, 1fr);
    }
}

@media (min-width: 1024px) {
    .sq-grid--grid {
        grid-template-columns: repeat(3, 1fr);
    }
}

.sq-grid--list {
    grid-template-columns: 1fr;
}

/* —— Card —— */
.sq-card {
    background: #fff;
```

```
border-radius: 18px;
border: 1px solid rgba(167, 139, 250, 0.14);
overflow: hidden;
transition: transform 0.4s cubic-bezier(0.16, 1, 0.3, 1), border-color 0.35s, box-shadow 0.4s;
animation: card-appear 0.55s cubic-bezier(0.16, 1, 0.3, 1) both;
position: relative;
box-shadow: 0 2px 12px rgba(109, 40, 217, 0.04);
}

.sq-card:hover {
  transform: translateY(-6px);
  border-color: rgba(124, 58, 237, 0.28);
  box-shadow:
    0 16px 48px rgba(109, 40, 217, 0.12),
    0 4px 16px rgba(109, 40, 217, 0.06);
}

@keyframes card-appear {
  from {
    opacity: 0;
    transform: translateY(20px) scale(0.98);
  }

  to {
    opacity: 1;
    transform: translateY(0) scale(1);
  }
}

/* Card Image */
.sq-card__image {
  aspect-ratio: 4 / 3;
```

```
overflow: hidden;
background: #F3F0FF;
position: relative;
}

.sq-card__placeholder {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #EDE9FE, #F5F0FF);
  color: rgba(124, 58, 237, 0.22);
}

/* Overlay */
.sq-card__overlay {
  position: absolute;
  inset: 0;
  background: rgba(30, 16, 64, 0.5);
  backdrop-filter: blur(3px);
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 0.6rem;
  opacity: 0;
  transition: opacity 0.28s;
}

.sq-card:hover .sq-card__overlay {
  opacity: 1;
}
```

```
.sq-card__action {  
  width: 38px;  
  height: 38px;  
  border-radius: 50%;  
  background: rgba(255, 255, 255, 0.2);  
  border: 1px solid rgba(255, 255, 255, 0.35);  
  color: rgba(255, 255, 255, 0.88);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  cursor: pointer;  
  transition: all 0.2s;  
  backdrop-filter: blur(10px);  
}
```

```
.sq-card__action:hover {  
  background: rgba(255, 255, 255, 0.35);  
  color: #fff;  
  transform: scale(1.1);  
}
```

```
.sq-card__action--fav {  
  color: #F472B6;  
  background: rgba(244, 114, 182, 0.18);  
  border-color: rgba(244, 114, 182, 0.35);  
}
```

```
.sq-card__action--main {  
  width: auto;  
  padding: 0 1.15rem;  
  height: 38px;
```

```
border-radius: 999px;
font-family: 'DM Sans', sans-serif;
font-weight: 600;
font-size: 0.8rem;
letter-spacing: 0.015em;
background: rgba(255, 255, 255, 0.22);
}

.sq-card__action--main:hover {
  background: rgba(124, 58, 237, 0.6);
  border-color: rgba(124, 58, 237, 0.5);
  transform: scale(1.04);
}

.sq-card__featured {
  position: absolute;
  top: 11px;
  left: 11px;
  padding: 2px 9px;
  border-radius: 999px;
  background: linear-gradient(135deg, #7C3AED, #A855F7);
  color: #fff;
  font-size: 0.67rem;
  font-weight: 700;
  letter-spacing: 0.07em;
  text-transform: uppercase;
  font-family: 'Syne', sans-serif;
  box-shadow: 0 2px 10px rgba(124, 58, 237, 0.35);
}

/* Card Body */
.sq-card__body {
```

```
padding: 1.15rem 1.3rem;
}

.sq-card__title {
  font-family: 'Syne', sans-serif;
  font-weight: 700;
  font-size: 0.97rem;
  color: #2E2A48;
  margin: 0 0 0.7rem;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  transition: color 0.2s;
  letter-spacing: -0.01em;
}

.sq-card:hover .sq-card__title {
  color: #6D28D9;
}

.sq-card__tags {
  display: flex;
  flex-wrap: wrap;
  gap: 0.35rem;
  margin-bottom: 0.95rem;
}

.sq-card__tag {
  font-size: 0.68rem;
  font-weight: 600;
  color: #7C3AED;
  background: rgba(124, 58, 237, 0.06);
```

```
border: 1px solid rgba(124, 58, 237, 0.15);
border-radius: 5px;
padding: 2px 7px;
letter-spacing: 0.015em;
transition: all 0.2s;
}
```

```
.sq-card:hover .sq-card__tag {
  border-color: rgba(124, 58, 237, 0.3);
  background: rgba(124, 58, 237, 0.1);
}
```

```
.sq-card__footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding-top: 0.8rem;
  border-top: 1px solid rgba(167, 139, 250, 0.1);
}
```

```
.sq-card__author {
  display: flex;
  align-items: center;
  gap: 0.45rem;
}
```

```
.sq-card__avatar {
  width: 25px;
  height: 25px;
  border-radius: 50%;
  object-fit: cover;
  border: 1.5px solid rgba(124, 58, 237, 0.2);
}
```

```
}
```

```
.sq-card__author-name {  
  font-size: 0.775rem;  
  font-weight: 500;  
  color: #9990BA;  
}
```

```
.sq-card__stats {  
  display: flex;  
  align-items: center;  
  gap: 0.8rem;  
}
```

```
.sq-stat {  
  display: flex;  
  align-items: center;  
  gap: 0.28rem;  
  background: none;  
  border: none;  
  cursor: pointer;  
  color: #C4BCDF;  
  font-size: 0.73rem;  
  font-weight: 600;  
  font-family: 'DM Sans', sans-serif;  
  transition: color 0.2s;  
  padding: 0;  
}
```

```
.sq-stat:hover {  
  color: #7C3AED;  
}
```

```
.sq-stat--liked {
  color: #EC4899;
}

.sq-stat--liked:hover {
  color: #DB2777;
}

/* —— Load More —— */
.sq-load-more {
  display: flex;
  justify-content: center;
  padding: 1.5rem 0 2rem;
}

.sq-load-btn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.72rem 2.4rem;
  border-radius: 999px;
  border: 1px solid rgba(124, 58, 237, 0.22);
  background: rgba(124, 58, 237, 0.05);
  color: #7C3AED;
  font-family: 'DM Sans', sans-serif;
  font-weight: 500;
  font-size: 0.86rem;
  cursor: pointer;
  transition: all 0.28s;
  position: relative;
  overflow: hidden;
```

```
}

.sq-load-btn::before {
  content: "";
  position: absolute;
  inset: 0;
  background: linear-gradient(90deg, transparent, rgba(124, 58, 237, 0.08), transparent);
  transform: translateX(-100%);
  transition: transform 0.55s;
}

.sq-load-btn:hover::before {
  transform: translateX(100%);
}

.sq-load-btn:hover {
  border-color: rgba(124, 58, 237, 0.38);
  color: #5B21B6;
  background: rgba(124, 58, 237, 0.1);
  box-shadow: 0 4px 18px rgba(109, 40, 217, 0.12);
}

.sq-load-btn.disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.sq-load-spinner {
  width: 13px;
  height: 13px;
  border: 2px solid rgba(124, 58, 237, 0.2);
  border-top-color: #7C3AED;
```

```
border-radius: 50%;  
animation: spinner 0.65s linear infinite;  
}  
  
@keyframes spinner {  
  to {  
    transform: rotate(360deg);  
  }  
}  
  
/* —— FAB —— */  
.sq-fab {  
  position: fixed;  
  bottom: 1.5rem;  
  right: 1.5rem;  
  width: 50px;  
  height: 50px;  
  border-radius: 50%;  
  background: linear-gradient(135deg, #7C3AED, #9333EA);  
  color: #fff;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  box-shadow: 0 4px 18px rgba(124, 58, 237, 0.35), 0 1px 4px rgba(124, 58, 237, 0.15);  
  border: none;  
  cursor: pointer;  
  z-index: 40;  
  transition: all 0.2s;  
}  
  
.sq-fab:hover {  
  transform: scale(1.08);
```

```
    box-shadow: 0 6px 28px rgba(124, 58, 237, 0.62);
  }

  .sq-fab:active {
    transform: scale(0.93);
  }

  /* —— Transitions —— */
  .sq-fade-enter-active,
  .sq-fade-leave-active {
    transition: all 0.38s ease;
  }

  .sq-fade-enter-from,
  .sq-fade-leave-to {
    opacity: 0;
    transform: translateY(14px) scale(0.975);
  }
</style>

// 3d预览组件

<template>
  <ClientOnly>
    <div ref="mountRef" class="relative w-full h-full rounded-2xl overflow-hidden">
      </div>
    </ClientOnly>
  </template>

<script setup lang="ts">
import * as THREE from 'three'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
```

```
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { DecalGeometry } from 'three/examples/jsm/geometries/DecalGeometry.js'

const props = defineProps({
  modelUrl: {
    type: String,
    default: '/models/tshirt.glb'
  },
  params: {
    type: Object as () => Record<string, any> | null,
    default: null
  }
})

const mountRef = ref<HTMLDivElement | null>(null)
let renderer: THREE.WebGLRenderer | null = null
let scene: THREE.Scene | null = null
let camera: THREE.PerspectiveCamera | null = null
let controls: OrbitControls | null = null
let model: THREE.Group | null = null
let rafId = 0
let running = true
let cleanupResize: (() => void) | null = null
const defaultLights: THREE.Light[] = []
const textureLoader = new THREE.TextureLoader()

const applyLightPreset = (preset: string) => {
  if (!scene) return
  // Remove default lights
  defaultLights.forEach(l => scene!.remove(l))
  defaultLights.length = 0
}
```

```

if (preset === 'outdoor') {
  scene.background = new THREE.Color(0xe0f2fe)
  const a = new THREE.AmbientLight(0xffffff, 0.7); scene.add(a); defaultLights.push(a)
  const s = new THREE.DirectionalLight(0xffffff, 1.2); s.position.set(5, 10, 5); s.castShadow = true; scene
} else if (preset === 'dark') {
  scene.background = new THREE.Color(0x111111)
  const sp = new THREE.SpotLight(0xffffff, 1.5); sp.position.set(0, 5, 5); sp.angle = Math.PI / 4; sp.pen
  const rim = new THREE.SpotLight(0x4c1d95, 2); rim.position.set(-5, 2, -5); rim.lookAt(0, 0, 0); scene
} else {
  // indoor (default)
  scene.background = new THREE.Color(0xf3f4f6)
  const a = new THREE.AmbientLight(0xffffff, 0.6); scene.add(a); defaultLights.push(a)
  const d = new THREE.DirectionalLight(0xffffff, 0.8); d.position.set(2, 4, 3); d.castShadow = true; scene
  const f = new THREE.DirectionalLight(0xffffff, 0.3); f.position.set(-2, 0, -2); scene.add(f); defaultLigh
}
}

const applyMaterials = (materials: Record<string, any>) => {
  if (!model) return
  model.traverse((obj) => {
    if (!(obj as THREE.Mesh).isMesh) return
    const mesh = obj as THREE.Mesh
    const config = materials[mesh.name]
    if (!config) return
    const mats = Array.isArray(mesh.material) ? mesh.material : [mesh.material]
    mats.forEach((m) => {
      if (!(m instanceof THREE.MeshStandardMaterial)) return
      m.color.set(config.color)
      m.roughness = config.roughness
      m.metalness = config.metalness
      if (config.emissive) m.emissive.set(config.emissive)
      if (config.opacity !== undefined) { m.opacity = config.opacity; m.transparent = config.opacity < 1 }
    })
  })
}

```

```

m.map = null
if (config.map) {
  textureLoader.load(config.map, (tex) => {
    tex.colorSpace = THREE.SRGBColorSpace
    tex.flipY = false
    m.map = tex
    m.needsUpdate = true
  })
}
m.needsUpdate = true
})
})
}

const applyLogos = async (logos: any[]) => {
  if (!model || !scene) return
  for (const logo of logos) {
    if (!logo.url) continue
    let texture: THREE.Texture
    try {
      texture = await textureLoader.loadAsync(logo.url)
      texture.colorSpace = THREE.SRGBColorSpace
    } catch { continue }

    let targetMesh: THREE.Mesh | null = null
    model.traverse((obj) => {
      if ((obj as THREE.Mesh).isMesh && obj.name === logo.meshName) targetMesh = obj as THREE.Mesh
    })
    if (!targetMesh) continue

    const position = new THREE.Vector3(logo.transform.position.x, logo.transform.position.y, logo.transform.position.z)
    const orientation = new THREE.Euler(0, 0, logo.transform.rotation)
  }
}

```

```

const size = new THREE.Vector3(logo.transform.scale, logo.transform.scale, logo.transform.scale)
const geometry = new DecalGeometry(targetMesh, position, orientation, size)
const material = new THREE.MeshPhongMaterial({
  map: texture, transparent: true, depthTest: true, depthWrite: false,
  polygonOffset: true, polygonOffsetFactor: -4
})
const mesh = new THREE.Mesh(geometry, material)
mesh.name = `logo-${logo.id}`
scene.add(mesh)
}
}

```

```

const init = async () => {
  await nextTick()
  if (!mountRef.value) return
  const width = mountRef.value.clientWidth
  const height = mountRef.value.clientHeight

```

```

renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true })
renderer.shadowMap.enabled = true
renderer.outputColorSpace = THREE.SRGBColorSpace
renderer.toneMapping = THREE.ACESFilmicToneMapping
renderer.toneMappingExposure = 1
const dpr = typeof window !== 'undefined' ? Math.min(window.devicePixelRatio || 1, 2) : 1
renderer.setPixelRatio(dpr)
renderer.setSize(width, height)
renderer.setClearColor(0x000000, 0)
mountRef.value.appendChild(renderer.domElement)

scene = new THREE.Scene()

```

```

camera = new THREE.PerspectiveCamera(50, width / height, 0.1, 100)

```

```
camera.position.set(0.8, 1.1, 3.2)
camera.lookAt(0, 1, 0)

// Default lights (replaced if params.scene.lightPreset is set)
const hemi = new THREE.HemisphereLight(0xFFFFFF, 0xD8B4FE, 0.8)
scene.add(hemi)
defaultLights.push(hemi)
const ambient = new THREE.AmbientLight(0xffffff, 0.6)
scene.add(ambient)
defaultLights.push(ambient)
const dir = new THREE.DirectionalLight(0xffffff, 1.0)
dir.position.set(2, 4, 3)
dir.castShadow = true
scene.add(dir)
defaultLights.push(dir)

const orbitTarget = new THREE.Vector3(0, 0, 0)

const loader = new GLTFLoader()

try {
  const glb = await loader.loadAsync(props.modelUrl)
  model = glb.scene
  const tmpBox = new THREE.Box3()
  model.traverse((obj) => {
    if ((obj as THREE.Mesh).isMesh) {
      const mesh = obj as THREE.Mesh
      mesh.castShadow = true
      mesh.receiveShadow = true
      mesh.frustumCulled = false
      mesh.geometry.computeBoundingBox()
      mesh.geometry.computeBoundingSphere()
```

```
const mat = mesh.material
const mats = Array.isArray(mat) ? mat : [mat]
mats.forEach((m) => {
  ; (m as THREE.Material).side = THREE.DoubleSide
  ; (m as THREE.Material).needsUpdate = true
})
}
})

const targetRadius = 1.2
const initialBox = new THREE.Box3().setFromObject(model)
const initialSphere = initialBox.getBoundingSphere(new THREE.Sphere())
const initialCenter = initialSphere.center.clone()
const initialRadius = initialSphere.radius || 1
const safeRadius = Math.max(initialRadius, 1e-6)
const scaleFactor = targetRadius / safeRadius
model.scale.setScalar(scaleFactor)
model.position.copy(initialCenter).multiplyScalar(-scaleFactor)
model.frustumCulled = false
scene.add(model)
model.updateWorldMatrix(true, true)

const finalBox = new THREE.Box3().makeEmpty()
model.traverse((obj) => {
  if (!(obj as THREE.Mesh).isMesh) return
  const mesh = obj as THREE.Mesh
  const bbox = mesh.geometry.boundingBox
  if (!bbox) return
  tmpBox.copy(bbox).applyMatrix4(mesh.matrixWorld)
  finalBox.union(tmpBox)
})
const finalSphere = finalBox.getBoundingSphere(new THREE.Sphere())
const finalRadius = Math.max(finalSphere.radius || targetRadius, 1e-6)
```

```
const fov = THREE.MathUtils.degToRad(camera.fov)
const dist = finalRadius / Math.sin(fov / 2)
camera.near = Math.max(dist / 100, 0.01)
camera.far = dist * 100
camera.position.set(0, finalRadius * 0.2, dist * 1.15)
camera.lookAt(finalSphere.center)
camera.updateProjectionMatrix()
orbitTarget.copy(finalSphere.center)

// Apply params after model is ready
if (props.params) {
  if (props.params.scene?.lightPreset) applyLightPreset(props.params.scene.lightPreset)
  if (props.params.materials) applyMaterials(props.params.materials)
  if (props.params.logos?.length) applyLogos(props.params.logos)
}
} catch (e) {
  const text = document.createElement('div')
  text.textContent = '模型加载失败'
  text.style.cssText = 'position:absolute;inset:0;display:flex;align-items:center;justify-content:center;color:'
  mountRef.value.appendChild(text)
}

controls = new OrbitControls(camera, renderer.domElement)
controls.enableDamping = true
controls.enableZoom = false
controls.autoRotate = true
controls.autoRotateSpeed = 1.2
controls.minPolarAngle = Math.PI * 0.25
controls.maxPolarAngle = Math.PI * 0.75
controls.target.copy(orbitTarget)
controls.update()
```

```
const onResize = () => {
  if (!mountRef.value || !renderer || !camera) return
  const w = mountRef.value.clientWidth
  const h = mountRef.value.clientHeight
  renderer.setSize(w, h)
  camera.aspect = w / h
  camera.updateProjectionMatrix()
}
window.addEventListener('resize', onResize)
cleanupResize = () => window.removeEventListener('resize', onResize)

const animate = () => {
  if (!renderer || !scene || !camera) return
  if (!running) return
  controls?.update()
  renderer.render(scene, camera)
  rafId = requestAnimationFrame(animate)
}
animate()
}

onMounted(() => {
  init()
})

onUnmounted(() => {
  running = false
  if (rafId) cancelAnimationFrame(rafId)
  cleanupResize?.()
  cleanupResize = null
  if (controls) {
    controls.dispose()
  }
})
```

```
controls = null
}
if (model) {
  model.traverse((obj) => {
    if ((obj as THREE.Mesh).isMesh) {
      const mesh = obj as THREE.Mesh
      mesh.geometry.dispose()
      const mat = mesh.material
      if (Array.isArray(mat)) mat.forEach(m => m.dispose())
      else (mat as THREE.Material).dispose()
    }
  })
  model = null
}
if (renderer) {
  renderer.dispose()
  if (renderer.domElement && renderer.domElement.parentElement) {
    renderer.domElement.parentElement.removeChild(renderer.domElement)
  }
}
scene = null
camera = null
renderer = null
})
</script>

<style scoped>
.rounded-2xl {
  backdrop-filter: blur(2px);
}
</style>
```

后端代码

```
package model
```

```
import (  
    "context"  
    "database/sql"  
    "encoding/json"  
    "fmt"  
    "log"  
    "net/url"  
    "strings"  
    "time"  
  
    "github.com/chosetop/bishe_back/internal/config"  
)
```

```
const (  
    squareListCacheTTL = 60 * time.Second  
    squareDetailCacheTTL = 60 * time.Second  
    squareTagsCacheTTL = 300 * time.Second  
)
```

```
type squareListCacheValue struct {  
    List []SquareWorkItem `json:"list"`  
    Total int             `json:"total"`  
}
```

```
type squareDetailCacheValue struct {  
    Item *SquareWorkItem `json:"item"`  
    Comments []CommentItem `json:"comments"`  
}
```

```
func squareListCacheKey(page, pageSize int, tag, sort, keyword string) string {  
    return fmt.Sprintf("square:list:page:%d:size:%d:tag:%s:sort:%s:keyword:%s",  
        page,  
        pageSize,  
        url.QueryEscape(tag),  
        url.QueryEscape(sort),  
        url.QueryEscape(keyword),  
    )  
}
```

```
func squareDetailCacheKey(workID int64) string {  
    return fmt.Sprintf("square:detail:%d", workID)  
}
```

```
func loadSquareCache(key string, target any) bool {  
    if config.RedisClient == nil {  
        return false  
    }  
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)  
    defer cancel()  
    raw, err := config.RedisClient.Get(ctx, key).Result()  
    if err != nil {  
        return false  
    }  
    return json.Unmarshal([]byte(raw), target) == nil  
}
```

```
func saveSquareCache(key string, value any, ttl time.Duration) {  
    if config.RedisClient == nil {  
        return  
    }  
    data, err := json.Marshal(value)
```

```
if err != nil {
    return
}

ctx, cancel := context.WithTimeout(context.Background(), time.Second)
defer cancel()

_ = config.RedisClient.Set(ctx, key, data, ttl).Err()
}

func deleteSquareCacheByPattern(pattern string) {
    if config.RedisClient == nil {
        return
    }

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    iter := config.RedisClient.Scan(ctx, 0, pattern, 100).Iterator()
    keys := make([]string, 0, 100)
    for iter.Next(ctx) {
        keys = append(keys, iter.Val())
        if len(keys) >= 100 {
            _ = config.RedisClient.Del(ctx, keys...).Err()
            keys = keys[:0]
        }
    }

    if len(keys) > 0 {
        _ = config.RedisClient.Del(ctx, keys...).Err()
    }
}

// SquareWorkItem is the response shape for square list/detail
type SquareWorkItem struct {
    ID          int64      `json:"id"`
    AuthorID    int64      `json:"author_id"`,
```

```

Title    string    `json:"title"`
Description string    `json:"description"`
Author   string    `json:"author"`
Avatar   string    `json:"avatar"`
Likes    int       `json:"likes"`
Comments int       `json:"comments"`
Favorites int      `json:"favorites"`
IsFeatured bool    `json:"is_featured"`
IsLiked   bool     `json:"is_liked"`
IsFavorited bool    `json:"is_favorited"`
Tags      []string  `json:"tags"`
ParamsJSON json.RawMessage `json:"params_json"`
CreatedAt time.Time  `json:"created_at"`
UpdatedAt time.Time  `json:"updated_at"`
}

```

```

type CommentItem struct {
    ID      int64    `json:"id"`
    User    string   `json:"user"`
    Avatar  string   `json:"avatar"`
    Content string   `json:"content"`
    CreatedAt time.Time `json:"created_at"`
}

```

// ListSquareWorks returns published works with pagination/filter/sort.

// userID=0 means unauthenticated.

```

func ListSquareWorks(page, pageSize int, tag, sort, keyword string, userID int64) ([]SquareWorkItem, int) {
    if userID == 0 {
        key := squareListCacheKey(page, pageSize, tag, sort, keyword)
        var cached squareListCacheValue
        if loadSquareCache(key, &cached) {
            return cached.List, cached.Total, nil
        }
    }
}

```

```

    }
    list, total, err := listSquareWorksFromDB(page, pageSize, tag, sort, keyword, userID)
    if err != nil {
        return nil, 0, err
    }
    saveSquareCache(key, squareListCacheValue{List: list, Total: total}, squareListCacheTTL)
    return list, total, nil
}
return listSquareWorksFromDB(page, pageSize, tag, sort, keyword, userID)
}

func listSquareWorksFromDB(page, pageSize int, tag, sort, keyword string, userID int64) ([]SquareWork,
offset := (page - 1) * pageSize

where := []string{"w.status = 5"}
args := []any{}

if tag != "" && tag != "全部" {
    where = append(where, "EXISTS (SELECT 1 FROM work_tags wt WHERE wt.work_id = w.id AND w")
    args = append(args, tag)
}
if keyword != "" {
    where = append(where, "w.title LIKE ?")
    args = append(args, "%" + keyword + "%")
}

whereClause := " WHERE " + strings.Join(where, " AND ")

// count
countArgs := make([]any, len(args))
copy(countArgs, args)
var total int

```

```

if err := config.DB.QueryRow("SELECT COUNT(1) FROM design_works w"+whereClause, countArgs,
log.Printf("ListSquareWorks count query error where=%q args=%v err=%v", whereClause, countArgs, err)
return nil, 0, err
}

```

```

// order
var orderClause string
switch sort {
case "hot":
orderClause = " ORDER BY w.likes_count DESC, w.id DESC"
case "trend":
orderClause = ` ORDER BY (
SELECT COUNT(*) FROM work_likes wl
WHERE wl.work_id = w.id AND wl.created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
) DESC, w.id DESC`
default:
orderClause = " ORDER BY w.id DESC"
}

```

```

query := `SELECT w.id, w.user_id, w.title, COALESCE(w.description,"), u.username, COALESCE(u.avatar,"),
w.likes_count, w.comments_count, w.favorites_count, w.is_featured,
w.created_at, w.updated_at,
(SELECT dv.params_json FROM design_versions dv WHERE dv.id = w.current_version_id LIMIT 1)
FROM design_works w
JOIN users u ON u.id = w.user_id
LEFT JOIN design_versions dv ON dv.id = w.current_version_id` +
whereClause + orderClause + " LIMIT ? OFFSET ?"

```

```

args = append(args, pageSize, offset)
rows, err := config.DB.Query(query, args...)
if err != nil {
log.Printf("ListSquareWorks main query error query=%s args=%v err=%v", query, args, err)
}

```

```
return nil, 0, err
}
defer rows.Close()

list := make([]SquareWorkItem, 0, pageSize)
for rows.Next() {
    var item SquareWorkItem
    var paramsRaw []byte
    var authorName sql.NullString
    if err := rows.Scan(
        &item.ID, &item.AuthorID, &item.Title, &item.Description,
        &authorName, &item.Avatar,
        &item.Likes, &item.Comments, &item.Favorites, &item.IsFeatured,
        &item.CreatedAt, &item.UpdatedAt,
        &paramsRaw,
    ); err != nil {
        log.Printf("ListSquareWorks scan error err=%v", err)
        return nil, 0, err
    }
    if authorName.Valid {
        item.Author = authorName.String
    }
    if paramsRaw != nil {
        item.ParamsJSON = json.RawMessage(paramsRaw)
    }
    item.Tags = []string{}
    list = append(list, item)
}
if err := rows.Err(); err != nil {
    log.Printf("ListSquareWorks rows error err=%v", err)
    return nil, 0, err
}
```

```

if len(list) == 0 {
    return list, total, nil
}

// batch load tags
ids := make([]any, len(list))
idIndex := make(map[int64]int, len(list))
for i, item := range list {
    ids[i] = item.ID
    idIndex[item.ID] = i
}
placeholders := strings.Repeat("?", len(ids))
placeholders = placeholders[:len(placeholders)-1]
tagRows, err := config.DB.Query("SELECT work_id, tag FROM work_tags WHERE work_id IN (" + placeholders + ")")
if err == nil {
    defer tagRows.Close()
    for tagRows.Next() {
        var wid int64
        var t string
        if tagRows.Scan(&wid, &t) == nil {
            if idx, ok := idIndex[wid]; ok {
                list[idx].Tags = append(list[idx].Tags, t)
            }
        }
    }
} else {
    log.Printf("ListSquareWorks tags query error err=%v", err)
}

// batch load like/favorite status for authenticated user
if userID > 0 {

```

```
likeRows, err := config.DB.Query("SELECT work_id FROM work_likes WHERE user_id = ? AND wo
if err == nil {
    defer likeRows.Close()
    for likeRows.Next() {
        var wid int64
        if likeRows.Scan(&wid) == nil {
            if idx, ok := idIndex[wid]; ok {
                list[idx].IsLiked = true
            }
        }
    }
} else {
    log.Printf("ListSquareWorks like status query error err=%v", err)
}

favRows, err := config.DB.Query("SELECT work_id FROM work_favorites WHERE user_id = ? AND
if err == nil {
    defer favRows.Close()
    for favRows.Next() {
        var wid int64
        if favRows.Scan(&wid) == nil {
            if idx, ok := idIndex[wid]; ok {
                list[idx].IsFavorited = true
            }
        }
    }
} else {
    log.Printf("ListSquareWorks favorite status query error err=%v", err)
}
}

return list, total, nil
}
```

```
func ListMyFavorites(userID int64, page, pageSize int) ([]SquareWorkItem, int, error) {
    offset := (page - 1) * pageSize

    var total int
    if err := config.DB.QueryRow(`
        SELECT COUNT(1)
        FROM work_favorites f
        JOIN design_works w ON w.id = f.work_id
        WHERE f.user_id = ? AND w.status = 5
    `, userID).Scan(&total); err != nil {
        log.Printf("ListMyFavorites count query error userID=%d err=%v", userID, err)
        return nil, 0, err
    }

    rows, err := config.DB.Query(`
        SELECT
            w.id,
            w.user_id,
            w.title,
            COALESCE(w.description,""),
            u.username,
            COALESCE(u.avatar,""),
            w.likes_count,
            w.comments_count,
            w.favorites_count,
            w.is_featured,
            w.created_at,
            w.updated_at,
            dv.params_json
        FROM work_favorites f
        JOIN design_works w ON w.id = f.work_id
```

```
JOIN users u ON u.id = w.user_id
LEFT JOIN design_versions dv ON dv.id = w.current_version_id
WHERE f.user_id = ? AND w.status = 5
ORDER BY f.created_at DESC, f.id DESC
LIMIT ? OFFSET ?
`, userID, pageSize, offset)
if err != nil {
    log.Printf("ListMyFavorites main query error userID=%d page=%d pageSize=%d err=%v", userID, page,
    return nil, 0, err
}
defer rows.Close()

list := make([]SquareWorkItem, 0, pageSize)
for rows.Next() {
    var item SquareWorkItem
    var paramsRaw []byte
    if err := rows.Scan(
        &item.ID,
        &item.AuthorID,
        &item.Title,
        &item.Description,
        &item.Author,
        &item.Avatar,
        &item.Likes,
        &item.Comments,
        &item.Favorites,
        &item.IsFeatured,
        &item.CreatedAt,
        &item.UpdatedAt,
        &paramsRaw,
    ); err != nil {
        log.Printf("ListMyFavorites scan error err=%v", err)
```

```

    return nil, 0, err
}
if paramsRaw != nil {
    item.ParamsJSON = json.RawMessage(paramsRaw)
}
item.IsFavorited = true
item.Tags = []string{}
list = append(list, item)
}
if err := rows.Err(); err != nil {
    log.Printf("ListMyFavorites rows error err=%v", err)
    return nil, 0, err
}

if len(list) == 0 {
    return list, total, nil
}

ids := make([]any, len(list))
idIndex := make(map[int64]int, len(list))
for i, item := range list {
    ids[i] = item.ID
    idIndex[item.ID] = i
}
placeholders := strings.Repeat("?", len(ids))
placeholders = placeholders[:len(placeholders)-1]

tagRows, err := config.DB.Query("SELECT work_id, tag FROM work_tags WHERE work_id IN (" + placeholders + ")")
if err == nil {
    defer tagRows.Close()
    for tagRows.Next() {
        var wid int64

```

```
var t string
if tagRows.Scan(&wid, &t) == nil {
    if idx, ok := idIndex[wid]; ok {
        list[idx].Tags = append(list[idx].Tags, t)
    }
}
}
} else {
    log.Printf("ListMyFavorites tags query error err=%v", err)
}

likeRows, err := config.DB.Query("SELECT work_id FROM work_likes WHERE user_id = ? AND wo
if err == nil {
    defer likeRows.Close()
    for likeRows.Next() {
        var wid int64
        if likeRows.Scan(&wid) == nil {
            if idx, ok := idIndex[wid]; ok {
                list[idx].IsLiked = true
            }
        }
    }
} else {
    log.Printf("ListMyFavorites like status query error err=%v", err)
}

return list, total, nil
}

// GetSquareWorkByID returns a single published work with recent comments.
func GetSquareWorkByID(workID, userID int64) (*SquareWorkItem, []CommentItem, error) {
    if userID == 0 {
```

```

key := squareDetailCacheKey(workID)
var cached squareDetailCacheValue
if loadSquareCache(key, &cached) {
    return cached.Item, cached.Comments, nil
}
item, comments, err := getSquareWorkByIDFromDB(workID, userID)
if err != nil {
    return nil, nil, err
}
if item == nil {
    return nil, nil, nil
}
saveSquareCache(key, squareDetailCacheValue{Item: item, Comments: comments}, squareDetailCache)
return item, comments, nil
}
return getSquareWorkByIDFromDB(workID, userID)
}

```

```

func getSquareWorkByIDFromDB(workID, userID int64) (*SquareWorkItem, []CommentItem, error) {
    var item SquareWorkItem
    var paramsRaw []byte
    var authorName sql.NullString
    err := config.DB.QueryRow(`
        SELECT w.id, w.user_id, w.title, COALESCE(w.description,""), u.username, COALESCE(u.avatar,""),
        w.likes_count, w.comments_count, w.favorites_count, w.is_featured,
        w.created_at, w.updated_at,
        (SELECT dv.params_json FROM design_versions dv WHERE dv.id = w.current_version_id LIMIT 1)
        FROM design_works w
        JOIN users u ON u.id = w.user_id
        LEFT JOIN design_versions dv ON dv.id = w.current_version_id
        WHERE w.id = ? AND w.status = 5`, workID).Scan(
        &item.ID, &item.AuthorID, &item.Title, &item.Description,

```

```
&authorName, &item.Avatar,
&item.Likes, &item.Comments, &item.Favorites, &item.IsFeatured,
&item.CreatedAt, &item.UpdatedAt,
&paramsRaw,
)
if err != nil {
    if err == sql.ErrNoRows {
        return nil, nil, nil
    }
    return nil, nil, err
}
if authorName.Valid {
    item.Author = authorName.String
}
if paramsRaw != nil {
    item.ParamsJSON = json.RawMessage(paramsRaw)
}

// tags
item.Tags = []string{}
tagRows, err := config.DB.Query("SELECT tag FROM work_tags WHERE work_id = ?", workID)
if err == nil {
    defer tagRows.Close()
    for tagRows.Next() {
        var t string
        if tagRows.Scan(&t) == nil {
            item.Tags = append(item.Tags, t)
        }
    }
}

// like/favorite status
```

```

if userID > 0 {
    var cnt int
    config.DB.QueryRow("SELECT COUNT(1) FROM work_likes WHERE work_id = ? AND user_id = ?")
    item.IsLiked = cnt > 0
    config.DB.QueryRow("SELECT COUNT(1) FROM work_favorites WHERE work_id = ? AND user_id = ?")
    item.IsFavorited = cnt > 0
}

// recent comments (latest 20)
comments := []CommentItem{}
cRows, err := config.DB.Query(`
    SELECT c.id, u.username, COALESCE(u.avatar, ""), c.content, c.created_at
    FROM work_comments c
    JOIN users u ON u.id = c.user_id
    WHERE c.work_id = ?
    ORDER BY c.id DESC LIMIT 20`, workID)
if err == nil {
    defer cRows.Close()
    for cRows.Next() {
        var ci CommentItem
        if cRows.Scan(&ci.ID, &ci.User, &ci.Avatar, &ci.Content, &ci.CreatedAt) == nil {
            comments = append(comments, ci)
        }
    }
}

return &item, comments, nil
}

// GetWorkDetailForAdmin returns a single work (any status) with recent comments.
// It reuses the square response shape but does not compute like/favorite status.
func GetWorkDetailForAdmin(workID int64) (*SquareWorkItem, []CommentItem, error) {

```

```
var item SquareWorkItem
var paramsRaw []byte
var authorName sql.NullString
err := config.DB.QueryRow(`
    SELECT w.id, w.user_id, w.title, COALESCE(w.description,"), u.username, COALESCE(u.avatar,"),
    w.likes_count, w.comments_count, w.favorites_count, w.is_featured,
    w.created_at, w.updated_at,
    dv.params_json
    FROM design_works w
    JOIN users u ON u.id = w.user_id
    LEFT JOIN design_versions dv ON dv.id = w.current_version_id
    WHERE w.id = ?`, workID).Scan(
    &item.ID, &item.AuthorID, &item.Title, &item.Description,
    &authorName, &item.Avatar,
    &item.Likes, &item.Comments, &item.Favorites, &item.IsFeatured,
    &item.CreatedAt, &item.UpdatedAt,
    &paramsRaw,
)
if err != nil {
    if err == sql.ErrNoRows {
        return nil, nil, nil
    }
    return nil, nil, err
}
if authorName.Valid {
    item.Author = authorName.String
}
if paramsRaw != nil {
    item.ParamsJSON = json.RawMessage(paramsRaw)
}

item.Tags = []string{}
```

```

tagRows, err := config.DB.Query("SELECT tag FROM work_tags WHERE work_id = ?", workID)
if err == nil {
    defer tagRows.Close()
    for tagRows.Next() {
        var t string
        if tagRows.Scan(&t) == nil {
            item.Tags = append(item.Tags, t)
        }
    }
}

comments := []CommentItem{}
cRows, err := config.DB.Query(`
SELECT c.id, u.username, COALESCE(u.avatar,""), c.content, c.created_at
FROM work_comments c
JOIN users u ON u.id = c.user_id
WHERE c.work_id = ?
ORDER BY c.id DESC LIMIT 20`, workID)
if err == nil {
    defer cRows.Close()
    for cRows.Next() {
        var ci CommentItem
        if cRows.Scan(&ci.ID, &ci.User, &ci.Avatar, &ci.Content, &ci.CreatedAt) == nil {
            comments = append(comments, ci)
        }
    }
}

return &item, comments, nil
}

// ToggleLike toggles like for a user on a work. Returns (isLiked, newCount, error).

```

```
func ToggleLike(workID, userID int64) (bool, int, error) {
    var cnt int
    config.DB.QueryRow("SELECT COUNT(1) FROM work_likes WHERE work_id = ? AND user_id = ?")

    if cnt > 0 {
        // unlike
        if _, err := config.DB.Exec("DELETE FROM work_likes WHERE work_id = ? AND user_id = ?", workID, userID); err != nil {
            return false, 0, err
        }
        if _, err := config.DB.Exec("UPDATE design_works SET likes_count = GREATEST(0, likes_count - 1) WHERE id = ?", workID); err != nil {
            return false, 0, err
        }
    } else {
        // like
        if _, err := config.DB.Exec("INSERT INTO work_likes(work_id, user_id) VALUES(?,?)", workID, userID); err != nil {
            return false, 0, err
        }
        if _, err := config.DB.Exec("UPDATE design_works SET likes_count = likes_count + 1 WHERE id = ?", workID); err != nil {
            return false, 0, err
        }
    }

    var newCount int
    config.DB.QueryRow("SELECT likes_count FROM design_works WHERE id = ?", workID).Scan(&newCount)
    invalidateSquareWorkCache(workID)
    return cnt == 0, newCount, nil
}
```

// ToggleFavorite toggles favorite. Returns (isFavorited, newCount, error).

```
func ToggleFavorite(workID, userID int64) (bool, int, error) {
    var cnt int
    config.DB.QueryRow("SELECT COUNT(1) FROM work_favorites WHERE work_id = ? AND user_id = ?")
```

```

if cnt > 0 {
    if _, err := config.DB.Exec("DELETE FROM work_favorites WHERE work_id = ? AND user_id = ?", workID, userID); err != nil {
        return false, 0, err
    }
    if _, err := config.DB.Exec("UPDATE design_works SET favorites_count = GREATEST(0, favorites_count - 1) WHERE id = ?", workID); err != nil {
        return false, 0, err
    }
} else {
    if _, err := config.DB.Exec("INSERT INTO work_favorites(work_id, user_id) VALUES(?,?)", workID, userID); err != nil {
        return false, 0, err
    }
    if _, err := config.DB.Exec("UPDATE design_works SET favorites_count = favorites_count + 1 WHERE id = ?", workID); err != nil {
        return false, 0, err
    }
}

var newCount int
config.DB.QueryRow("SELECT favorites_count FROM design_works WHERE id = ?", workID).Scan(&newCount)
invalidateSquareWorkCache(workID)
return cnt == 0, newCount, nil
}

// AddComment inserts a comment and increments comments_count.
func AddComment(workID, userID int64, content string) (*CommentItem, error) {
    res, err := config.DB.Exec("INSERT INTO work_comments(work_id, user_id, content) VALUES(?,?,?)", workID, userID, content); err != nil {
        if err != nil {
            return nil, err
        }
    }
    commentID, _ := res.LastInsertId()
    config.DB.Exec("UPDATE design_works SET comments_count = comments_count + 1 WHERE id = ?", workID)
}

```

```
var ci CommentItem
ci.ID = commentID
config.DB.QueryRow(`
    SELECT u.username, COALESCE(u.avatar,""), c.content, c.created_at
    FROM work_comments c JOIN users u ON u.id = c.user_id
    WHERE c.id = ?`, commentID).Scan(&ci.User, &ci.Avatar, &ci.Content, &ci.CreatedAt)
invalidateSquareWorkCache(workID)
return &ci, nil
}
```

// ListSquareTags returns top 20 tags used in published works, by frequency.

```
func ListSquareTags() ([]string, error) {
    var cached []string
    if loadSquareCache("square:tags", &cached) {
        return cached, nil
    }
    tags, err := listSquareTagsFromDB()
    if err != nil {
        return nil, err
    }
    saveSquareCache("square:tags", tags, squareTagsCacheTTL)
    return tags, nil
}
```

```
func listSquareTagsFromDB() ([]string, error) {
    rows, err := config.DB.Query(`
        SELECT wt.tag FROM work_tags wt
        JOIN design_works w ON w.id = wt.work_id
        WHERE w.status = 5
        GROUP BY wt.tag ORDER BY COUNT(*) DESC LIMIT 20`)
    if err != nil {
        return nil, err
    }
}
```

```
}
defer rows.Close()
tags := []string{"全部"}
for rows.Next() {
    var t string
    if rows.Scan(&t) == nil {
        tags = append(tags, t)
    }
}
return tags, rows.Err()
}

// SetWorkTags replaces all tags for a work.
func SetWorkTags(workID int64, tags []string) error {
    tx, err := config.DB.Begin()
    if err != nil {
        return err
    }
    if _, err := tx.Exec("DELETE FROM work_tags WHERE work_id = ?", workID); err != nil {
        tx.Rollback()
        return err
    }
    for _, t := range tags {
        t = strings.TrimSpace(t)
        if t == "" {
            continue
        }
        if _, err := tx.Exec("INSERT INTO work_tags(work_id, tag) VALUES(?,?)", workID, t); err != nil {
            tx.Rollback()
            return err
        }
    }
}
```

```
if err := tx.Commit(); err != nil {  
    return err  
}  
invalidateSquareWorkCache(workID)  
invalidateSquareTagsCache()  
return nil  
}  
  
func invalidateSquareListCache() {  
    deleteSquareCacheByPattern("square:list:*")  
}  
  
func invalidateSquareDetailCache(workID int64) {  
    deleteSquareCacheByPattern(squareDetailCacheKey(workID))  
}  
  
func invalidateSquareTagsCache() {  
    deleteSquareCacheByPattern("square:tags")  
}  
  
func invalidateSquareWorkCache(workID int64) {  
    invalidateSquareListCache()  
    invalidateSquareDetailCache(workID)  
}
```

附录 E 作品管理模块

前端代码

```
<template>
```

```
<div class="min-h-screen bg-[#F8FAFC] text-slate-800 relative overflow-hidden">
```

```
<!-- Background Elements -->
```

```
<div class="fixed top-0 left-0 w-full h-full overflow-hidden pointer-events-none z-0">
```

```
<div class="absolute top-[-20%] right-[-10%] w-[600px] h-[600px] rounded-full bg-purple-200/30 bl
```

```
<div class="absolute bottom-[-10%] left-[-10%] w-[500px] h-[500px] rounded-full bg-blue-200/30 bl
```

```
</div>
```

```
<div class="max-w-7xl mx-auto px-6 py-12 relative z-10">
```

```
<!-- Header Section -->
```

```
<div class="flex flex-col md:flex-row justify-between items-start md:items-center mb-12 gap-6">
```

```
<div>
```

```
<h1 class="text-4xl md:text-5xl font-black tracking-tight text-slate-900 mb-2 font-display">
```

```
  我的作品 <span class="text-purple-600">.</span>
```

```
</h1>
```

```
<p class="text-slate-500 text-lg font-light tracking-wide">
```

```
  这里展示了所有的设计与创作
```

```
</p>
```

```
</div>
```

```
<el-button type="primary" size="large"
```

```
  class="!rounded-full !px-8 !py-6 !text-lg !font-bold shadow-lg shadow-purple-500/30 hover:!shado
```

```
  @click="router.push('/design')">
```

```
<el-icon class="mr-2">
```

```
<Plus />
```

```
</el-icon>
```

```
  开始创作
```

```
</el-button>
```

```
</div>
```

```
<!-- Filter Tabs -->
```

```

<div
  class="mb-10 flex flex-wrap gap-2 p-1.5 bg-white/50 backdrop-blur-sm rounded-2xl w-fit border bo
  <button v-for="tab in tabs" :key="tab.value" @click="activeTab = tab.value"
    class="px-6 py-2.5 rounded-xl text-sm font-medium transition-all duration-300 relative overflow-h
    :class="activeTab === tab.value ? 'text-white shadow-md' : 'text-slate-500 hover:text-slate-900 hov
    <div v-if="activeTab === tab.value" class="absolute inset-0 bg-slate-900 rounded-xl"></div>
    <span class="relative z-10">{{ tab.label }}</span>
  </button>
</div>

<!-- Loading State -->
<div v-if="loading" class="flex flex-col items-center justify-center py-32 animate-pulse">
  <div class="w-16 h-16 border-4 border-purple-200 border-t-purple-600 rounded-full animate-spin m
  <p class="text-slate-400 font-mono text-sm">LOADING ASSETS...</p>
</div>

<!-- Empty State -->
<div v-else-if="works.length === 0" class="flex flex-col items-center justify-center py-32 text-center
  <div class="w-24 h-24 bg-slate-100 rounded-full flex items-center justify-center mb-6 text-slate-300
    <el-icon :size="40">
      <FolderOpened />
    </el-icon>
  </div>
  <h3 class="text-xl font-bold text-slate-900 mb-2">暂无作品</h3>
  <p class="text-slate-500 mb-8 max-w-md">
    看起来你还没有创建任何作品。点击右上角的按钮开始你的第一次创作吧。
  </p>
  <el-button type="primary" plain round @click="router.push('/design')">
    立即创建
  </el-button>
</div>

```

```

<!-- Grid Layout -->
<div v-else class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
  <div v-for="(item, index) in works" :key="item.work?.id || index"
    class="group relative bg-white rounded-[2rem] overflow-hidden border border-slate-100 shadow-[
  <!-- Preview Area -->
  <div class="h-72 bg-slate-50 relative cursor-pointer overflow-hidden" @click="handleWorkClick(
    <!-- 3D Preview or Fallback -->
    <div class="w-full h-full relative z-10 transition-transform duration-700 ease-out group-hover:sc
      <ModelPreview v-if="getModelUrl(item)" :model-url="getModelUrl(item)" :params="getParam
        class="w-full h-full" />
      <div v-else class="w-full h-full flex items-center justify-center text-slate-200 bg-slate-50">
        <el-icon :size="64">
          <Picture />
        </el-icon>
      </div>
    </div>
  </div>

  <!-- Overlay Gradient on Hover -->
  <div
    class="absolute inset-0 bg-gradient-to-t from-black/60 via-transparent to-transparent opacity-0 g
  </div>

  <!-- Status Badge -->
  <div class="absolute top-4 right-4 z-30">
    <span
      class="px-3 py-1 rounded-full text-xs font-bold uppercase tracking-wider backdrop-blur-md bo
      :class="getStatusClass(item.work?.status)">
      {{ getStatusLabel(item.work?.status) }}
    </span>
  </div>
</div>

```

```

<!-- Content Area -->
<div class="p-6 flex flex-col flex-1 relative z-30 bg-white">
  <div class="mb-4">
    <h3
      class="text-xl font-bold text-slate-900 mb-2 truncate group-hover:text-purple-600 transition-co
      :title="item.work?.title">
      {{ item.work?.title || '未命名作品' }}
    </h3>
    <p class="text-slate-500 text-sm line-clamp-2 h-10 leading-relaxed">
      {{ item.work?.description || '暂无描述信息...' }}
    </p>
  </div>

  <div class="mt-auto pt-4 border-t border-slate-50 flex justify-between items-center">
    <span class="text-xs font-mono text-slate-400">
      {{ formatDate(item.work?.updated_at) }}
    </span>

    <!-- Action Buttons -->
    <div
      class="flex gap-2 opacity-0 group-hover:opacity-100 transition-all duration-300 transform tran
    <template v-if="[1, 4].includes(item.work?.status)">
      <button class="p-2 rounded-full hover:bg-purple-50 text-purple-600 transition-colors"
        @click.stop="editWork(item.work.id)" title="编辑">
        <el-icon>
          <Edit />
        </el-icon>
      </button>
      <button v-if="item.work?.status === 1"
        class="p-2 rounded-full hover:bg-green-50 text-green-600 transition-colors"
        @click.stop="submitWork(item.work.id)" title="提交审核">
        <el-icon>

```

```
<Check />
</el-icon>
</button>
</template>
<button v-else class="p-2 rounded-full hover:bg-blue-50 text-blue-600 transition-colors"
  @click.stop="viewDetail(item.work.id)" title="查看详情">
  <el-icon>
    <View />
  </el-icon>
</button>

<button v-if="item.work?.status === 5"
  class="p-2 rounded-full hover:bg-orange-50 text-orange-600 transition-colors"
  @click.stop="handleUnpublish(item.work.id)" title="下架">
  <el-icon>
    <Bottom />
  </el-icon>
</button>

<button v-if="![2, 5].includes(item.work?.status)"
  class="p-2 rounded-full hover:bg-red-50 text-red-600 transition-colors"
  @click.stop="handleDelete(item.work.id)" title="删除">
  <el-icon>
    <Delete />
  </el-icon>
</button>
</div>
</div>
</div>
</div>
</div>
```

```
<!-- Pagination -->
<div class="mt-16 flex justify-center" v-if="total > pageSize">
  <el-pagination v-model:current-page="currentPage" v-model:page-size="pageSize" :total="total"
    layout="prev, pager, next" @current-change="fetchData" background class="!gap-2" />
</div>
</div>
</div>
</template>

<script setup lang="ts">
import { ref, onMounted, watch } from 'vue'
import { useRouter } from 'vue-router'
import {
  Loading,
  Picture,
  FolderOpened,
  Plus,
  Edit,
  Check,
  View,
  Delete,
  Bottom
} from '@element-plus/icons-vue'
import { ElMessage, ElMessageBox } from 'element-plus'

const router = useRouter()
const { fetchWorks, submitWorkReview, unpublishWork, deleteWork } = useWorks()

const works = ref<any[]>([])
const loading = ref(false)
const activeTab = ref('all')
const currentPage = ref(1)
```

```
const pageSize = ref(12)
const total = ref(0)

const tabs = [
  { label: '全部', value: 'all' },
  { label: '草稿箱', value: '1' },
  { label: '审核中', value: '2' },
  { label: '已发布', value: '5' },
  { label: '需修改', value: '4' },
]

const statusMap: Record<number, { label: string; class: string }> = {
  1: { label: '草稿', class: 'bg-slate-100 text-slate-600' },
  2: { label: '审核中', class: 'bg-amber-100 text-amber-700' },
  3: { label: '已通过', class: 'bg-emerald-100 text-emerald-700' },
  4: { label: '已驳回', class: 'bg-red-100 text-red-700' },
  5: { label: '已发布', class: 'bg-purple-100 text-purple-700' }
}

const getStatusLabel = (status: number) => statusMap[status]?.label || '未知'
const getStatusClass = (status: number) => statusMap[status]?.class || 'bg-gray-100 text-gray-500'

const formatDate = (dateStr: string) => {
  if (!dateStr) return ''
  return new Date(dateStr).toLocaleDateString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit'
  })
}

// Helper to parse params_json
```

```
const getParams = (item: any): Record<string, any> | null => {
  if (!item?.version?.params_json) return null
  try {
    return typeof item.version.params_json === 'string'
      ? JSON.parse(item.version.params_json)
      : item.version.params_json
  } catch (e) {
    console.error('Error parsing params_json:', e)
    return null
  }
}

const getModelUrl = (item: any) => getParams(item)?.modelUrl ?? null

const fetchData = async () => {
  loading.value = true
  try {
    const status = activeTab.value === 'all' ? undefined : parseInt(activeTab.value)
    const res = await fetchWorks({
      page: currentPage.value,
      page_size: pageSize.value,
      status: status
    })

    if (res.success) {
      works.value = res.data.list
      total.value = res.data.total
    }
  } catch (error) {
    console.error(error)
    ElMessage.error('获取作品列表失败')
  } finally {
```

```
    loading.value = false
  }
}

watch(activeTab, () => {
  currentPage.value = 1
  fetchData()
})

const handleWorkClick = (work: any) => {
  if (!work) return
  if ([1, 4].includes(work.status)) {
    editWork(work.id)
  } else {
    viewDetail(work.id)
  }
}

const editWork = (id: number) => {
  router.push(`/design/edit/${id}`)
}

const viewDetail = (id: number) => {
  router.push(`/design/edit/${id}?mode=view`)
}

const submitWork = async (id: number) => {
  try {
    await ElMessageBox.confirm('确定要提交审核吗?提交后将无法修改。','提示',{
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning',
    })
  } catch {}
}
```

```
        center: true,
        customClass: '!rounded-2xl'
    })

    await submitWorkReview(id)
    ElMessage.success('提交成功')
    fetchData()
  } catch (e) {
    // Cancelled or error
  }
}

const handleUnpublish = async (id: number) => {
  try {
    await ElMessageBox.confirm('确定要下架该作品吗?下架后将从创意广场移除。','提示',{
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning',
      center: true,
      customClass: '!rounded-2xl'
    })

    await unpublishWork(id)
    ElMessage.success('下架成功')
    fetchData()
  } catch (e) {
    // Cancelled or error
  }
}

const handleDelete = async (id: number) => {
  try {
```

```

    await ElMessageBox.confirm('确定要删除该作品吗? 此操作不可恢复。', '警告', {
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'error',
      center: true,
      customClass: '!rounded-2xl'
    })

    await deleteWork(id)
    ElMessage.success('删除成功')
    fetchData()
  } catch (e) {
    // Cancelled or error
  }
}

onMounted(() => {
  fetchData()
})
</script>

<style scoped>
/* Custom Font Face if needed, otherwise using system stack with preferences */
.font-display {
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Can

}

/* Custom Scrollbar for the page if needed */
::-webkit-scrollbar {
  width: 8px;
}

```

```

::-webkit-scrollbar-track {
  background: transparent;
}

::-webkit-scrollbar-thumb {
  background-color: #cbd5e1;
  border-radius: 4px;
}

::-webkit-scrollbar-thumb:hover {
  background-color: #94a3b8;
}

/* Override Element Plus Pagination Styles */
:deep(.el-pagination.is-background .el-pager li:not(.is-disabled).is-active) {
  background-color: #0f172a !important;
  color: #fff;
  border-radius: 8px;
}

:deep(.el-pagination.is-background .el-pager li) {
  border-radius: 8px;
  background-color: #fff;
  border: 1px solid #e2e8f0;
}
</style>

```

后端代码

```
package model
```

```
import (
  "database/sql"
```

```

"encoding/json"
"fmt"
"strings"
"time"

"github.com/chosetop/bishe_back/internal/config"
)

```

```

type DesignVersion struct {
    ID      int64      `json:"id"`
    WorkID   int64      `json:"work_id"`
    VersionNo int        `json:"version_no"`
    ParamsJSON json.RawMessage `json:"params_json"`
    ModelFileID *int64     `json:"model_file_id"`
    SubmittedAt *time.Time `json:"submitted_at"`
    CreatedAt   time.Time  `json:"created_at"`
    UpdatedAt   time.Time  `json:"updated_at"`
}

```

```

type versionScanner interface {
    Scan(dest ...any) error
}

```

```

func scanDesignVersion(scanner versionScanner) (*DesignVersion, error) {
    var v DesignVersion
    var modelID sql.NullInt64
    var submittedAt sql.NullTime
    var paramsRaw []byte
    if err := scanner.Scan(
        &v.ID,
        &v.WorkID,
        &v.VersionNo,

```

```
&paramsRaw,
&modelID,
&submittedAt,
&v.CreatedAt,
&v.UpdatedAt,
); err != nil {
    return nil, err
}
if modelID.Valid {
    val := modelID.Int64
    v.ModelFileID = &val
}
if submittedAt.Valid {
    val := submittedAt.Time
    v.SubmittedAt = &val
}
if paramsRaw != nil {
    v.ParamsJSON = json.RawMessage(paramsRaw)
}
return &v, nil
}
```

```
func CreateDesignVersion(workID int64, versionNo int, paramsJSON json.RawMessage, modelFileID *int64) (
res, err := config.DB.Exec(
    "INSERT INTO design_versions(work_id, version_no, params_json, model_file_id) VALUES(?, ?, ?, ?)",
    workID,
    versionNo,
    paramsJSON,
    modelFileID,
)
if err != nil {
    return 0, err
}
```

```
}  
return res.LastInsertId()  
}  
  
func GetDesignVersionByID(id int64) (*DesignVersion, error) {  
    row := config.DB.QueryRow(  
        "SELECT id, work_id, version_no, params_json, model_file_id, submitted_at, created_at, updated_at FROM design_version WHERE id = ?",  
        id,  
    )  
    v, err := scanDesignVersion(row)  
    if err != nil {  
        if err == sql.ErrNoRows {  
            return nil, nil  
        }  
        return nil, err  
    }  
    return v, nil  
}  
  
func GetLatestDesignVersionByWorkID(workID int64) (*DesignVersion, error) {  
    row := config.DB.QueryRow(  
        "SELECT id, work_id, version_no, params_json, model_file_id, submitted_at, created_at, updated_at FROM design_version WHERE work_id = ?",  
        workID,  
    )  
    v, err := scanDesignVersion(row)  
    if err != nil {  
        if err == sql.ErrNoRows {  
            return nil, nil  
        }  
        return nil, err  
    }  
    return v, nil  
}
```

```
}
```

```
func ListDesignVersionsByWorkID(workID int64) ([]DesignVersion, error) {  
    rows, err := config.DB.Query(  
        "SELECT id, work_id, version_no, params_json, model_file_id, submitted_at, created_at, updated_at FROM design_versions WHERE work_id = ?",  
        workID,  
    )  
    if err != nil {  
        return nil, err  
    }  
    defer rows.Close()
```

```
  
    list := make([]DesignVersion, 0)  
    for rows.Next() {  
        v, err := scanDesignVersion(rows)  
        if err != nil {  
            return nil, err  
        }  
        list = append(list, *v)  
    }  
    if err := rows.Err(); err != nil {  
        return nil, err  
    }  
    return list, nil  
}
```

```
func UpdateDesignVersionSubmittedAt(id int64, submittedAt time.Time) (int64, error) {  
    res, err := config.DB.Exec("UPDATE design_versions SET submitted_at = ? WHERE id = ?", submittedAt, id)  
    if err != nil {  
        return 0, err  
    }  
    return res.RowsAffected()  
}
```

```
}
```

```
func UpdateDesignVersionFields(id int64, paramsJSON json.RawMessage, modelFileID *int64, clearSub
```

```
fields := make([]string, 0, 3)
```

```
args := make([]any, 0, 3)
```

```
if len(paramsJSON) > 0 {
```

```
fields = append(fields, "params_json = ?")
```

```
args = append(args, paramsJSON)
```

```
}
```

```
if modelFileID != nil {
```

```
fields = append(fields, "model_file_id = ?")
```

```
args = append(args, *modelFileID)
```

```
}
```

```
if clearSubmittedAt {
```

```
fields = append(fields, "submitted_at = NULL")
```

```
}
```

```
if len(fields) == 0 {
```

```
return 0, nil
```

```
}
```

```
args = append(args, id)
```

```
query := fmt.Sprintf("UPDATE design_versions SET %s WHERE id = ?", strings.Join(fields, ", "))
```

```
res, err := config.DB.Exec(query, args...)
```

```
if err != nil {
```

```
return 0, err
```

```
}
```

```
return res.RowsAffected()
```

```
}
```

附录 F 后台管理模块

前端代码

// 模型管理

<template>

<div class="template-list">

<div class="bg-white p-4 rounded-xl shadow-sm mb-4 flex justify-between items-center">

<div class="flex gap-4">

<el-input v-model="searchKeyword" placeholder="搜索模板名称" prefix-icon="Search" class="flex-grow-1">

@clear="handleSearch" @keyup.enter="handleSearch" />

<el-button type="primary" :icon="Search" @click="handleSearch">搜索</el-button>

</div>

<el-button type="primary" :icon="Plus" @click="handleAdd">新增模板</el-button>

</div>

<div class="bg-white p-4 rounded-xl shadow-sm">

<el-table :data="tableData" style="width: 100%" stripe v-loading="loading">

<el-table-column prop="name" label="名称" min-width="160" />

<el-table-column prop="url" label="文件地址" min-width="260" show-overflow-tooltip>

<template #default="{ row }">

<div class="flex items-center gap-2">

{{ row.url }}

<el-button link type="primary" @click="handleCopy(row.url)">复制</el-button>

</div>

</template>

</el-table-column>

<el-table-column prop="size" label="大小(Byte)" width="120" />

<el-table-column prop="created_at" label="创建时间" width="180">

<template #default="{ row }">

{{ formatDate(row.created_at) }}

</template>

</el-table-column>

<el-table-column prop="updated_at" label="更新时间" width="180">

```

    <template #default="{ row }">
      {{ formatDate(row.updated_at) }}
    </template>
  </el-table-column>
  <el-table-column label="操作" width="220" fixed="right">
    <template #default="{ row }">
      <el-button link type="primary" :icon="Edit" @click="handleEdit(row)">编辑</el-button>
      <el-button link type="danger" :icon="Delete" @click="handleDelete(row)">删除</el-button>
    </template>
  </el-table-column>
</el-table>

<div class="mt-4 flex justify-end">
  <el-pagination v-model:current-page="currentPage" v-model:page-size="pageSize"
    :page-sizes="[10, 20, 50, 100]" layout="total, sizes, prev, pager, next, jumper" :total="total" />
</div>
</div>

<el-dialog v-model="dialogVisible" :title="dialogType === 'add' ? '新增模板' : '编辑模板'" width="500px">
  <el-form ref="formRef" :model="formData" :rules="rules" label-width="90px">
    <el-form-item label="模板名称" prop="name">
      <el-input v-model="formData.name" placeholder="请输入模板名称" />
    </el-form-item>
    <el-form-item v-if="dialogType === 'add'" label="模板文件" prop="path">
      <el-upload v-model:file-list="fileList" :auto-upload="false" :limit="1"
        :on-change="handleFileChange" :before-remove="handleBeforeRemove" accept=".png,.jpg,.jpeg"
        class="w-full">
        <el-button type="primary">选择文件</el-button>
      <template #tip>
        <div class="text-xs text-gray-400 mt-1">
          将通过后台上传到 OSS 并自动写入模型表
        </div>
      </template>
    </el-form-item>
  </el-form>
</el-dialog>

```

```
        </el-upload>
      </el-form-item>
    </el-form>
    <template #footer>
      <div class="dialog-footer">
        <el-button @click="dialogVisible = false">取消</el-button>
        <el-button type="primary" @click="handleSubmit(formRef)">确定</el-button>
      </div>
    </template>
  </el-dialog>
</div>
</template>

<script setup lang="ts">
import { ref, reactive, onMounted, watch } from 'vue'
import { Plus, Search, Edit, Delete } from '@element-plus/icons-vue'
import { ElMessage, ElMessageBox, type UploadUserFile } from 'element-plus'
import dayjs from 'dayjs'
import { getModels, deleteModel, updateModel, uploadModel } from '@api/models'
import type { ModelFile } from '@api/types'

const tableData = ref<ModelFile[]>([])
const loading = ref(false)

const searchKeyword = ref("")
const currentPage = ref(1)
const pageSize = ref(10)
const total = ref(0)

const dialogVisible = ref(false)
const dialogType = ref<'add' | 'edit'>('add')
const formRef = ref()
```

```
const fileList = ref<UploadUserFile[]>([])

const formData = reactive({
  id: 0,
  name: "",
  path: ""
})

const rules = {
  name: [{ required: false, message: '请输入名称', trigger: 'blur' }],
  path: [{ required: true, message: '请上传模板文件', trigger: 'change' }]
}

const formatDate = (value: string) => {
  if (!value) return ""
  return dayjs(value).format('YYYY-MM-DD HH:mm:ss')
}

const fetchData = async () => {
  loading.value = true
  try {
    const res = await getModels({
      page: currentPage.value,
      page_size: pageSize.value,
      name: searchKeyword.value || undefined
    })
    if (res.success) {
      tableData.value = res.data.list
      total.value = res.data.total
      if (res.data.page) currentPage.value = res.data.page
      if (res.data.page_size) pageSize.value = res.data.page_size
    }
  }
```

```
    } finally {  
      loading.value = false  
    }  
  }  
}  
  
onMounted(() => {  
  fetchData()  
})  
  
watch([currentPage, pageSize], () => {  
  fetchData()  
})  
  
const handleSearch = () => {  
  currentPage.value = 1  
  fetchData()  
}  
  
const handleAdd = () => {  
  dialogType.value = 'add'  
  dialogVisible.value = true  
  formData.id = 0  
  formData.name = "  
  formData.path = "  
  fileList.value = []  
}  
  
const handleEdit = (row: ModelFile) => {  
  dialogType.value = 'edit'  
  dialogVisible.value = true  
  formData.id = row.id  
  formData.name = row.name
```

```
    formData.path = "
    fileList.value = []
  }

const handleFileChange = (_file: UploadUserFile, files: UploadUserFile[]) => {
  fileList.value = files
  const first = files[0]
  if (first && first.name) {
    formData.path = first.name
  } else {
    formData.path = "
  }
}

const handleBeforeRemove = () => {
  formData.path = "
  return true
}

const handleSubmit = async (formEl: any) => {
  if (!formEl) return
  await formEl.validate(async (valid: boolean) => {
    if (!valid) return

    if (!fileList.value.length && dialogType.value === 'add') {
      ElMessage.error('请先选择模板文件')
      return
    }

    if (dialogType.value === 'add') {
      const first = fileList.value[0]
      if (!first || !first.raw) {
```

```
    ElMessage.error('请选择有效的模板文件')
    return
  }
  const uploadRes = await uploadModel(first.raw as File, 'models')
  const modelRes = uploadRes as unknown as { success: boolean; model: ModelFile }
  let model = modelRes.model

  if (formData.name && model && formData.name !== model.name) {
    const updateRes = await updateModel(model.id, { name: formData.name })
    model = updateRes.model
  }

  ElMessage.success('新增模板成功')
  dialogVisible.value = false
  fetchData()
} else {
  const currentId = formData.id
  let finalModel: ModelFile | null = null

  if (formData.name) {
    const updateRes = await updateModel(currentId, { name: formData.name })
    finalModel = updateRes.model
  }

  if (finalModel) {
    tableData.value = tableData.value.map(item =>
      item.id === currentId ? finalModel as ModelFile : item
    )
  }

  ElMessage.success('编辑模板成功')
  dialogVisible.value = false
```

```

    }
  })
}

const handleCopy = async (text: string) => {
  try {
    if (navigator.clipboard && navigator.clipboard.writeText) {
      await navigator.clipboard.writeText(text)
    } else {
      const textarea = document.createElement('textarea')
      textarea.value = text
      textarea.style.position = 'fixed'
      textarea.style.left = '-9999px'
      document.body.appendChild(textarea)
      textarea.select()
      document.execCommand('copy')
      document.body.removeChild(textarea)
    }
    ElMessage.success('已复制到剪贴板')
  } catch (error) {
    ElMessage.error('复制失败，请手动复制')
  }
}

const handleDelete = (row: ModelFile) => {
  ElMessageBox.confirm(`确定删除模板「${row.name}」吗?`, '提示', {
    confirmButtonText: '确定',
    cancelButtonText: '取消',
    type: 'warning'
  })
  .then(async () => {
    await deleteModel(row.id)
  })
}

```

```
    ElMessage.success('删除成功')
    fetchData()
  })
  .catch(() => { })
}
</script>
```

```
<style scoped>
```

```
.template-list {
  min-height: 100%;
}
</style>
```

```
// 作品审核
```

```
<template>
```

```
<div class="work-list">
```

```
<div class="bg-white p-4 rounded-xl shadow-sm mb-4 flex justify-between items-center">
```

```
<div class="flex gap-4 items-center">
```

```
<span class="text-sm text-gray-500">作品状态</span>
```

```
<el-select
```

```
  v-model="statusFilter"
```

```
  placeholder="请选择状态"
```

```
  class="w-48"
```

```
  clearable
```

```
  @change="handleStatusChange"
```

```
>
```

```
<el-option
```

```
  v-for="item in statusOptions"
```

```
  :key="item.value"
```

```
  :label="item.label"
```

```
  :value="item.value"
```

```
/>
```

```

</el-select>
</div>
<el-button type="primary" :icon="Search" @click="fetchData">刷新</el-button>
</div>

<div class="bg-white p-4 rounded-xl shadow-sm">
  <el-table :data="tableData" style="width: 100%" stripe v-loading="loading">
    <el-table-column prop="id" label="ID" width="80" />
    <el-table-column prop="title" label="标题" min-width="180" />
    <el-table-column prop="description" label="描述" min-width="240" show-overflow-tooltip />
    <el-table-column prop="status" label="状态" width="120">
      <template #default="{ row }">
        <el-tag :type="getStatusTagType(row.status)">
          {{ getStatusText(row.status) }}
        </el-tag>
      </template>
    </el-table-column>
    <el-table-column prop="created_at" label="创建时间" width="180">
      <template #default="{ row }">
        {{ formatDate(row.created_at) }}
      </template>
    </el-table-column>
    <el-table-column prop="updated_at" label="更新时间" width="180">
      <template #default="{ row }">
        {{ formatDate(row.updated_at) }}
      </template>
    </el-table-column>
    <el-table-column prop="published_at" label="发布时间" width="180">
      <template #default="{ row }">
        {{ formatDate(row.published_at) }}
      </template>
    </el-table-column>
  </el-table>
</div>

```

```

<el-table-column label="操作" width="220" fixed="right">
  <template #default="{ row }">
    <el-button link type="primary" :icon="View">查看</el-button>
    <el-button
      v-if="row.status === 2"
      link
      type="success"
      :icon="Check"
      @click="openReview(row)"
    >
      审核
    </el-button>
  </template>
</el-table-column>
</el-table>
<div class="mt-4 flex justify-end">
  <el-pagination
    v-model:current-page="currentPage"
    v-model:page-size="pageSize"
    :page-sizes="[10, 20, 50, 100]"
    layout="total, sizes, prev, pager, next, jumper"
    :total="total"
  />
</div>
</div>

<el-dialog v-model="reviewDialogVisible" title="作品审核" width="500px">
  <div v-if="currentWork" class="space-y-3 mb-4">
    <div class="text-base font-medium text-gray-800">
      {{ currentWork.title }}
    </div>
    <div class="text-sm text-gray-500">

```

```

    {{ currentWork.description }}
  </div>
  <div class="text-sm text-gray-400">
    当前状态:
    <el-tag :type="getStatusTagType(currentWork.status)" size="small">
      {{ getStatusText(currentWork.status) }}
    </el-tag>
  </div>
</div>

<el-form :model="reviewForm" label-width="80px">
  <el-form-item label="审核结果">
    <el-radio-group v-model="reviewForm.result">
      <el-radio :label="1">通过并发布</el-radio>
      <el-radio :label="2">驳回</el-radio>
    </el-radio-group>
  </el-form-item>
  <el-form-item label="原因">
    <el-input
      v-model="reviewForm.reason"
      type="textarea"
      :rows="3"
      placeholder="请输入驳回原因（驳回时必填）"
    />
  </el-form-item>
</el-form>

<template #footer>
  <div class="dialog-footer">
    <el-button @click="reviewDialogVisible = false">取消</el-button>
    <el-button type="primary" :loading="reviewLoading" @click="handleSubmitReview">
      确定
    </el-button>
  </div>
</template>

```

```
        </el-button>
      </div>
    </template>
  </el-dialog>
</div>
</template>

<script setup lang="ts">
import { ref, reactive, onMounted, watch } from 'vue'
import { Search, Check, View } from '@element-plus/icons-vue'
import { ElMessage } from 'element-plus'
import dayjs from 'dayjs'
import { getWorks, reviewWork } from '@api/admin'
import type { Work } from '@api/types'

const tableData = ref<Work[]>([])
const loading = ref(false)

const currentPage = ref(1)
const pageSize = ref(10)
const total = ref(0)

const statusFilter = ref<number | ">(2)
const statusOptions = [
  { label: '全部状态', value: " },
  { label: '草稿', value: 1 },
  { label: '已提交', value: 2 },
  { label: '已通过', value: 3 },
  { label: '已驳回', value: 4 },
  { label: '已发布', value: 5 }
]
```

```
const reviewDialogVisible = ref(false)
const reviewLoading = ref(false)
const currentWork = ref<Work | null>(null)
const reviewForm = reactive({
  result: 1 as 1 | 2,
  reason: ""
})

const formatDate = (value: string | null) => {
  if (!value) return ""
  return dayjs(value).format('YYYY-MM-DD HH:mm:ss')
}

const getStatusTagType = (status: number) => {
  if (status === 2) return 'warning'
  if (status === 3 || status === 5) return 'success'
  if (status === 4) return 'danger'
  return 'info'
}

const getStatusText = (status: number) => {
  switch (status) {
    case 1:
      return '草稿'
    case 2:
      return '已提交'
    case 3:
      return '已通过'
    case 4:
      return '已驳回'
    case 5:
      return '已发布'
```

```
    default:
      return ""
    }
  }

const fetchData = async () => {
  loading.value = true
  try {
    const res = await getWorks({
      page: currentPage.value,
      page_size: pageSize.value,
      status: statusFilter.value || undefined
    })
    if (res.success) {
      tableData.value = res.data.list
      total.value = res.data.total
      if (res.data.page) currentPage.value = res.data.page
      if (res.data.page_size) pageSize.value = res.data.page_size
    }
  } catch (error) {
  } finally {
    loading.value = false
  }
}

onMounted(() => {
  fetchData()
})

watch([currentPage, pageSize], () => {
  fetchData()
})
```

```
const handleStatusChange = () => {
  currentPage.value = 1
  fetchData()
}

const openReview = (row: Work) => {
  currentWork.value = row
  reviewForm.result = 1
  reviewForm.reason = ""
  reviewDialogVisible.value = true
}

const handleSubmitReview = async () => {
  if (!currentWork.value) return
  if (reviewForm.result === 2 && !reviewForm.reason.trim()) {
    ElMessage.error('驳回必须填写原因')
    return
  }
  reviewLoading.value = true
  try {
    const res = await reviewWork(currentWork.value.id, {
      result: reviewForm.result,
      reason: reviewForm.result === 2 ? reviewForm.reason : ""
    })
    if (res.success) {
      ElMessage.success(reviewForm.result === 1 ? '审核通过并已发布' : '已驳回该作品')
      reviewDialogVisible.value = false
      fetchData()
    }
  } catch (error) {}
  finally {}
}
```

```
    reviewLoading.value = false
  }
}
</script>
```

```
<style scoped></style>
```

后端代码

```
// 模型管理
```

```
package model
```

```
import (
    "fmt"
    "strings"
    "time"

    "github.com/chosetop/bishe_back/internal/config"
)
```

```
type ModelFile struct {
    ID      int    `json:"id"`
    Name    string `json:"name"`
    URL     string `json:"url"`
    Size    int64  `json:"size"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}
```

```
func CreateModelFile(name, url string, size int64) (int, error) {
    res, err := config.DB.Exec(
        "INSERT INTO model_files(name, url, size) VALUES(?, ?, ?)",
        name, url, size,
```

```
)
if err != nil {
    return 0, err
}
id64, err := res.LastInsertId()
if err != nil {
    return 0, err
}
return int(id64), nil
}

func GetModelFileByID(id int) (*ModelFile, error) {
    row := config.DB.QueryRow(
        "SELECT id, name, url, size, created_at, updated_at FROM model_files WHERE id = ?",
        id,
    )

    var m ModelFile
    err := row.Scan(&m.ID, &m.Name, &m.URL, &m.Size, &m.CreatedAt, &m.UpdatedAt)
    if err != nil {
        return nil, err
    }
    return &m, nil
}

func CountModelFiles(name string) (int, error) {
    var (
        where string
        args []any
    )
    if name != "" {
        where = " WHERE name LIKE ?"
    }
```

```
args = append(args, "%" + name + "%")
}

row := config.DB.QueryRow("SELECT COUNT(1) FROM model_files" + where, args...)
var total int
if err := row.Scan(&total); err != nil {
    return 0, err
}
return total, nil
}

func ListModelFiles(name string, limit, offset int) ([]ModelFile, error) {
    var (
        where string
        args []any
    )
    if name != "" {
        where = " WHERE name LIKE ?"
        args = append(args, "%" + name + "%")
    }
    args = append(args, limit, offset)

    rows, err := config.DB.Query(
        "SELECT id, name, url, size, created_at, updated_at FROM model_files" + where + " ORDER BY id DES
        args...,
    )
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    list := make([]ModelFile, 0, limit)
```

```

for rows.Next() {
    var m ModelFile
    if err := rows.Scan(&m.ID, &m.Name, &m.URL, &m.Size, &m.CreatedAt, &m.UpdatedAt); err != nil {
        return nil, err
    }
    list = append(list, m)
}
if err := rows.Err(); err != nil {
    return nil, err
}
return list, nil
}

func UpdateModelFileFields(id int, name *string, url *string) (int64, error) {
    fields := make([]string, 0, 2)
    args := make([]any, 0, 3)

    if name != nil {
        fields = append(fields, "name = ?")
        args = append(args, *name)
    }
    if url != nil {
        fields = append(fields, "url = ?")
        args = append(args, *url)
    }
    if len(fields) == 0 {
        return 0, nil
    }

    args = append(args, id)
    query := fmt.Sprintf("UPDATE model_files SET %s WHERE id = ?", strings.Join(fields, ", "))
    res, err := config.DB.Exec(query, args...)

```

```
if err != nil {
    return 0, err
}
return res.RowsAffected()
}

func DeleteModelFile(id int) (int64, error) {
    res, err := config.DB.Exec("DELETE FROM model_files WHERE id = ?", id)
    if err != nil {
        return 0, err
    }
    return res.RowsAffected()
}

// 作品审核
package model

import (
    "time"

    "github.com/chosetop/bishe_back/internal/config"
)

type DesignReview struct {
    ID      int64    `json:"id"`
    WorkID  int64    `json:"work_id"`
    VersionID int64    `json:"version_id"`
    AdminID int64    `json:"admin_id"`
    Result  int      `json:"result"`
    Reason  string   `json:"reason"`
    ReviewedAt time.Time `json:"reviewed_at"`
}
```

```

func CreateDesignReview(workID, versionID, adminID int64, result int, reason string) (int64, error) {
    res, err := config.DB.Exec(
        "INSERT INTO design_reviews(work_id, version_id, admin_id, result, reason) VALUES(?, ?, ?, ?, ?)",
        workID,
        versionID,
        adminID,
        result,
        reason,
    )
    if err != nil {
        return 0, err
    }
    return res.LastInsertId()
}

func ListDesignReviewsByWorkID(workID int64) ([]DesignReview, error) {
    rows, err := config.DB.Query(
        "SELECT id, work_id, version_id, admin_id, result, reason, reviewed_at FROM design_reviews WHERE work_id = ?",
        workID,
    )
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    list := make([]DesignReview, 0)
    for rows.Next() {
        var r DesignReview
        if err := rows.Scan(&r.ID, &r.WorkID, &r.VersionID, &r.AdminID, &r.Result, &r.Reason, &r.ReviewedAt); err != nil {
            return nil, err
        }
        list = append(list, r)
    }
}

```

```
list = append(list, r)
}
if err := rows.Err(); err != nil {
    return nil, err
}
return list, nil
}
```